



PROGRAMAÇÃO BACK END I



Professor Esp. Eduardo Herbert Ribeiro Bona
Revisor Técnico: Me. Rafael Alves Florindo

UNICESUMAR

Av. Guedner, 1610 - Jardim Aclimação
Cep 87050-900 - MARINGÁ - PARANÁ
unicesumar.edu.br
44 3027.6360

UNICESUMAR EDUCAÇÃO A DISTÂNCIA

NEAD - Núcleo de Educação a Distância
Bloco 4 - MARINGÁ - PARANÁ
unicesumar.edu.br
0800 600 6360

as imagens utilizadas neste
livro foram obtidas a partir
do site SHUTTERSTOCK.COM

FICHA CATALOGRÁFICA

C397 **CENTRO UNIVERSITÁRIO DE MARINGÁ.** Núcleo de Educação a Distância; **BONA**, Eduardo Herbert Ribeiro.

Programação Back End I. Eduardo Herbert Ribeiro Bona.
Maringá-Pr.: UniCesumar, 2018. Reimpresso em 2021.
184 p.

"Graduação - EaD".

1. Programação. 2. Back end. 4. EaD. I. Título.

ISBN 978-85-459-1157-9

CDD - 22 ed. 005.1
CIP - NBR 12899 - AACR/2



Reitor

Wilson de Matos Silva

Vice-Reitor

Wilson de Matos Silva Filho

Pró-Reitor Executivo de EAD

William Victor Kendrick de Matos Silva

Pró-Reitor de Ensino de EAD

Janes Fidélis Tomelin

Presidente da Mantenedora

Cláudio Ferdinandi

NEAD - Núcleo de Educação a Distância

Diretoria Executiva

Chrystiano Mincoff

James Prestes

Tiago Stachon

Diretoria de Graduação e Pós-graduação

Kátia Coelho

Diretoria de Permanência

Leonardo Spaine

Diretoria de Design Educacional

Débora Leite

Head de Produção de Conteúdos

Celso Luiz Braga de Souza Filho

Head de Curadoria e Inovação

Tania Cristiane Yoshie Fukushima

Gerência de Produção de Conteúdo

Diogo Ribeiro Garcia

Gerência de Projetos Especiais

Daniel Fuverki Hey

Gerência de Processos Acadêmicos

Taessa Penha Shiraishi Vieira

Supervisão de Produção de Conteúdo

Nádila Toledo

Coordenador de Conteúdo

Danillo Xavier Saes

Designer Educacional

Bárbara Neves

Projeto Gráfico

Jaime de Marchi Junior

José Jhonny Coelho

Arte Capa

Arthur Cantareli Silva

Editoração

Ana Eliza Martins

Qualidade Textual

Produção de Materiais

Ficha catalográfica elaborada pelo bibliotecário
João Vivaldo de Souza - CRB-8 - 6828

Impresso por:



Professor
Wilson de Matos Silva
Reitor

Em um mundo global e dinâmico, nós trabalhamos com princípios éticos e profissionalismo, não somente para oferecer uma educação de qualidade, mas, acima de tudo, para gerar uma conversão integral das pessoas ao conhecimento. Baseamo-nos em 4 pilares: intelectual, profissional, emocional e espiritual.

Iniciamos a Unicesumar em 1990, com dois cursos de graduação e 180 alunos. Hoje, temos mais de 100 mil estudantes espalhados em todo o Brasil: nos quatro campi presenciais (Maringá, Curitiba, Ponta Grossa e Londrina) e em mais de 300 polos EAD no país, com dezenas de cursos de graduação e pós-graduação. Produzimos e revisamos 500 livros e distribuímos mais de 500 mil exemplares por ano. Somos reconhecidos pelo MEC como uma instituição de excelência, com IGC 4 em 7 anos consecutivos. Estamos entre os 10 maiores grupos educacionais do Brasil.

A rapidez do mundo moderno exige dos educadores soluções inteligentes para as necessidades de todos. Para continuar relevante, a instituição de educação precisa ter pelo menos três virtudes: inovação, coragem e compromisso com a qualidade. Por isso, desenvolvemos, para os cursos de Engenharia, metodologias ativas, as quais visam reunir o melhor do ensino presencial e a distância.

Tudo isso para honrarmos a nossa missão que é promover a educação de qualidade nas diferentes áreas do conhecimento, formando profissionais cidadãos que contribuam para o desenvolvimento de uma sociedade justa e solidária.

Vamos juntos!



Janes Fidélis Tomelin

Pró-Reitor de Ensino de EaD

Kátia Solange Coelho

Diretoria de Graduação e Pós

Débora do Nascimento Leite

Diretoria de Design Educacional

Leonardo Spaine

Diretoria de Permanência

Seja bem-vindo(a), caro(a) acadêmico(a)! Você está iniciando um processo de transformação, pois quando investimos em nossa formação, seja ela pessoal ou profissional, nos transformamos e, consequentemente, transformamos também a sociedade na qual estamos inseridos. De que forma o fazemos? Criando oportunidades e/ou estabelecendo mudanças capazes de alcançar um nível de desenvolvimento compatível com os desafios que surgem no mundo contemporâneo.

O Centro Universitário Cesumar mediante o Núcleo de Educação a Distância, o(a) acompanhará durante todo este processo, pois conforme Freire (1996): "Os homens se educam juntos, na transformação do mundo".

Os materiais produzidos oferecem linguagem dialógica e encontram-se integrados à proposta pedagógica, contribuindo no processo educacional, complementando sua formação profissional, desenvolvendo competências e habilidades, e aplicando conceitos teóricos em situação de realidade, de maneira a inseri-lo no mercado de trabalho. Ou seja, estes materiais têm como principal objetivo "provocar uma aproximação entre você e o conteúdo", desta forma possibilita o desenvolvimento da autonomia em busca dos conhecimentos necessários para a sua formação pessoal e profissional.

Portanto, nossa distância nesse processo de crescimento e construção do conhecimento deve ser apenas geográfica. Utilize os diversos recursos pedagógicos que o Centro Universitário Cesumar lhe possibilita. Ou seja, acesse regularmente o Studeo, que é o seu Ambiente Virtual de Aprendizagem, interaja nos fóruns e enquetes, assista às aulas ao vivo e participe das discussões. Além disso, lembre-se que existe uma equipe de professores e tutores que se encontra disponível para sanar suas dúvidas e auxiliá-lo(a) em seu processo de aprendizagem, possibilitando-lhe trilhar com tranquilidade e segurança sua trajetória acadêmica.

Professor Esp. Eduardo Herbert Ribeiro Bona

Eduardo Bona é desenvolvedor web profissionalmente desde 2003, tendo tido seus primeiros contatos ainda em 2000. É cofundador da vivaweb.net, empresa criada em 2005 na Incubadora Tecnológica de Maringá. Na Vivaweb é desenvolvedor chefe CTO e gerente de projetos. Também é cofundador da spinoff vivaintra.com e da fintech multibot.com.br.

Desde 2010 atua como professor universitário em cursos de graduação e pós-graduação e como palestrante, já atuou nos principais eventos de tecnologia como *PHP Conference Brasil* e *The Developers Conference TDC* e mantém, anualmente, agenda com participações em eventos universitários em instituições federais, estaduais e privadas pelo Brasil.

É apaixonado por desenvolvimento web. Atua como orientador e pesquisador em assuntos relacionados a *cloud computing*, banco de dados para web, *frameworks php* e *frontend*.

<<http://lattes.cnpq.br/9781942626672838>>.

SEJA BEM-VINDO(A)!

Primeiramente, gostaria de dar as boas-vindas a você e lhe convidar para mais um desafio, agora com a programação para internet.

Todo o conteúdo deste livro foi preparado pensando em você, desenvolvedor, com base em experiências que possuo desde a virada do século, praticamente quando muitos sites eram desenvolvidos apenas com HTML, sem a parte de backend.

Você irá perceber no decorrer de seus estudos que a programação para a internet (web) possui características próprias, mas a principal delas, que é conhecer a linguagem de programação, não é o suficiente para tornar-se um bom programador, que deve conhecer boa parte da arquitetura de *frontend* também.

Pode ter certeza de uma coisa: um programador web é diferenciado dos demais!

Será apresentado, na primeira unidade, uma introdução ao HTML, que será fundamental para continuidade de nossas atividades e seus principais recursos: elementos e atributos por tipo. Você poderá, a partir disto, criar um formulário simples ou um relatório. Na segunda unidade, é apresentada uma introdução à linguagem de programação PHP para que você comece a compreender as características desta linguagem e suas diferenças em relação a qualquer outra que você já conheça. Se ainda não conhece nenhuma, será uma primeira oportunidade de entender os conceitos básicos. Para fechar a primeira parte do livro, na terceira unidade, serão aplicados os primeiros conceitos de PHP como uma linguagem backend para aplicações web, onde você entenderá como as URLs e formulários funcionam, por exemplo.

Já em uma parte final, na quarta unidade, serão catalogadas e demonstradas as principais funções existentes no PHP para manipulação de dados como textos, arquivos, números, entre outros. E, por fim, na última unidade, será desenvolvido ao longo da mesma uma mini aplicação funcional em HTML com PHP construída e explicada passo a passo.

Tudo que será mostrado é parte fundamental para seu currículo como desenvolvedor web e com certeza será o começo para outros estudos e desafios que virão pela frente.

Te convido para participar deste desafio e fico à disposição para te ajudar!

Bons estudos!

■ UNIDADE I

HTML

15	Introdução
16	Introdução a Linguagem de Marcação HTML e ao HTML5
19	Tags, Atributos e Semântica HTML
22	Criação e Organização de Conteúdo
28	Hyperlinks
30	Imagens e Multimídia
33	Formulários Eletrônicos
37	Listas e Tabulação de Dados
41	Considerações Finais
47	Referências
48	Gabarito



■ UNIDADE II

INTRODUÇÃO AO PHP

53	Introdução
54	Introdução e a História do PHP
56	A Função do PHP nas Aplicações Web
59	Sintaxe Básica
66	Tipos de Dados
70	Operadores de Atribuição e Aritméticos
72	Variáveis com Referência, Constantes e Variáveis Variáveis
76	Considerações Finais
83	Referências
84	Gabarito



■ UNIDADE III

PRIMEIROS PASSOS COM PHP

89	Introdução
90	Saídas de Conteúdo HTML com Scripts PHP
93	Estruturas de Controle Condicional
96	Estruturas de Controle de Repetição e Comandos de Desvio
101	Variáveis Pré-Definidas
103	Recebimento de Dados Pela URL Via GET
104	Recebimento de Dados por Formulário Via POST
107	Considerações Finais
113	Referências
114	Gabarito



■ UNIDADE IV

FUNÇÕES NO PHP

119	Introdução
120	Funções Definidas pelo Usuário
124	Funções para Processamento de Texto (Strings)
128	Funções Matemáticas
130	Funções para Manipulação de Arrays
136	Funções para Sistemas de Arquivos
140	Considerações Finais
148	Referências
149	Gabarito

■ UNIDADE V

LABORATÓRIO PHP

153	Introdução
154	Organização e Reutilização de Códigos PHP com Include e Require
159	Validação de Dados e Redirecionamento Entre as Páginas
166	Upload e Manipulação de Arquivos com Formulários
171	Envio de Email
176	Considerações Finais
180	Referências
181	Gabarito
182	Conclusão



HTML

UNIDADE

I

Objetivos de Aprendizagem

- Descrever de maneira didática as características do HTML e das motivações e evoluções com o HTML 5.
- Apresentar a estrutura de uma tag, além de suas características com a presença ou não de atributos e do significado das tags.
- Demonstrar as principais tags para separação de conteúdo e para a criação de conteúdo em bloco ou em linha.
- Explicar o conceito de hyperlinks que é base para o HTML e sua utilização para referências a outros documentos por meio de URLs relativas e absolutas.
- Apresentar os tipos de imagens suportadas por documentos HTML, bem como a tag utilizada para imagens e demonstrar as tags do HTML5 para uso em elementos multimídia como áudio e vídeo.
- Descrever as principais tags utilizadas para criação de formulários e tabulação de dados para criação de relatórios.
- Demonstrar como é realizada a tabulação de dados no HTML para composição de tabelas com cabeçalho, corpo, rodapé, linhas e colunas.

Plano de Estudo

A seguir, apresentam-se os tópicos que você estudará nesta unidade:

- Introdução a linguagem de marcação HTML e ao HTML5
- Tags, atributos e semântica do HTML
- Criação e organização de conteúdo
- Hyperlinks
- Imagens e multimídia
- Formulários eletrônicos
- Listas e Tabulação de dados

INTRODUÇÃO

Caro(a) aluno(a), seja muito bem-vindo(a)! Costumo dizer que o HTML é para as aplicações web algo similar ao oxigênio em relação ao meio ambiente. A internet como conhecemos hoje evoluiu e continua a evoluir juntamente com o HTML. A linguagem de marcação para *hyper-texto* teve sua última grande mudança alguns anos atrás com o HTML5 que fincou definitivamente esta linguagem de marcação como conhecimento fundamental para qualquer desenvolvedor que queira trabalhar com a web.

Entendemos, em lógica de programação, que um programa ou scripts possuem entrada, processamento e saída. A entrada e o resultado (saída) de imensa parte do que é trafegado na rede atualmente, por meio dos navegadores, é no fim das contas HTML. Ele é responsável pela organização e disponibilização de nossos conteúdos.

Desenvolvedores *frontend* possuem habilidades com design, usabilidade, programas gráficos, diagramação de interfaces e muito HTML, CSS e Javascript. A função de um *frontend* é criar e estruturar grande parte de interfaces que terão contato com o usuário final. Um bom desenvolvedor *backend* vai precisar entender de lógica de programação, arquiteturas, linguagens de programação, banco de dados, segurança e entre muitos outros conhecimentos, dentre eles HTML.

O HTML para a disciplina de Backend I é mais focado na conceituação para aplicação dentro sistemas que poderão servir ao desenvolvimento de plataformas, softwares ou até mesmo um painel de controle para sites ou sistemas desenvolvidos sob medida.

Assim como o oxigênio é essencial para nossa qualidade de vida, o HTML também é essencial para as aplicações web. Aprenderemos a utilizar da maneira correta esta linguagem de marcação e produzir códigos mais produtivos, organizados e compatíveis para o nosso meio ambiente, no caso, a internet.

Esta unidade será um desafio e um marco para sua carreira como desenvolvedor web. É fundamental muita aplicação e repetição nas atividades propostas, pois HTML é diferente de tudo que já viu e com certeza irá gostar muito.



INTRODUÇÃO A LINGUAGEM DE MARCAÇÃO HTML E AO HTML5

HTML é a sigla em inglês para *Hyper Text Markup Language*, que podemos traduzir literalmente para o português como linguagem de marcação para hipertexto. É importante que desde já você se acostume com o termo "linguagem de marcação", pois o HTML não deve ser confundido com uma linguagem de programação, erro comum até para quem está começando, mas não será o seu caso, não é?

Não poderei me prender muito com a história do HTML mas é extremamente relevante que você saiba que a web foi inventada em 1992 por Sir Tim Berners-Lee ainda enquanto trabalhava no CERN, Organização Europeia de Pesquisa Nuclear. Neste sentido, Tim Berners-Lee criou principalmente o conceito de web e do funcionamento dos links que permitem atualmente a navegação e a criação desta grande rede que é a internet.

Depois de alguns longos anos para os desenvolvedores, depois de uma guerra entre os navegadores que brigavam para dominar a internet, depois de muito tempo o HTML5 começou a dar seus primeiros passos em 2007 e em 2011, oficialmente, deu seus primeiros sinais para uma adoção entre os navegadores.

Atualmente, o HTML5 já está presente nos navegadores em suas versões mais recentes e também em nossos celulares.

E o que vem a ser um documento HTML?

```
<!DOCTYPE HTML>
<html>
  <head>
    <title>Minha Primeira Página</title>
    <meta charset="UTF-8">
  </head>
  <body>
    <h1>Olá mundo!</h1>
    <p>Este é meu primeiro código com HTML</p>
    <h2>O primeiro de muitos!</h2>
    <p>
      Vamos continuar com a leitura
      para se aprofundar ainda mais...
    </p>
  </body>
</html>
```

Eis um exemplo! E você, o que achou deste código? Normal? Bonito? Estranho? Diferente?

Se está sendo seu primeiro contato com HTML provavelmente o código acima seja totalmente fora do habitual - e é mesmo. Um desenvolvedor web é diferente dos demais porque mesmo conhecendo uma linguagem de programação ainda vai precisar dominar outras linguagens como o HTML, CSS e Javascript.

Antes de mais nada, é importante entender como criar, editar e visualizar os códigos que serão criados em HTML. A vantagem do HTML é que podemos visualizar tudo que estamos criando com os próprios navegadores já instalados em nossas máquinas e para a criação e manutenção de nossos códigos deixarei algumas dicas a seguir.

EDITORES DE TEXTO

Para criar códigos HTML você deverá possuir um editor de textos que trabalhe bem com este tipo de código. Não recomendo o Bloco de Notas, instalado por padrão do Windows, por possuir diversas limitações. No mercado, teremos



dois tipos de propostas para editores de textos: os simples (leves) e os complexos (pesados). Nesta unidade, iremos trabalhar com editores mais simples e a partir da segunda unidade quando estivermos falando da linguagem de programação PHP trataremos de editores mais complexos.



SAIBA MAIS

Dentre as principais possibilidades farei 4 sugestões:

Notepad++: é aquele editor de textos "pai de todos". Muito antigo, mas bem funcional. Gratuito e disponível em: <<https://notepad-plus-plus.org/>>.

Sublime: é aquele editor de textos que está na moda e sendo muito utilizado. Todo mundo ama. Seu uso sem licença fica emitindo uma mensagem na tela periodicamente, mas nada que atrapalhe muito. Disponível para instalação em: <<https://www.sublimetext.com/>>.

Brackets: é aquele que tem uma empresa grande por trás e ao mesmo tempo gratuito. Tem sido minha escolha ultimamente. Disponível para instalação em: <<http://brackets.io>>.

Atom: uma alternativa ao Sublime. Muitos desenvolvedores têm migrado para este editor. Disponível para download em: <<https://atom.io>>.

Fonte: o autor.

Uma grande ajuda para a escolha dos editores de texto é ler o que outros desenvolvedores têm a dizer. Por ser um tema tão debatido e amado não é muito difícil encontrar bons artigos sobre o assunto. Eu destacarei três artigos do portal <tableless.com.br>, que é uma referência em desenvolvimento web desde os primórdios dos blogs e sites técnicos na internet brasileira.

No primeiro artigo "Sublime Text 2 - meu novo editor"^a <[https://tableless.com.br/sublime-text-2-meu-novo-editor/](http://tableless.com.br/sublime-text-2-meu-novo-editor/)>, Diego Eis trata sobre o Sublime. Depois, em "Editor de textos Open Source da Adobe, o Brackets!"^b <[https://tableless.com.br/o-editor-de-textos-open-source-da-adobe-o-brackets/](http://tableless.com.br/o-editor-de-textos-open-source-da-adobe-o-brackets/)>, Felipe Correia aborda o Brackets que, como dito acima, tem sido minha escolha nos últimos três anos. E, por fim, outro artigo tratando somente do Atom^c <[https://tableless.com.br/atom-o-novo-editor-github/](http://tableless.com.br/atom-o-novo-editor-github/)>, escrito por Diego Eis.

REFLITA



A escolha do editor de texto é, muitas vezes, como um relacionamento. Escolha o que for mais produtivo para você ou para sua equipe. Isso será muito importante, não faça a escolha de qualquer jeito.

TAGS, ATRIBUTOS E SEMÂNTICA HTML

Um documento HTML nada mais é do que um conjunto de informações devidamente organizado. Por ser uma linguagem de marcação, tal organização é desenvolvida por meio das tags (etiquetas, em português). Com isso, temos em um documento uma mistura de conteúdo e de marcações através de tags.



Vamos imaginar um dicionário. Ele é composto de palavras. Palavras podem ser classificadas como substantivo ou verbo, por exemplo. Palavras também possuem um significado e podem se encaixar de diversas maneiras na construção de uma frase. Palavras dão sentido aos nossos textos.

Assim como um dicionário, a linguagem HTML se caracteriza por possuir uma especificação bem definida para a utilização de tags para marcação de conteúdo, sendo que cada tag tem uma característica específica. Você não terá possibilidade de criar suas próprias tags, mas não se preocupe, com a quantidade de tags existentes você não terá necessidade de criá-las.

Assim como já demonstrado no primeiro código HTML, você irá notar que as marcações são bem simples de serem feitas. Sua principal preocupação será em construir os documentos de forma correta e conhecer todos os recursos disponíveis do HTML.

TAGS

Analise o trecho de código a seguir:

```
<h1>Seja bem vindo</h1>  
<p>É um <strong>prazer</strong> receber sua visita.</p>
```

Mesmo não conhecendo muito o que cada tag faz, podemos inicialmente afirmar que o código faz uso de três tags: a tag `<h1>`, a tag `<p>` e a tag `<strong>`. Identificamos também que a estrutura apresenta dois níveis principais e que o segundo nível (tag `<p>`) possui um nível interno (subnível). Tudo bem até aqui?

No linguajar mais técnico, níveis são chamados de nós. Quando um nó possui um ou mais nós internos a ele, não se assuste se encontrar algum artigo chamando estes subníveis de filhos. Não só o HTML, mas tudo na área de Tecnologia da Informação por sinal se apropria de termos existentes em outras áreas, mas isso é assunto para um bate-papo nosso em outro momento.

Todas as tags de marcação de conteúdo ou de estruturação deverão ser iniciadas por `<nome_da_tag>` + `seu_conteudo_ou_tags` + `</nome_da_tag>`. Apesar de parecer a mesma coisa, note que o fechamento possui uma barra como sinal de fechamento.

Além disso, ainda iremos estudar, principalmente em formulários tags, que possuem fechamento automático, ou seja, elas não criam esta marcação para o seu fechamento, como poderá ser visto no código abaixo:

```
<form action="enviar.php" method="post" class="formulario">  
  <input type="text" name="nome" />  
  <input type="submit" value="enviar">  
</form>
```

O código mostra a utilização de duas formas da tag `<input>` sendo a primeira finalizada como uma barra antes do fechamento e na linha abaixo sem a barra. No HTML5, as duas formas são aceitas para este tipo de tag, contudo, eu sempre recomendo a primeira opção.

ATRIBUTOS

Toda tag ao ser declarada pode vir acompanhada de atributos e estes podem ser informativos, auxiliares, referenciadores e até mesmo comportamentais. Existem atributos globais que podem ser utilizados em qualquer tag e existem também os específicos para cada tipo de elemento.

Analise este trecho de código:

```
<form action="enviar.php" method="post" class="formulario">
  <input type="text" name="nome" />
  <input type="submit" value="enviar">
</form>
```

Podemos considerar que o código é formado por uma tag `<form>` e dois filhos compostos por tags `<input>` sendo que a tag `<form>` possui três atributos (`action`, `method` e `class`) enquanto as duas tags `<input>` é possível identificar mais três atributos (`type`, `name` e `value`). Atributos, por sua vez, possuem valores e muitos destes atributos possuem valores pré-definidos como é o caso de `method` para `<form>` ou `type` para `<input>` enquanto outros atributos são dinâmicos e informados de acordo com a necessidade do desenvolvedor como `action`, `name` ou `value`.

TIPOS DE TAGS

Nesta unidade, daremos destaque para tags que são usadas para metadados, que são usados para fornecer informação e vincular nosso documento a outros documentos como Javascript e CSS e elementos de conteúdo.



CRIAÇÃO E ORGANIZAÇÃO DE CONTEÚDO

A criação de conteúdo textual com HTML é uma tarefa relativamente simples. São poucas tags e, como as mesmas possuem uma semântica própria, ou seja, possuem significado, você terá poucas dúvidas sobre qual tag utilizar.

TÍTULOS E PARÁGRAFOS

As tags `<h1>`, `<h2>`, `<h3>`, `<h4>`, `<h5>` e `<h6>` representam dentro de um documento HTML os títulos mais importantes para o documento sendo que o elemento `<h1>` seria o equivalente ao mais importante e o `<h6>` ao menos importante dentre os títulos. Indexadores de conteúdo como o Google levam muito a sério a utilização destes recursos em documentos web.

Parágrafos simples, por outro lado, só existem por meio da tag `<p>` sendo que cada parágrafo novo exige o fechamento do texto anterior e abertura de uma nova marcação. Caso necessite, é possível conseguir dentro do próprio parágrafo a quebra de linhas com o uso da tag `<br />`.

```
<!DOCTYPE HTML>
<html>
  <head>
    <title>Minha Primeira Página</title>
    <meta charset="UTF-8">
  </head>
  <body>
    <h1>HTML</h1>
    <p>Estudo sobre HTML</p>
    <h2>Elementos de título</h2>
    <p>Podemos usar do h1 até o h6.</p>
    <p>h1 é o mais importante</p>
    <h2>Elementos de parágrafo</h2>
    <p>Use sempre que precisar criar seu parágrafo</p>
    <p>Se precisar de uma quebra de linha
      use <br /> a tag br</p>
  </body>
</html>
```

Note que, no HTML, a quebra de linha no código por si só não causa nenhum efeito no navegador. Por isso, é importante utilizar as tags de parágrafo para isso.

Além destas tags principais, poderemos também utilizar a tag <pre> para determinar conteúdo pré-formatado, ou seja, conteúdo em que deverá ser mantido uma pré-formatação, a tag <blockquote> para indicar citações e as tags de listas que veremos mais à frente.

SEMÂNTICA INLINE

Determinamos que uma tag inline é uma tag que pode ser usada dentro de outras tags de conteúdo, por exemplo, dentro de um parágrafo. A tag
 que vimos anteriormente é uma destas tags. Dentro de um parágrafo poderemos dar sentido a algumas partes de nossos textos e por isso outras tags foram desenvolvidas. Veja no exemplo, a seguir:

```
<!DOCTYPE HTML>
<html>
  <head>
    <title>Minha Primeira Página</title>
    <meta charset="UTF-8">
  </head>
```



```
<body>
  <h1>HTML</h1>
  <p>Estudo sobre HTML</p>
  <h2>Elementos de conteúdo inline</h2>
  <p>Podemos usar algo mais <strong>importante</strong>.</p>
  <p>Ou também podemos usar <small>HTML</small></p>
</body>
</html>
```

Neste exemplo, a tag `<strong>` e `<small>` se caracterizam por elementos internos de outras tags de conteúdo com valores semânticos. Além destas, o quadro abaixo apresenta mais opções para seus códigos.

Quadro 1- Tags de conteúdo com semântica interna (inline)

ELEMENTO	DESCRIÇÃO
<code><a></code>	Representa um hyperlink, ligando a outro recurso.
<code></code>	Representa a ênfase do conteúdo.
<code></code>	Representa a importância de um pedaço de texto com o forte elemento não altera o sentido da frase.
<code><small></code>	Representa o lado comentário, que é o texto como um aviso legal, um autor que não é essencial para a compreensão do documento.
<code><s></code>	Define texto com efeito sublinhado.
<code><cite></code>	Representa o título de uma obra.
<code><q></code>	Define uma curta citação.
<code><dfn></code>	Representa um termo cuja definição está contida em seu conteúdo ancestral mais próximo.
<code><abbr></code>	Representa uma abreviatura ou acrônimo, eventualmente, com o seu significado.
<code><data></code> HTML5	Associa o seu conteúdo a um equivalente legível por máquina (este elemento está apenas na versão padrão HTML do WHATWG, e não documentado na versão HTML5 da W3C).
<code><time></code> HTML5	Representa um valor de data e hora, eventualmente com um equivalente legível por máquina.
<code><code></code>	Representa uma codificação.
<code><var></code>	Representa uma variável, que pode ser uma expressão matemática, ou código de programação, um identificador representando uma constante, um símbolo identificando uma quantificação física, um parâmetro de função ou um mero placeholder.

ELEMENTO	DESCRIÇÃO
<samp>	Representa uma saída de um programa de computador.
<kbd>	Representa uma entrada do usuário, geralmente pelo teclado, mas não necessariamente, podendo representar outro tipo de entrada como comandos de voz transcritos.
<sub>, <sup>	Representa subscrito e sobrescrito, respectivamente.
<i>	Representa texto em uma voz ou humor alternativo ou em uma qualidade diferente.
	Representa um texto que chama a atenção para fins utilitários. Ele não transmite importância extra e não implica uma voz alternativa.
	Representa um texto sem significado. Ele deve ser usado quando nenhum outro elemento de texto semântico representar um significado adequado.
 	Representa uma quebra de linha.

Fonte: Mozilla Developer Network ([2018], on-line)¹.

SEPARAÇÃO DE CONTEÚDO

Até a chegada do HTML5, a principal forma de realizar a organização e separação de conteúdo era por meio da tag <div> que cria divisões em nossos códigos. Este método é presente ainda na maioria dos portais. Esta organização permite a identificação de maneira mais simples de blocos, bem como a contenção de recursos dentro de um agrupamento.

```
<!DOCTYPE HTML>
<html>
  <head>
    <title>Minha Primeira Página</title>
    <meta charset="UTF-8">
  </head>
  <body>
    <!-- div para contenção total -->
    <div id="principal">
      <-- estruturas de organização -->
      <div id="logomarca">
        ... Imagem da Logomarca ...
      </div>
    </div>
  </body>
</html>
```



```
    </div>
    <div id="conteudo">
        ... Código do Menu Lateral ...
    </div>
</div>
</body>
</html>
```

A utilização de tags de separação de conteúdo não gera nenhuma mudança direta na interface, mas fornece mecanismos para que, através de outras tecnologias como CSS ou Javascript, estas divisões possam receber aplicações de estilos. Veja o mesmo exemplo com uma aplicação simples de CSS.

```
<!DOCTYPE HTML>
<html>
  <head>
    <title>Minha Primeira Página</title>
    <meta charset="UTF-8">
    <style>
      div#logomarca{height: 250px; background-color: #ff0000}
    </style>
  </head>
  <body>
    <!-- div para contenção total -->
    <div id="principal">
      <-- estruturas de organização -->
      <div id="logomarca">
        Imagem da Logomarca
      </div>
      <div id="conteudo">
        Código do Menu Lateral
      </div>
    </div>
  </body>
</html>
```

Note que a aplicação de estilo só foi possível pela identificação de uma referência em nosso documento e só surtiu efeito onde autorizamos a aplicação. Javascript e CSS não são objetos de estudo nesta disciplina mas deixarei ao longo do material boas referências para o aprofundamento de seus estudos.

Além da tag <div> o HTML5 trouxe uma série de elementos com marcação semântica, ou seja, que possuem significado e que serão mostrados no quadro, a seguir:

Quadro 2 - Tags para separação de conteúdo

ELEMENTO	DESCRIÇÃO
<section>	Define a seção do documento.
<nav>	Define uma seção que contém apenas links de navegação.
<article>	Define que pode existir de forma independente do resto do conteúdo. Esta tag poderia ser uma postagem no fórum ou um artigo de revista ou jornal.
<aside>	Define um conteúdo reservado do resto do conteúdo da página. Se for removida, o conteúdo restante ainda faz sentido.
<h1>, <h2>, <h3>, <h4>, <h5>, <h6>	São elementos que representam os seis níveis de títulos de cabeçalhos dos documentos. Um elemento título descreve brevemente o tema da seção.
<hgroup>	Agrupar os elementos de cabeçalho h1, h2, h3, h4, h5 e h6.
<header>	Define o cabeçalho de uma página ou seção. Muitas vezes contém um logotipo, o título do site e um menu de navegação do conteúdo.
<footer>	Define o rodapé de uma página ou seção.
<address>	Define uma seção que contém informações de contato, como endereço e telefone.
<main>	Define o conteúdo principal ou importante no documento. Existe apenas um elemento <main> no documento.

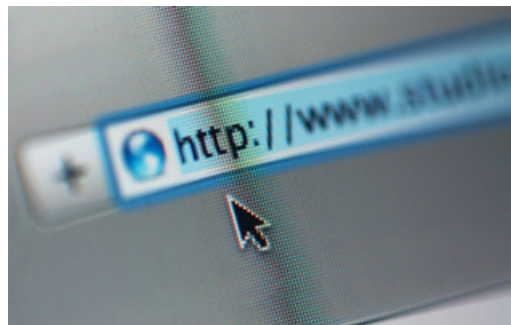
Fonte: Mozilla Developer Network ([2018], on-line)¹.

REFLITA



Neste ponto, sugiro que crie seus primeiros códigos com o que já foi aprendido e, abrindo seu documento HTML no navegador, veja os resultados obtidos. O que acha?

Estamos quase chegando na metade desta unidade. Nos próximos tópicos explicarei mais detalhadamente outros recursos e usarei somente o código HTML interno para os exemplos e você precisa estar habituado já com a criação, edição e visualização de códigos HTML.



HYPERLINKS

A navegação na internet só é possível graças aos *hyperlinks*. Você provavelmente já deve ter copiado uma *URL* de algum site ou notícia e colado em uma rede social ou enviado por e-mail. Enquanto uma

URL é o endereço de localização de um documento HTML, o *hyperlink* é a forma como você cria tal referência dentro de documentos HTML.

Vamos analisar o site da UniCesumar, por meio da URL principal: <<https://www.unicesumar.edu.br>>.



Figura 1 - Página principal do site Unicesumar
Fonte: Unicesumar (2018, on-line)².

Se analisarmos praticamente tudo na página principal se transforma em *hyperlinks* que direcionam para outras páginas por meio de suas URLs. O botão Educação a Distância contido ali entre os três botões principais permite que o usuário ao clicar nele seja imediatamente redirecionado para outra URL <<https://www.unicesumar.edu.br/ead/>>, permitindo a navegação entre as páginas do portal da instituição.

Por isso, dizemos que a base da internet atualmente é obtida com a utilização de *hyperlinks* referenciando URLs internas presentes em nosso próprio site ou para outros domínios.

A tag `<a>` é utilizada para criação de um *hyperlink* e sempre realizará a marcação de algum conteúdo textual podendo ser, por exemplo, um texto ou uma imagem. Analise o exemplo, a seguir:

```
<p>
  Use um hyperlink para acessar o site da
  <a href="https://www.unicesumar.edu.br">Unicesumar</a>
</p>
<p>
  Ou use também para linkar uma imagem como a seguir
  <a href="http://www.uol.com.br">
    
  </a>
</p>
```

Segundo a documentação do portal W3Schools ([2018], on-line)³, "o atributo mais importante de um elemento `<a>` é o atributo *href* que referencia um *hyperlink* como destino" para a navegação após o clique. O atributo *href* espera uma URL podendo a mesma ser relativa ou absoluta.

Tittel e Noble (2014) definem que um link absoluto "usa uma URL completa para conectar os navegadores a uma página web ou um recurso online", enquanto um link relativo "usa um tipo de abreviação para especificar uma URL".

```
<ul>
  <li><a href="contato.html">Página de Contato</a></li>
  <li><a href="ead/contratar.html">Ouvidoria</a></li>
  <li><a href="ead/contratar.html#informatica">Contratar</a></li>
</ul>
```

No exemplo demonstrado acima, todas as URLs se comportam como *hyperlinks* para URLs relativas. É notada uma distinção entre o padrão apresentado na primeira e segunda URL em comparação com a terceira.

No terceiro caso, é permitido que um *hyperlink* faça referência para um ponto de referência dentro do próprio documento principal por meio de uma âncora obtida com o uso do símbolo `#` popularmente conhecido como *hashtag* ou "jogo da velha" para os mais antigos.



Para a criação desta âncora basta haver no documento `ouvidoria.html` um apontamento para esta âncora através do código a seguir:

```
<a name="informatica">Cursos de Informática</a>
```



REFLITA

Se no decorrer de um projeto você tiver dúvidas entre optar pelo formato absoluto ou relativo para suas URLs, use sempre **hyperlinks** relativos enquanto for possível. Assim, se alterar o nome do domínio não será exigida nenhuma manutenção em seu código.



IMAGENS E MULTIMÍDIA

Ainda que a característica principal do hipertexto e seu sucesso possam ser atribuídos aos *hyperlinks* as imagens e outros elementos de multimídia são recursos mais do que essenciais nos projetos na internet atualmente.

IMAGENS

As imagens estão presentes desde as primeiras versões do HTML. É uma tag bem simples de ser usada e sua atenção deve estar em seus atributos e em qual formato de imagem irá escolher.

Analise o código:

```

<p>
  Aluga-se casa na praia com 5 quartos. <br />
  
  
</p>
```

Uma *tag* de imagem é tratada como um elemento *inline*, ou seja, um elemento de texto que pode ser inserido em outras *tags* de organização ou até mesmo *hyperlinks* sem qualquer problema. Ela possui alguns atributos próprios dos quais podemos destacar:

- **src:** origem do arquivo. É o nome do arquivo podendo ser relativo ou absoluto. Principais extensões utilizadas são: jpg, png e gif.
- **width e height:** determinam a largura e a altura, respectivamente. Se não forem informados, serão apresentados em sua resolução original.
- **alt:** utilizado para descrever as devidas legendas de fotos, recurso que além de ajudar na indexação deste conteúdo por buscadores como Google também auxiliam demais leitores de telas para deficientes visuais (acessibilidade).

O Google escreveu um artigo interessante em seu Guia para Desenvolvedores que trata especificamente dos formatos aceitos e qual escolher para seu projeto como veremos no quadro, a seguir:

Quadro 3 - Formatos e suas características

FORMATO	TRANSPARÊNCIA	ANIMAÇÃO	NAVEGADOR
GIF	Sim	Sim	Todos
PNG	Sim	Não	Todos
JPEG	Não	Não	Todos

Fonte: Google Developers (2018, on-line)⁴.

Dica do professor: 90% das imagens de conteúdo são JPG. Elas são mais rápidas de carregar por serem mais leves e são formatos muito mais simples de serem tratados por programas gráficos. Dê preferência por JPG desde que não precise de transparência nem de animação.

ÁUDIOS

"Simples e direto sem necessidade de plugins" é como Silva (2011) define o elemento `<audio>`. Ele reitera esta questão, visto que até o lançamento do HTML multimídias como áudio e vídeo dependiam de terceiros para funcionar. Isso é ótimo para nós, pois será uma preocupação a menos. Segue um exemplo de uso para `<audio>`:

```
<audio controls>
  <source src="musica_1.mp3" type="audio/mp3">
  <p>Mensagem para informar ao usuário que o navegador dele não
suporta estes tipos de arquivos/extensões</p>
</audio>
```

No caso acima, a tag `<audio>` representa o contexto do player enquanto a tag `<source>` determina quem são os elementos de som. É permitido mais de um pois você pode realizar a compatibilidade entre navegadores e permitir várias extensões de áudios. O padrão mais comum é a extensão mp3. Para que o código rode na sua máquina, no entanto, é preciso que o arquivo `musica_1.mp3` exista no mesmo local onde irá criar o arquivo com este HTML.

Segundo o artigo "HTML: Elemento AUDIO", de Elis (2010, on-line)⁵, publicado pelo blog Tableless, um dos pioneiros no Brasil em conteúdo sobre HTML semântico e desenvolvimento web, "o elemento `<audio>` possui diversos atributos como *autoplay*, *loop*, *controls*, *muted*, *preload* e *src*", dentre os quais destaco o *autoplay* para início automático, *loop* para manter em repetição o áudio e *controls* para exibir os botões de controle do recurso.

VÍDEOS

Os vídeos no HTML5 terão estrutura e comportamento similar ao elemento `<audio>` mostrado anteriormente.


```
<video width="800" height="600" controls>
  <source src="video1.mp4" type="video/mp4">
  <source src="video1.ogv" type="video/ogv">
  <p>Mensagem para informar ao usuário que o navegador dele não
  suporta estes tipos de arquivos/extensões</p>
</video>
```

De diferente, você notará somente a utilização dos atributos `height` para determinar altura e `width` para largura. Além disso, extensões e formatos mudam entre áudio e vídeo. Outros atributos mostrados para áudio como *controls*, *muted*, *autoplay* e *loop* também funcionam para vídeo.

Para vídeos, ocorre a mesma situação que citamos para a tag `<audio>`, sendo que os arquivos *sources* representam as opções de arquivos disponíveis. No caso de vídeos é ainda mais importante, pois a compatibilidade pode ser bem comprometida. O padrão mais comum de vídeo é o mp4.



FORMULÁRIOS ELETRÔNICOS

Os formulários serão um dos principais pontos de entrada de dados entre o usuário e uma aplicação web, pois ele será capaz de enviar dados por meio de diversos tipos de campos do navegador (usuário) para o nosso servidor.

De maneira bem simples ainda, um sistema que pretende realizar um cadastro, por exemplo, primeiramente terá um formulário construído em HTML (com

ou sem ajuda de uma linguagem de programação) que submeterá estes dados para um destino que será um script PHP (no nosso caso) que, por sua vez, receberá os arquivos (entrada) e realizará o tratamento (processamento) e emitirá uma mensagem de sucesso e o relatório de cadastros (saída).

No HTML, a tag responsável por um formulário é a tag `<form>` e todos os elementos (campos/inputs) devem estar contidos dentro deste elemento. Esta tag possui pelo menos dois atributos que merecem sua atenção:

- **action:** o valor será o destino do formulário e será utilizado para que todos os dados presentes no formulário sejam enviados para o script de destino. Sem esta propriedade, o próprio documento atual receberá tais informações. Exemplo de uso `action="cadastrar_usuario.php"`
- **method:** essencialmente receberá o valor GET ou POST sendo o método POST o mais recomendável por trafegar as informações de um formulário de maneira transparente ao usuário enquanto o método GET envia todo o formulário por parâmetros da URL e com isso fica muito visível aos usuários.

Um formulário em sua essência é composto de campos. Os rótulos de campos, aqueles textos acima do mesmo para facilitar o entendimento do formulário e ao final um botão de submissão. Este tipo de situação pode variar, mas é sempre muito improvável, por exemplo, ter um formulário apenas com os campos sem rótulos, mas já vi muitas telas de login e senha que são compostos apenas de campos. É uma escolha de cada desenvolvedor, no entanto, vamos manter a ideia do que é normal: campos, rótulos de campos e um botão de submissão.

Um rótulo de campo por sua vez pode ser precedido por um texto simples ou por uma tag específica para rótulos `<label>`. Um exemplo de formulário simples poderá ser obtido por meio do código abaixo:

```
<form action="cadastrar_usuario.php" method="post">
  <label for="nome">Nome: </label>
  <input type="text" name="nome" /><br />
  <label for="email">Email: </label>
  <input type="text" name="email" /><br />
  <input type="submit" value="Enviar" />
</form>
```

Segundo Niederauer (2017, p. 127), os valores para a opção type da tag input são mostradas no quadro a seguir:

Quadro 4 - Principais tipos disponíveis para elemento input

TIPO (TYPE)	DESCRIÇÃO
text	mostra uma caixa de texto de uma linha e permite a entrada de valores numéricos ou alfanuméricos
password	utilizado para a digitação de senhas. São mostrados asteriscos (*) no lugar dos caracteres.
hidden	é um campo escondido. Não aparece na tela.
checkbox	exibe uma caixa de seleção que pode ser marcada ou desmarcada.
radio	são botões de seleção em que o usuário escolhe uma entre várias opções dispostas.
file	permite o envio de arquivos.
submit	botão que aciona o envio dos dados do formulário.
image	tem a mesma função do submit, mas utiliza uma imagem ao invés do botão tradicional
reset	limpa todos os campos do formulário

Fonte: Niederauer (2017, p.127).

Além disso, é importante notar a utilização do atributo *name* que é utilizado como referência para quem irá receber estes dados e também do atributo *value* que atribui um valor para um elemento input. Formulários que realizam uma edição, ou seja, que devem vir preenchidos, usam este atributo para seu funcionamento.

Outros tipos de campos em formulários também podem ser utilizados como apresentado no quadro abaixo:

Quadro 5 - Outros elementos para utilização em um formulário

select	cria uma lista de seleção (combobox) que permite a escolha dentro de uma lista de itens
option	são os elementos internos de um elemento select
textarea	faz a criação de um elemento de texto que permite o envio de mais de uma linha diferentemente do tipo text.
button	tem comportamento similar ao input do tipo submit e é muito comum para uso em aplicações javascript.
label	cria os rótulos para dar anteceder os nossos campos. Não são obrigatórios mas eu recomendo o uso até por uma questão de semântica.

Fonte: o autor.



O código, a seguir, mostra um exemplo de utilização de um formulário com grande variedade de campos. Fique atento a este formulário e suas particularidades. Voltaremos a falar mais detalhadamente de formulários quando estivermos tratando do recebimento deles por scripts PHP.

```
<form action="cadastro.php" method="post">
  <label for="nome">Nome: </label>
  <input type="text" name="nome" /><br />
  <label>Nacionalidade</label>
  <select name="nacionalidade">
    <option value="bra">Brasileiro</option>
    <option value="arg">Argentino</option>
    <option value="out">Outra nacionalidade</option>
  </select><br />
  <label name="estado_civil">Estado Civil</label>
  <input type="radio" name="estado_civil" value="cas" /> Casado
  <input type="radio" name="estado_civil" value="sol" /> Solteiro
  <input type="radio" name="estado_civil" value="out" /> Outro<br
/>
  <label name="mensagem">Mensagem</label>
  <textarea name="mensagem"></textarea><br />
  <label name="aceito">Concorda e aceita os termos de uso?</label>
  <input type="checkbox" name="aceito" /> Sim, aceito os termos de
  uso.<br />
  <input type="submit" value="Enviar" />
</form>
```



SAIBA MAIS

O Mozilla Developer Network (MDN) é uma fonte rica de estudos para tecnologias web como HTML. Um dos guias trata somente sobre formulários e pode ser acessado por meio do link: <<https://developer.mozilla.org/pt-BR/docs/Web/Guide/HTML/Forms>>.

Dentre as principais seções deste guia, destaco um estudo mais focado: o tópico denominado "Meu primeiro formulário HTML". O MDN possui traduções para os principais artigos, inclusive contribuo com algumas traduções).

Outro material muito interessante é do site w3schools.com (em inglês) que permitem testes em tempo real. É possível encontrar exemplos de formulários no endereço: <https://www.w3schools.com/html/html_forms.asp>.

Fonte: o autor.



```
<ul>
  <li>Carnes
    <ul>
      <li>Peito de Frango</li>
      <li>Peito de Peru</li>
    </ul>
  </li>
  <li>Frutas
    <ul>
      <li>Laranja</li>
      <li>Banana</li>
    </ul>
  </li>
</ul>
```

A única diferença para o código de uma lista não ordenada para uma lista ordenada é sua tag principal. Enquanto uma lista não ordenada usa a tag `<ul>`, a lista ordenada usa a tag `<ol>`.

O seu uso é muito pequeno em aplicações, pois geralmente os desenvolvedores fazem uso da tag `<ul>`. Uma lista ordenada faz uso de marcadores de ordem ao invés daqueles marcadores com símbolos que vimos anteriormente como mostra o exemplo abaixo:

```
<ol>
  <li>Brasil</li>
  <li>Argentina</li>
  <li>Alemanha</li>
</ol>
```

LISTAS DESCRITIVAS

As listas descritivas, por sua vez, são utilizadas quando a representação única pelo item não é atendida e se dá o nome de descritiva, pois esta lista é composta sempre por um título e uma descrição. A criação desta lista é feita mediante a utilização da tag `<dl>` seguida por cada `<dt>` e `<dd>` para indicar o título e a descrição de cada item respectivamente.

Como você poderá observar não é feito uso da tag `<li>` e todos os itens necessários para compor a lista são inseridos em sequência dentro da tag principal. Assim como é para a tag `<li>`, as tags `<dt>` e `<dd>` suportam elementos

de conteúdo dentro de sua marcação e diferentemente das tags `<ul>` e `<li>` a tag `<dl>` não possui por padrão marcadores. O código ficaria assim:

```
<p>Dicas caseiras para dor de dente</p>
<dl>
  <dt>Chupar gelo</dt>
  <dd>Tem a intenção de resfriar o local</dd>
  <dt>Enxaguar a boca com água morna e sal</dt>
  <dd>Estimula a limpeza do local e infecções</dd>
  <dt>Passar fio dental</dt>
  <dd>Retirar pedaços de alimentos que podem causar inflamação</dd>
</dl>
```

TABELAS

Um dos recursos mais utilizados dentro de um sistema interno com certeza será a tabela. Relatórios são fornecidos por intermédio da tabulação dos dados entre linhas e colunas e uma tabela no HTML permite exatamente um elemento com estas características.

Toda tabela obrigatoriamente fará uso das tags `<table>`, `<tr>` e `<td>`. A criação de uma tabela no HTML será feita de maneira organizada primeiramente com a declaração de uma linha e em seguida das colunas que irão compor aquela linha como no exemplo abaixo:

```
<table>
  <tr>
    <td>Linha 1 Coluna 1</td>
    <td>Linha 1 Coluna 2</td>
  </tr>
  <tr>
    <td>Linha 2 Coluna 1</td>
    <td>Linha 2 Coluna 2</td>
  </tr>
</table>
```

A tag `<table>` contém a estrutura global de uma tabela enquanto a tag `<tr>` é responsável pela criação de linhas, lembrando que linhas são criadas anteriormente às colunas, que são representadas com a tag `<td>`. Esta é uma representação de uma tabela simples.



Outras tags no HTML darão à tabela ainda mais sentido, que nada mais é do que a semântica que falamos anteriormente e que buscamos em nossos códigos HTML.

```
<table>
  <thead>
    <tr>
      <th>Cabeçalho 1</th>
      <th>Cabeçalho 2</th>
    </tr>
  </thead>
  <tfoot>
    <tr>
      <th>Rodapé 1</th>
      <th>Rodapé 2</th>
    </tr>
  </tfoot>
  <tbody>
    <tr>
      <td>Linha 1 Coluna 1</td>
      <td>Linha 1 Coluna 2</td>
    </tr>
    <tr>
      <td>Linha 2 Coluna 1</td>
      <td>Linha 2 Coluna 2</td>
    </tr>
  </tbody>
</table>
```

As tags <thead>, <tbody> e <tfoot> realizam a segmentação da estrutura de nossas tabelas com cabeçalho, corpo e rodapé. Linhas e colunas ficam inseridas dentro destas seções quando utilizadas. A tag <th> também poderá ser usada para caracterizar uma coluna de título e aparecerá sempre em cabeçalhos. Apesar de não serem obrigatórias, se por esta separação a recomendação é que você escreva <thead>, <tfoot> e <tbody> nesta sequência ou mesmo <thead> e <tbody> que provavelmente é o formato mais encontrado nos códigos na internet.

Além disso, assim como em listas, uma tabela também pode conter outra tabela dentro de uma coluna, bem como conteúdo HTML dos mais diversos tipos como uma lista, por exemplo.

CONSIDERAÇÕES FINAIS

Ainda que HTML não seja uma linguagem de programação ela já é sem dúvida a linguagem mais importante para desenvolvedores web. É a única linguagem que une todos os desenvolvedores para internet, pois ela é a única linguagem de marcação de hipertexto interpretada por navegadores e enquanto se discute muito sobre as linguagens de programação backend como PHP ou JAVA, o HTML continua absoluto.

Com a chegada do HTML5 muitas portas se abrirão e está exigindo atualização constante de todos os profissionais web. Quando mudanças disruptivas como esta chegam no mercado de tecnologia gera uma oportunidade única para desenvolvedores iniciantes se alinharem rapidamente em termos de conhecimento com desenvolvedores mais antigos. Profissionais iniciantes levam até certa vantagem nestes cenários em virtude de não terem vícios ou demandas anteriores a estas mudanças.

Quero dizer com isso que você é um privilegiado e precisa entender desde já que HTML faz parte do conhecimento obrigatório seja você um futuro desenvolvedor backend ou frontend.

Será possível por meio do HTML optar, neste momento, por um aprofundamento em elementos de marcação para sistemas web como formulários, relatórios (tabelas) e listas ou elementos de marcação para diagramação de páginas que irão compor layouts como as tags de conteúdo e separação de conteúdo, como demonstrado inicialmente.

Além disso, se seu foco é em se tornar um desenvolvedor frontend o estudo mais avançado em estilização com CSS e Javascript será exigido e fundamental para seu desenvolvimento profissional.

Como nosso foco será em sistemas será importante um reforço em elementos de formulário e criação de formulários com HTML e se estiver se sentindo bem confortável com HTML recomendo que comece a estudar paralelamente sobre validação com formulários HTML5.

A seguir, trataremos muito de PHP e o HTML será elemento coadjuvante nos exemplos e estudos de caso.

Treine muito HTML!

ATIVIDADES



1. Todo documento HTML, assim como outros documentos XML, é composto por uma estrutura de marcação. Um primeiro elemento é utilizado para realizar a marcação de todo o documento fazendo com que as demais marcações sejam filhas desta marcação principal exigida. Qual é esta tag que realiza a marcação principal em um documento HTML?
 - a) <head>
 - b) <body>
 - c) <html>
 - d) <div>
 - e) <a>
2. Qual elemento do HTML dentro de um formulário é responsável por criar um botão que ao ser clicado realiza o disparo (envio) do formulário em questão?
 - a) <input type="text" />
 - b) <input type="confirm" />
 - c) <input type="go" />
 - d) <input type="submit" />
 - e) <input type="form" />
3. Uma tabela é composta pelo conjunto de diversos elementos que irão compô-la e dentre estes elementos está o elemento necessário para a criação de uma nova linha. Qual opção abaixo representa este elemento?
 - a) <td>
 - b) <th>
 - c) <tbody>
 - d) <tr>
 - e) <thead>
4. Um formulário HTML provavelmente será das estruturas de conteúdo e marcação de elementos a mais diversa que iniciantes terão contato no início de seus estudos. Analise os elementos apresentados abaixo:
 - I. <input type="text">
 - II. <input type="radio">
 - III. <input type="checkbox">
 - IV. <input type="textarea">

ATIVIDADES



Com base nos elementos apresentados abaixo, assinale qual alternativa representa elementos válidos do HTML para a composição de um formulário:

- a) Apenas I, II e III estão corretas.
 - b) Apenas I e IV estão corretas.
 - c) Apenas I, II, III e IV estão corretas.
 - d) Apenas II, III e IV estão corretas.
 - e) Apenas I está correta.
5. Quais atributos presentes no elemento de formulário são responsáveis respectivamente por definir qual será a URL destino do formulário e método após submissão do mesmo?



O HTML5, nova versão para a linguagem de marcação HTML, além de contar com um novo arsenal de tags com muito foco em semântica também deu nova vida aos nossos famosos formulários que até então tinham como limitação principal não ter validações simples, como obrigatoriedade de preenchimento, por exemplo.

Formulários em HTML5 são praticamente outro mundo em comparação com a versão anterior e praticamente tudo que foi atualizado em sua especificação já está sendo aceito pelos navegadores em suas últimas versões. Por isso, já é possível contar com muitos destes recursos e passar a fazer uso deles sem qualquer tipo de receio.

Se antes, só podíamos contar com as tags inputs tradicionais como text, radio, checkbox, file, submit, reset e button agora temos outros atributos com maior precisão semântica como:

- tel: usado para inserir número de telefone suportando máscaras através dos patterns e validação.
- search: usado especificamente para buscas.
- url: para digitação de URLs absolutas ou relativas com verificação do padrão.
- e-mail: para digitação de e-mails com verificação do padrão.
- datetime, date e time: utilizados para acrescentar recurso de calendário para escolha de dia, mês e ano. No caso de datetime e time com abrindo a possibilidade para escolher horário.
- week e month: para digitação da semana do ano ou seleção do mês.
- number: específico para números.
- range: abrindo a possibilidade de um tipo de campo que faz um intervalo para escolher, por exemplo, entre 10 e 100.
- color: utilizado para componentes onde o usuário precisa escolher uma cor e para isso será disponibilizado uma paleta de cores amigável.

Como se não bastasse isso, novos atributos foram criados e destaco aqui alguns deles como:

- autofocus: faz com que o cursor do mouse vá para o campo em questão no carregamento da página.
- placeholder: insere um texto anterior já no campo de texto, geralmente como auxílio. Quando o usuário clica no campo o texto se apaga. Este recurso é muito comum.
- required: valida se o campo foi preenchido e do contrário trava o campo.
- pattern: especifica um padrão de preenchimento para o campo como máscara de telefone ou cep, por exemplo.





Um excelente artigo sobre formulários em HTML5 foi desenvolvido pelo site [html5rocks](https://www.html5rocks.com/pt/tutorials/forms/html5forms/) e pode ser consultado no link:

[<https://www.html5rocks.com/pt/tutorials/forms/html5forms/>](https://www.html5rocks.com/pt/tutorials/forms/html5forms/), pois meus destaques são apenas alguns dos novos recursos do HTML5 que deu poderes extraordinários para esta linguagem de marcação e os navegadores.

Aprofundando-se no assunto verá as APIs que o HTML5 apresentou como webstorage que permite que sejam salvos valores no navegador ou recursos para aplicações offline que salvam informações e possibilitam que aplicações sejam desenvolvidas para trabalhar mesmo sem internet com conteúdo local da máquina.

HTML5 é vida! Estude muito!

Fonte: o autor.





LIVRO

HTML5: A linguagem de marcação que revolucionou a web

Maurício Samy Silva

Editora: Novatec

Sinopse: HTML5 é a linguagem de marcação que amplia de forma surpreendente as funcionalidades da HTML, alterando de maneira significativa como você desenvolve para a web. Trata-se da mais extensa especificação para a HTML focada em criar funcionalidades para desenvolvimento não só de sites, mas também de aplicações de internet rica (RIA). Aborda-se as funcionalidades da HTML5, de forma clara, em linguagem didática.

Comentário: Excelente livro! Desenvolvido pelo principal autor nacional nos assuntos referentes a web como HTML e CSS.



NA WEB

A próxima web (original: The next web) - em inglês com legendas em português.

Tim Berners-Lee apresenta ainda em 2009 o que seria o futuro da web e a essência de sua abordagem já está vinculada ao HTML5. Um vídeo curto e inspirador de, nada mais nada menos, que o criador da web.

Web: <https://www.ted.com/talks/tim_berniers_lee_on_the_next_web>.

Uma carta magna para a web (original: A Magna Carta for the web) - em inglês com legendas em português.

Ao completar 25 anos, a web ganha de seu criador uma palestra rápida de quase 7 minutos sobre seu compromisso com o mundo.

Web: <https://www.ted.com/talks/tim_berniers_lee_a_magna_carta_for_the_web>.

A metáfora da eletricidade para a web do futuro (original: The electricity metaphor for the web's future) - em inglês com legendas em português.

Em 2003, Jeff Bezos, criador da Amazon e homem mais rico do mundo em 2017, fez uma palestra comparando a indústria web com a eletricidade. Se você ainda tem dúvidas sobre o mercado web e o futuro de programadores web ao assistir esta palestra provavelmente não terá mais dúvidas.

Web: <https://www.ted.com/talks/jeff_bezos_on_the_next_web_innovation>.

REFERÊNCIAS

GOOGLE DEVELOPERS. **Web Fundamentals: Otimização de Imagens**. Disponível em: <<https://developers.google.com/web/fundamentals/performance/optimizing-content-efficiency/image-optimization?hl=pt>>. Acesso em: 01 dez. 2017.

MDN MOZILLA DEVELOPER NETWORK. **Elementos HTML**. Disponível em: <<https://developer.mozilla.org/pt-BR/docs/Web/HTML/Element>>. Acesso em: 01 dez. 2017

MDN MOZILLA DEVELOPER NETWORK. **Guia de Formulários HTML**. <<https://developer.mozilla.org/pt-BR/docs/Web/Guide/HTML/Forms>>. Acesso em: 01 dez. 2017.

NIEDERAUER, Juliano. **Desenvolvimento Websites com PHP**. São Paulo: Novatec, 2017.

SILVA, Maurício Samy. **Desenvolva aplicações web profissionais com uso dos poderosos recursos de estilização das CSS3**. São Paulo: Novatec, 2011.

UNICESUMAR. **Site**. Disponível em: <<https://www.unicesumar.edu.br>>. Acesso em: 01 dez. 2017.

TABLELESS. **HTML5: Elemento Audio**. <<https://tableless.com.br/elemento-tag-audio>>. Acesso em: 01 dez. 2017.

TITTEL, ED e NOBLE, Jeff. **HTML, XHTML e CSS Para Leigos**. São Paulo. Alta Books, 2014.

W3SCHOOLS. **HTML Elemento <a>**. Disponível em: <https://www.w3schools.com/tags/tag_a.asp>. Acesso em: 01 dez. 2017.

W3SCHOOLS. **HTML Forms**. Disponível em: <https://www.w3schools.com/html/html_forms.asp>. Acesso em: 01 dez. 2017.

REFERÊNCIAS ON-LINE

¹ Em: <<https://developer.mozilla.org/pt-BR/docs/Web/HTML/Element>>. Acesso em: 05 fev. 2018.

² Em: <<https://www.unicesumar.edu.br>>. Acesso em: 05 fev. 2018.

³ Em: <https://www.w3schools.com/tags/tag_a.asp>. Acesso em: 05 fev. 2018.

⁴ Em: <<https://developers.google.com/web/fundamentals/performance/optimizing-content-efficiency/image-optimization?hl=pt>>. Acesso em: 05 fev. 2018.

⁵ Em: <<https://tableless.com.br/elemento-tag-audio/>>. Acesso em: 05 fev. 2018.



1.

HTML

Todo documento HTML tem por seu principal elemento, o chamado "pai de todos", uma tag <html> como no exemplo abaixo que demonstra um documento básico.

```
<html>
  <head>
    <!-- metadados do cabeçalho -->
  </head>
  <body>
    <h1>Página de Teste</h1>
  </body>
</html>
```

2.

```
<input type="submit" />
```

Todo formulário para ser disparado dentro de um documento HTML (sem qualquer tipo de intervenção) precisa de um botão para submissão. O elemento <input> dispõe de um tipo específico, o submit que assume esta responsabilidade. Além dele também pode ser utilizado o tipo (image) que representaria uma imagem como botão mas é pouco utilizado atualmente.

3.

```
<tr>
```

Uma tabela tem como seu elemento principal de agregação o <table> que, por sua vez, concentrará todos os demais elementos de uma tabela. Dentro de uma tabela podemos ter um cabeçalho, corpo e rodapé caso haja esta demanda para segregação de conteúdo e seria usado neste caso <thead>, <tbody> e <tfoot>. No entanto, os elementos principais de uma tabela são as famosas linhas e colunas sendo as linhas representadas por <tr> e colunas por <td> sendo que a variação de <td> para <th> compõe uma coluna do tipo importante, de marcação de cabeçalho (header), que poderia ser usado para os títulos de um cabeçalho, por exemplo. Colunas <td> sempre precisam estar dentro de linhas <tr>.

4.

```
<input type="text" />
<input type="radio" />
<input type="checkbox" />
```

Alternativa I, II e III estão corretas.

Das opções apresentadas apenas <input type="text" /> não corresponde a um elemento <input> válido pois um <textarea> na verdade é um elemento mesmo com tag própria e não um atributo do elemento <input>. Com o uso de <textarea> é possível criar um elemento que suporte o recebimento de textos com quebra de linhas e tudo mais.



5.

Atributo action e atributo method

Um elemento <form> pode ter atributos para destinar o envio de seu formulário através do atributo action que deverá receber uma URL absoluta ou relativa (se não informada será enviado para a página atual) e o atributo method que informa o método pretendido, podendo ser GET ou POST (se não informado será interpretado como GET).



INTRODUÇÃO AO PHP

UNIDADE



Objetivos de Aprendizagem

- Conhecer a história sobre a criação e motivação do PHP.
- Apresentar os usos de aplicações em PHP.
- Descrever a sintaxe básica e característica da linguagem.
- Demonstrar os tipos de dados suportados pelo PHP.
- Apontar os operadores de atribuição e aritméticos presentes na linguagem.
- Demonstrar utilização de por referência, constantes e variáveis dinâmicas.

Plano de Estudo

A seguir, apresentam-se os tópicos que você estudará nesta unidade:

- Introdução e a história do PHP
- A função do PHP nas aplicações web
- Sintaxe Básica
- Tipos de Dados
- Operadores de atribuição e aritméticos
- Variáveis com referência, constantes e variáveis variáveis

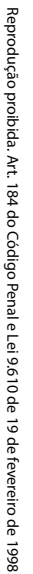
INTRODUÇÃO

Quando o assunto for linguagem de programação para desenvolvimento web, não tenha dúvidas de que o PHP estará dentre as opções listadas. Enquanto no ambiente corporativo e desktop há uma predominância de aplicações em linguagens como JAVA, C#, .NET ou Delph é possível destacar três motivos para o uso do PHP: compatibilidade, flexibilidade e curva de aprendizagem.

O PHP é compatível praticamente com todos os servidores e plataformas, ou seja, roda tanto em servidores Linux quanto Windows e funciona tanto em um servidor Apache como IIS (Microsoft). Além disso, não é uma linguagem conhecida como muito rígida e com muitas regras e permitirá que você programe de maneira estruturada ou orientada a objetos. E, por fim, e não menos importante, PHP é fácil de aprender possuindo uma das curvas de aprendizagem mais simples para já sair desenvolvendo.

Esta linguagem já passou por muita evolução ao longo dos anos e atualmente, em sua versão 7, está ainda melhor. Após as primeiras versões, e principalmente a partir da versão 5, o PHP tem implementado as melhores práticas de outras linguagens e a orientação a objetos foi profundamente absorvida. O *Wordpress*, ferramenta de gerenciamento de conteúdo mais utilizada no mundo, usa PHP. O Facebook, maior rede social do mundo com mais de 2 bilhões de usuários, durante anos usava somente PHP e ao longo do tempo inclusive ajudou o PHP a melhorar sua arquitetura e recursos.

O PHP trabalha em conjunto com HTML e por isso seus estudos até agora serão extremamente relevantes. Apresentarei a você, neste primeiro momento, o fundamental sobre a linguagem e criaremos nossos primeiros *scripts*. Leia e revise este conteúdo quantas vezes forem necessários, pois a prática o levará ao conhecimento e, em muitos momentos, a prática nada mais é do que a repetição.



Segundo a definição da documentação oficial, "PHP significa Processador Hipertexto (*Hypertext Preprocessor*), é uma linguagem de programação de ampla utilização e pode ser mesclada dentro do HTML" (MANUAL DO PHP, [2018], on-line)¹. PHP foi criado em 1994 por Rasmus Lerdorf e, inicialmente, era simplesmente um conjunto de binários *Common Gateway Interface* (CGI) escrito em linguagem de programação C e até hoje a influência da linguagem C é sentida nas sintaxes do PHP.

INTRODUÇÃO AO PHP

Ainda segundo a documentação oficial do PHP, entre os primeiros anos do PHP, Rasmus tomou conta de diversas idas e vindas, inclusive do nome da linguagem e o PHP como conhecemos hoje começou a acontecer a partir da versão 3.0, escrita/revisada em 1997 por Andi Gutmans e Zeev Suraski, de Tel Aviv (Israel), e, em 1998, com ajuda de novos desenvolvedores foi lançado para o público. Das versões anteriores, que chegavam a estar presentes em 1% dos domínios, com a versão 3.0 passou a ser de 10%. Depois disso, o PHP foi crescendo e incorporando cada vez mais colaboradores a sua equipe de desenvolvimento e passou pelo lançamento da versão 4 em maio de 2000. PHP 5 veio em 2004, o PHP 7 em 2015 e o PHP 7.3, a versão mais recente, em 2018. A versão 7.3, lançada em dezembro de 2018, traz correções de bugs, novas implementações e também tornará alguns métodos obsoletos.

O PHP lembra muito a linguagem de programação C e atualmente é suportada na maior parte dos servidores web. Segundo o site W3techs([2018], on-line)², no ano de 2017, o PHP estava sendo utilizado por 82.40% dos sites analisados enquanto ASP.NET (segundo colocado) era utilizado por 14.30%. Neste cenário de análise, é possível concluir que um site pode possuir várias tecnologias, porém quando o assunto é desenvolvimento web o PHP é uma linguagem muito comum e largamente utilizada.

Outra vantagem do uso desta linguagem de programação é que ela é "*open source*" (código aberto) e conta com o apoio de uma comunidade imensa para suporte, manutenção e evolução desta tecnologia. No final do ano de 2015 foi lançada a versão 7 do PHP com mudanças mínimas de sintaxe, mas com muita melhoria na velocidade chegando a ficar em média duas vezes mais rápido ou até nove vezes mais rápido como é apresentado no artigo "PHP 7: até 9 vezes mais rápido que o PHP 5.6", do portal IMasters (2015, on-line)³. A versão 7.3, lançada em dezembro de 2018, traz correções de bugs, novas implementações e também tornará alguns métodos obsoletos.

O PHP pode ser usado como *script* do lado do servidor, *script* linha de comando (sistema operacional) ou até mesmo como aplicação *desktop* mediante o projeto PHP-GTK. Contudo, vamos nos concentrar no PHP como *script* do lado do servidor, que é acessado por meio de um navegador e renderiza páginas e sistemas web. A seguir, o quadro 1 mostra as principais vantagens do PHP:

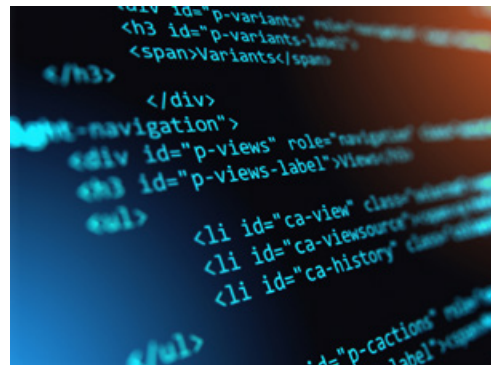
Quadro 1 - Vantagens do PHP

compatibilidade com sistemas operacionais	Pode ser utilizado na maioria dos sistemas operacionais incluindo Linux, variantes do UNIX, Microsoft Windows, Mac OS X, RISC OS e outros.
compatibilidade com servidores	Além disso, o PHP é suportado pelos principais servidores web incluindo Apache e o IIS. Essa liberdade poucas linguagens de programação te darão.
flexibilidade	you pode optar por utilizar linguagem estrutural ou programação orientada a objetos (OOP) ou as duas simultaneamente;
multiusuário	o PHP faz mais do que gerar HTML dinâmico como gerar imagens, arquivos PDF ou documentos de textos;
multi banco de dados	suporte a uma ampla variedade de banco de dados de maneira simples usando extensões específicas ou uma camada de abstração como o PDO ou ODBC;
suporte a protocolos	suporte para comunicação com outros serviços utilizando protocolos como LDAP, IMAP, POP3, HTTP.

Fonte: PHP INTRODUÇÃO ([2018], on-line)⁴.

A FUNÇÃO DO PHP NAS APLICAÇÕES WEB

Mesmo sendo suportado tanto em versão linha de comando e ter um suporte para versão Desktop, o PHP é mundialmente conhecido como linguagem de programação para web através de servidores web. E o que é possível fazer com PHP? Uma infinidade de projetos. O quadro 2 apresenta a seguir os principais tipos de aplicações com PHP:



Quadro 2 - Uso de PHP nas aplicações web

Sites Institucionais	São páginas de caráter organizacional onde informações são apresentadas ao público. Neste tipo de projeto, o conteúdo geralmente pode ser estático ou também pode ser cadastrado através de um ambiente administrativo (painel de controle). No caso de conteúdo estático, toda mudança dependerá de alguém com entendimento técnico ir até o documento e alterá-lo enquanto que um site com ambiente administrativo fica acessível a qualquer administrador gerenciar as informações.
Hotsites	São páginas que possuem ou a característica de temporalidade ou de imutabilidade. Uma promoção de fim de ano, por exemplo, poderia ser entendido como um hotsite pois tem um ciclo de vida bem definido, curto e sofre poucas alterações. Geralmente este tipo de projeto possui muitos conteúdos estáticos e tem projetos muito mais simples.
Portais de Conteúdo	São páginas com ambiente administrativo para gerenciamento de conteúdo mais aprofundado e com recursos mais avançados para gerenciamento de conteúdo. Além disso, possuem a característica de ter muito conteúdo cadastrado e páginas com muito mais informação. Este tipo de projeto traz para o desenvolver novas responsabilidades além da gestão mais complexa do conteúdo. Questões como performance, banco de dados, consultas mais rápidas e SEO farão parte do dia a dia do projeto.
Sistemas Internos (Intranet)	Toda organização provavelmente tem uma demanda interna que muitas vezes é delegado a alguém com conhecimento de programação desenvolver. Com a chegada da web, muitas destas empresas passaram a demandar que este tipo de sistema funcionasse pelo navegador. A complexidade deste tipo de projeto varia de acordo com a empresa e com a equipe de TI, mas PHP em vários casos tem se mostrado uma boa solução para desenvolvimento e manutenção.
Comércio Eletrônico (e-commerce)	Um site de comércio eletrônico já traz consigo uma responsabilidade enorme implícita: o fato de movimentar dinheiro e dados como cartão de crédito. É fortemente recomendado usar plataformas e tecnologias já consolidadas no mercado para esta tramitação e o PHP tem se mostrado a principal escolha para plataformas que serão customizadas. Grandes sistemas como Magento e osCommerce usam PHP para disponibilizar suas plataformas.

Aplicações / Plataformas / Startups	Plataformas como o Facebook que começam pequenas (protótipos) e que vão crescendo geralmente demandam um desenvolvimento mais ágil e mudanças frequentes até sua maturidade. O PHP também se apresenta como uma boa escolha para este tipo de demanda por possuir uma velocidade considerável em sua curva de aprendizagem e também por permitir o desenvolvimento mais rápido de muitas destas demandas.
APIs	Em uma tradução para o português, significa Interface de Programação de Aplicativos. Na prática, são padrões de arquitetura que permitem criar uma interface de comunicação entre uma aplicação final e seu sistema ou banco de dados através do consumo de informações. Usam protocolo HTTP para consumir estes dados e atualmente são muito comuns para consumir e enviar dados nos aplicativos móveis.

Fonte: o autor.

Esta lista de possibilidades são apenas as principais com que você terá contato inicialmente com PHP provavelmente.

Se você é um desenvolver da nova geração que já se preocupa com aplicativos móveis também pode usar o PHP para desenvolver suas APIs e fazer com que seu aplicativo móvel apenas consuma esta API. Todo aplicativo móvel consome algum tipo de API que é uma arquitetura padrão de comunicação entre aplicações e que permite que aplicativos em diferentes linguagens e plataformas consigam se conectar e realizar o consumo de dados.



SAIBA MAIS

Atualmente existe até uma tendência mais conhecida, como API First, que prega que todo projeto deve ser iniciado por uma API primeiramente e depois por suas aplicações web, móvel e administrativa, por exemplo. Se você quer saber mais sobre o assunto um artigo do portal iMasters, disponível em: <<https://imasters.com.br/apis/o-que-e-uma-estrategia-api-first/?trace=1519021197&source=single>>, aborda muito bem o assunto.

Fonte: o autor.



SINTAXE BÁSICA

Por se tratar de uma linguagem interpretada, o PHP vai exigir sempre que um *software* (interpretador) faça uma conversão dos *scripts* em respostas. Quando falamos de PHP para a Web, estaremos nos referindo ao interpretador do PHP para servidores web como Apache, Lighttpd ou IIS e neste caso as respostas serão sempre através do protocolo HTTP podendo vir misturadas com HTML.

Diferentemente do HTML, que podemos abrir em qualquer lugar, um *script* PHP precisará ser acessado de maneira lógica, e não física, através de um domínio virtual que seu servidor web se encarregará de instalar.

Como não temos a disposição para inicialmente desenvolver nossos scripts um servidor pronto, precisaremos instalar um software que faça isso por nós em nossas próprias máquinas de desenvolvimento. Este pacote que instala o Apache + PHP + Mysql que é mais tradicional entre desenvolvedores tem o nome de XAMPP e está disponível para download gratuitamente em seu site (PELOI, 2015, on-line).

Com isso, por padrão, o servidor Apache (pacote XAMPP) acessará de forma lógica seus *scripts* através do domínio virtual `<http://localhost/sua_pasta/`

seu_nome_de_arquivo> e com isso seus arquivos físicos deverão ser alocados na pasta C:\xampp\htdocs.

Entendendo melhor: para dar manutenção nos arquivos ou criar novos arquivos você precisará abrir a pasta lógica e interagir com os documentos criados mas para ver seu projeto rodando dentro de um navegador você não poderá abrir o arquivo com um duplo clique, ou seja, precisará digitar <http://localhost/sua_pasta/seu_arquivo.php> para que o arquivo seja interpretado e exibido para seu navegador corretamente.

Para fazer o primeiro acesso a um *script* em sua máquina será preciso seguir os seguintes passos:

- Ter um servidor web instalado em sua máquina de desenvolvimento. Baixe o projeto XAMPP disponível em <http://www.apachefriends.org/>, instale-o e inicie os serviços de Apache por meio de sua central de serviços XAMPP.
- Após a instalação seu XAMPP Control Panel deve ficar com os serviços inicializados ("startados"), conforme imagem a seguir:

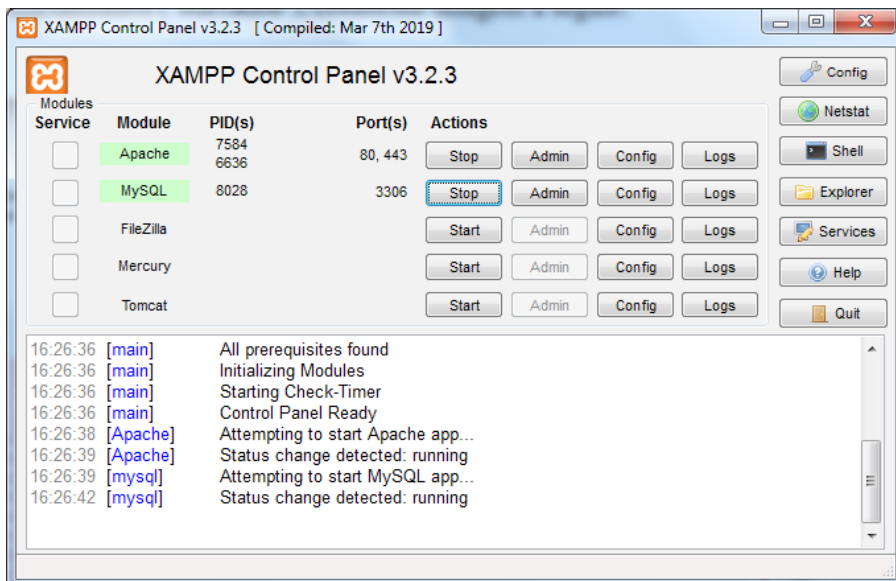


Figura 1 - Painel de controle XAMPP

Fonte: Apache Friends (2018, on-line)⁵.

- certifique-se que o servidor instalado está funcionando acessando pelo seu navegador <http://localhost> onde deverá ser exibida uma mensagem do projeto dando as boas-vindas para você, conforme imagem apresentada a seguir:

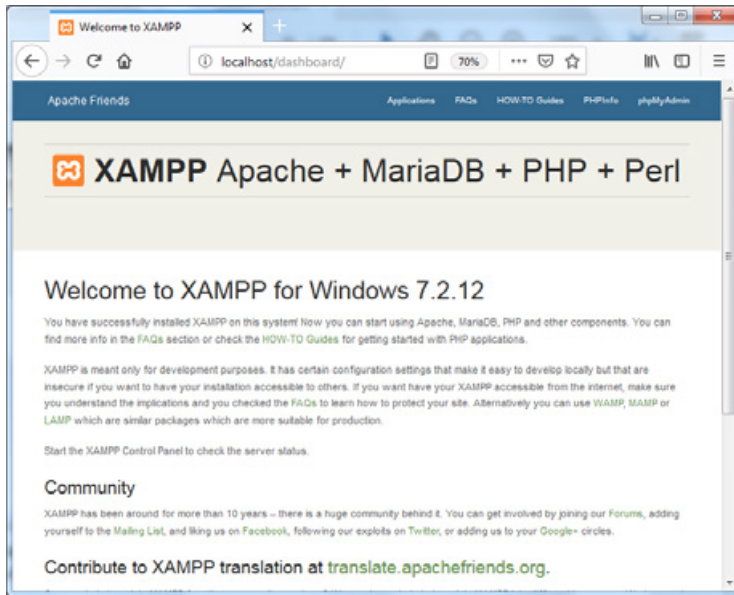


Figura 2 - Bem vindo ao XAMPP

fonte: Apache Friends (2018, on-line)⁵.

- Com o servidor web instalado, você deverá criar um arquivo chamado exemplo1.php, conforme código fornecido no exemplo a seguir.
- Este arquivo precisará estar localizado dentro da pasta htdocs dentro de C:\XAMPP\htdocs (no caso de máquinas Windows e instalação padrão).
- Por fim, abra seu navegador e digite <http://localhost/exemplo1.php>.

```
arquivo: exemplo1.php
<?php
    echo "Olá mundo!";
?>
```

Apareceu a mensagem "Olá mundo!" para você no navegador?

Se sim, continue. Se não, refaça o passo a passo informado acima até garantir o funcionamento.



REFLITA

Ter um ambiente de servidor instalado na própria máquina pode parecer inicialmente muito confuso, mas é muito comum que desenvolvedores saibam instalar servidores na própria máquina.

Você irá notar que a extensão padrão interpretada pelos servidores para scripts PHP segue a lógica e é .php. Sendo assim, todo arquivo com a extensão .php será interpretado pelo servidor como script e se assim não for, o navegador somente abrirá seu arquivo em seu formato texto.

Como um script PHP é sempre interpretado e pode se misturar ao código HTML, você sempre irá precisar separar os scripts PHP através de uma tag `<?php`. Deste modo, tudo que estiver dentro desta tag até o seu fechamento será interpretado pelo servidor como código e sintaxe PHP.

Diferentemente do que se espera um código PHP não é composto somente de códigos PHP, pois ele pode possuir somente códigos PHP, PHP com HTML ou somente HTML. Isso ocorre porque o PHP realiza saídas de código (*output*) que podem ser desde textos simples (puros) como HTML e também podem estar embutidos dentro de estruturas HTML existentes, esta última, sendo a forma mais comum.

Todo arquivo PHP deverá possuir a tag de início `<?php` e, nos casos em que o script só executar códigos PHP, o fechamento desta tag não é recomendado e o script funcionará normalmente com ou sem a tag `?>`. A recomendação serve somente por questões de performance e prevenção de erros. Quando o código PHP for misturado ao HTML será necessário a abertura e o fechamento da tag. Mostrarei em breve um script funcionando sem o fechamento da tag.

A separação de instruções é feita por meio do símbolo ponto-vírgula ";". Com isso, podemos utilizar a quebra de linhas sem muitos problemas, pois o PHP sempre esperará por esta separação para entender a instrução.

Arquivo: exemplo2.php

```
<html>
  <head>
    <title>Exemplo 2</title>
  </head>
  <body>
    <!-- Comentários no HTML -->

    <?php
      // Comentários no PHP
      echo "Olá Mundo!";

      /* Sendo possível fazer um comentário em
      mais de uma linha até achar
      o fechamento novamente */

      echo "<br />";
      echo "Misturando HTML e PHP";
    ?>

  </body>
</html>
```

O código acima apresenta alguns conceitos importantes sobre a estrutura do PHP. A primeira é sobre os comentários que são bem diferentes de comentários em HTML e outro detalhe é que o PHP suporta dois tipos de comentários. Sobre comentários, o PHP fornece um primeiro tipo de comentário de única linha que sempre é precedido pela marcação //. Além disso, comentários em PHP que exijam mais de uma linha podem ser criados a partir da instrução /* e valerão até a marcação final com */ também.

REFLITA



Comentários ajudam muito a documentar nossos códigos e auxiliam o entendimento de outros desenvolvedores sobre seus objetivos com relação à estrutura do código. Bons programadores documentam muito bem.

Outro ponto importante sobre o script `exemplo2.php` é que além de você poder separar os códigos HTML do código PHP, você também pode imprimir códigos HTML de dentro dos scripts PHP por meio da função `echo` ou `print`. Você irá se habituar a isso conforme for praticando mais e ir avançando no conteúdo deste material.

Antes de finalizarmos esta primeira parte mais básica sobre PHP, gostaria de te apresentar algo que será comum no decorrer dos seus primeiros scripts em PHP: erros. E, sinceramente falando, eles sempre estarão presentes em nosso dia a dia e com o passar do tempo. O que precisamos fazer é diminuir a incidência e compreender o mais rapidamente quando eles ocorrerem tornando, assim, a correção menos custosa. Veja a seguir:

```
Arquivo: exemplo3.php
<?php
    echo "Olá Mundo!";

    /* Vou causar um erro em breve... */

    echo <br />;
    echo "Misturando HTML e PHP";
?>
```

Note que não separei o código PHP dentro de aspas e que deve sempre ser feito para saída de dados literais (desconhecidos do PHP). Com isso, o seguinte erro foi impresso em nossa tela: "Parse Error: syntax error, unexpected '<' [...] line 3".

No PHP, erros terão o tipo *Notice*, *Warning*, *Parse Error* e *Fatal Error*, geralmente. Destes, o tipo de erro *Notice* indica uma inconsistência encontrada contornada (ou ignorada sem erro) mais como um nível de aviso, pois o PHP ainda vai tentar contornar. Erros do tipo *Warning* significam que o código executado não funcionou realmente e *Parse Error* já é um tipo de erro presente na sintaxe do seu código e é o mais comum para quem está iniciando.

Se *Parse Error* acontecer, é porque seu código precisa ser revisado pois você esqueceu de alguma coisa e posso garantir que no começo será um ponto-vírgula, aspas ou o sinal de dólar (\$) que declara as variáveis. *Fatal Error* é o que de pior poderia ocorrer em um código PHP, pois geralmente está ligado a algum erro de sintaxe e uma vez ocorrido o interpretador imediatamente interrompe sua execução.

O PHP avisa o tipo de erro, faz uma breve descrição do erro e ainda aponta a linha. Isso deverá ser o suficiente para que, com o passar do tempo, você resolva estes problemas o mais rápido possível e que evite cometê-los novamente.

No entanto, antes de começarmos a produzir nossos primeiros códigos e termos nossos primeiros erros, vamos entender também sobre as variáveis e os tipos de dados no PHP.

É importante que você configure seu ambiente de desenvolvimento para sempre mostrar erros e faça o processo inverso no servidor de produção não exibindo nenhum erro ao usuário final, pois mostrar erros em ambiente de produção é considerado falha grave de segurança. Para isso, existem duas configurações no PHP como *display_errors* que podem estar ligadas ou desligadas com valores *on* ou *off* e *error_reporting* com o valor *E_ALL* para sempre reportar qualquer erro.

Estas alterações podem ser realizadas a nível de ambiente dentro do arquivo de configuração do PHP ou por meio de uma função no código. O arquivo de configuração se chama *php.ini* e a depender do servidor instalado ele pode ser localizado em pastas como *bin*, *etc* ou *conf*. No caso do código, é mais simples, basta inserir a seguinte linha de código em seu script que os erros deixarão de serem exibidos na tela, veja:

```
<?php
    ini_set('display_errors', 0);
    echo "Olá Mundo!";

    /* Vou causar um erro em breve... */

    echo <br />;
    echo "Misturando HTML e PHP";
?>
```



TIPOS DE DADOS

Lisboa (2013, p.70) defende que "o PHP tem uma tipagem fraca, mas que isso não seja interpretado como uma deficiência e sim como uma estratégia sendo possível dados do tipo escalares, compostos e especiais". Isso ocorre, pois no ato da atribuição de valores às variáveis, o contexto dela definirá sua tipagem. No PHP, você não define uma variável para receber valores do tipo inteiro, ou seja, você somente declara a variável e, ao atribuir um valor a ela, o próprio PHP verificará qual o tipo que será definido. Isso é tipagem dinâmica. No quadro, a seguir, veremos os tipos escalares:

Quadro 3 - Tipos de variáveis escalares

Boolean (booleano)	Pode ser true (verdadeiro) ou false (falso). Outro valor se declarado num contexto boolean será interpretado e convertido para true ou false. 0 por exemplo é false e qualquer número maior que 0 é convertido para true.
Integer (inteiro)	São números inteiros podendo ser inclusive negativos. 10 ou -58 são exemplos de integers.
Float (flutuante)	São números que podem conter frações e o separador destes valores no PHP é representado pelo símbolo ponto ".". 12.59 ou 1.298 são exemplos de floats.

String (texto)	Conceitualmente nada mais é do que um array (vetor) de bytes composto de caracteres. O PHP dispõe de muitas funções para se trabalhar com strings que veremos a seguir. "Eduardo Bona" ou "Programação PHP" são exemplos de strings.
Object (objetos)	Um objeto é uma entidade com um determinado comportamento definido por seus métodos (ações) e propriedades (dados). Para criar um objeto deve-se utilizar o operador new.

Fonte: Dall'Oglio (2015, p. 31).

Dos tipos compostos, o que merece um destaque especial é o Array. Os arrays são mapas ordenados. São as matrizes e vetores que conhecemos de outras linguagens. Um mapa relaciona valores a uma chave.

Já os tipos especiais não serão abordados neste contexto, mas você poderá observar estes tipos para aplicações bem mais específicas através da documentação oficial PHP^b.

As variáveis no PHP, então, servem para armazenar dados de acordo com os tipos permitidos pela linguagem. Cada variável está associada a uma posição em memória e, inicialmente, você não precisa se preocupar com esta questão de memória e entrar em um processo de economizar ao máximo variáveis. Não fique com medo, pode declarar e usar suas variáveis.

No PHP, a declaração de variáveis deve sempre ser iniciado pelo símbolo de dólar (\$) e, em seguida, de um sinal de underline (_) ou de letras. Não será aceito após o sinal (\$) um dígito numérico, por exemplo, conforme mostra a Tabela 1.

Tabela 1 - Declaração de variáveis válidas e inválidas

VARIÁVEL	VÁLIDA OU INVÁLIDA?	MOTIVO
\$nome	Válida	\$ + letra + (qualquer letra, número e _)
\$_nome	Válida	\$ + _ + (qualquer letra, número e _)
\$meu_nome	Válida	\$ + letra + (qualquer letra, número e _)
\$meunome2017	Válida	\$ + letra + (qualquer letra, número e _)
\$2017meunome	Inválida	Obrigatoriamente após a declaração com o sinal \$ é necessário a utilização de uma letra ou sinal underline.

VARIÁVEL	VÁLIDA OU INVÁLIDA?	MOTIVO
<code>\$_2017meu_nome</code>	Válida	<code>\$ + _ +</code> (qualquer letra, número e <code>_</code>)
<code>\$com_acentuação</code>	Válida	Nota: por incrível que pareça, funciona! Porém, minha dica é que você evite. A chance de confusão é muito alta. Programe sem acentuação na sintaxe da linguagem. Mas é válido.

Fonte: o autor.

Com base nos exemplos, vejamos a seguir de forma prática como podemos atribuir valores de maneira direta às variáveis:

Arquivo: exemplo4.php

```
<?php
    $nome = 'Eduardo';
    $sobrenome = "Bona";
    $ano_nascimento = 1986;
    $altura = 1.71;
    $hobbies = array('futebol', 'bateria', 'csgo', 'war');

    var_dump($nome);
    var_dump($sobrenome);
    var_dump($ano_nascimento);
    var_dump($altura);
    var_dump($hobbies);
?>
```

Para o exemplo4.php, o retorno será:

```
string(7) "Eduardo"
string(4) "Bona"
int(1986)
float(1.71)
array(4) {
    [0]=> string(7) "futebol"
    [1]=> string(7) "bateria"
    [2]=> string(4) "csgo"
    [3]=> string(3) "war"
}
```

Usamos a função **var_dump** sempre que for necessário verificarmos o valor e o tipo atribuído a uma variável durante o desenvolvimento de código. É uma informação muito importante para o desenvolvedor, mas para o usuário a saída sempre deve ser tratada com o uso dos comandos de saída `echo` ou `print`.

Outro detalhe ainda relevante neste exemplo é que para strings, quando for necessário declararmos valores faremos uso de aspas simples ou duplas. A diferença fundamental entre elas que é que as aspas duplas são interpretadas pelo PHP enquanto que, com o uso de aspas simples, o PHP não realiza a interpretação do código interno, caso haja. Veja o exemplo a seguir:

Arquivo: exemplo5.php

```
<?php
    $nome = "Eduardo";
    $sobrenome = "Bona";
    var_dump("Meu nome é $nome $sobrenome.");
    var_dump('Meu nome é $nome $sobrenome');
?>
```

E o resultado será:

```
string(25) "Meu nome é Eduardo Bona."
string(28) "Meu nome é $nome $sobrenome"
```

Pratique alguns exemplos de declaração e impressão de valores em tela para se ambientar com colocar nomes nas variáveis e, principalmente, em criar variáveis com nomes válidos e coerentes. Evite a criação de variáveis do tipo \$a ou \$a1 que não representam nada e crie variáveis como \$nome ou \$nome_completo ou \$total.

Além disso, no PHP é possível utilizar o que chamamos de conversão de tipos e, deste modo, forçar que o PHP ao término do processamento de uma informação force a tipagem de um tipo. Veja o exemplo:

Arquivo: exemplo.php

```
<?php
    $situacao = 1;
    var_dump($situacao);
    $situacao = (bool) 1;
    var_dump($situacao);
?>
```

O resultado com este código será que no primeiro momento \$situacao será int (inteiro) e depois passará a ser booleano (bool) em virtude da conversão. Este recurso é muito utilizado para receber valores através da URL e garantir que mesmo que um texto seja informado ainda assim com a conversão se tornará um inteiro.

As conversões podem ser obtidas por meio sempre de (bool) para booleanos, (int) para inteiros, (string) para cadeia de caracteres, (float) para decimais e (array) para vetores ou matrizes.

OPERADORES DE ATRIBUIÇÃO E ARITMÉTICOS



O primeiro operador que já notamos nos exemplos acima é o operador de atribuição. Ele é responsável por atribuir valor a uma variável. A atribuição é feita pelo sinal de igual (=) seguido pelo valor desejado.

Existem os operadores aritméticos que serão capazes de realizar algumas operações matemáticas básicas no PHP, conforme mostra a Tabela a seguir:

Tabela 2 - Operadores Aritméticos no PHP

OPERADOR	DESCRIÇÃO	EXEMPLO
+	Adição	<code>\$a + \$b</code>
-	Subtração	<code>\$a - \$b</code>
*	Multiplicação	<code>\$a * \$b</code>
/	Divisão	<code>\$a / \$b</code>
%	Resto da Divisão	<code>\$a % \$b</code>

Fonte: o autor.

O exemplo 6 demonstra uma combinação de todos estes operadores para uma melhor compreensão sua de como eles podem ser utilizados.

Arquivo: `exemplo6.php`

```
<?php
    $nota1 = 9;
    $nota2 = 8.5;
    $nota3 = 8.5;
    $nota4 = 9;

    $total = $nota1 + $nota2 + $nota3 + $nota4;
    $media = $total / 4;

    $media_aprovacao = 6;
    $total_aprovacao = 6 * 4;

    echo "Você obteve nota total de $total sendo necessário
    $total_aprovacao para sua aprovação. <br />";
    echo "Você obteve uma média final de $media";
?>
```

Agora que você já conhece estes operadores básicos, vou poder lhe contar também que existem operadores ainda mais interessantes que permitem uma atribuição juntamente com operadores aritméticos sendo eles os próprios operadores citados para operações simples como adição, subtração, multiplicação e divisão com o operador de atribuição. O exemplo 7 é bem didático, mas em algum momento, de algum projeto, você se deparará com a necessidade de realizar uma operação assim e com isso economizar código:

Arquivo: exemplo7.php

```
<?php
    // podem ser usados operadores += -= *= e /=

    $nota_total = 9;
    $nota_total += 8.5; // recebendo o próprio valor + 8.5
    echo $nota_total;
?>
```

E a saída será:

17.5

E assim como o PHP veio da linguagem C, também temos os operadores de incremento e decremento, presença constante em nossos laços de repetição.

Arquivo: exemplo8.php

```
<?php
    $contador = 5;
    $contador++; // valor passará a ser 6;
    $contador--; // valor voltará a ser 5;
    echo $contador;

    // você poderá usar $variavel++ ou ++$variavel
    // você poderá usar $variavel-- ou --$variavel
    // a diferença é na ordem de execução
?>
```

E a saída será:

5

VARIÁVEIS COM REFERÊNCIA, CONSTANTES E VARIÁVEIS VARIÁVEIS

No PHP, quando fazemos uma instrução do tipo "\$a = \$b;" estamos atribuindo o valor da variável \$b para a variável \$a, mas no desenrolar do código cada uma passa a ter seu ciclo de vida de forma diferente. Contudo, existe uma forma de proporcionar a uma variável não só receber o valor de uma variável, mas sim receber a própria variável passando a ser ela.

Variáveis referenciadas, ao invés de receber o valor como se fosse uma cópia, recebem a localização da variável presente em memória e, em seguida, passa a responder também por ela. As variáveis variáveis não são um recurso muito comum de se encontrar, mas você verá muitas variáveis por referência em funções do PHP, como no caso de muitas funções de variáveis do tipo array.

Abaixo, segue dois exemplos para sua análise:

Arquivo: exemplo9_1.php




```
<?php
    $valor1 = 10;
    $valor2 = 20;
    $valor1 += 10;
    $valor3 = $valor1;
    $valor3 += 50;
    var_dump($valor1);
    var_dump($valor2);
    var_dump($valor3);
?>
```

Neste primeiro exemplo, o resultado será (int) 20 (int) 20 e (int) 70. É o comportamento normal de variáveis que, quando são atribuídas, copiam somente os valores. Agora, vamos ao segundo exemplo:

Arquivo: exemplo9_2.php

```
<?php
    $valor1 = 10;
    $valor2 = 20;
    $valor1 += 10;
    $valor3 = &$valor1;
    $valor3 += 50;
    var_dump($valor1);
    var_dump($valor2);
    var_dump($valor3);
?>
```

E agora com uma suave alteração na atribuição da variável \$valor3, o resultado final já será (int) 70 (int) 20 (int) 70, pois ao receber a referência da variável \$valor1, qualquer alteração será feita em ambos os casos.

Já uma constante, segundo a Documentação Oficial PHP ([2018], on-line)³, "uma constante é um identificador (nome) para um valor único" e que "como o nome sugere, esse valor não pode mudar durante a execução do script" e salienta que "as constantes são case-sensitive por padrão e que "por convenção, identificadores de constantes são sempre em maiúsculas".

Como uma constante possui um escopo global durante o tempo de execução de um script e não pode ser alterada, geralmente utilizamos ela para armazenar alguns dados como dados de acesso ao banco de dados (servidor, usuário e senha) ou outros valores próprios da aplicação. Quem define o uso é o desenvolvedor, mas as constantes são muito comuns.

Constantes têm sua declaração e forma de acesso bem diferentes de uma variável. A declaração é feita por meio de uma função e seu acesso feito diretamente pelo nome da constante no código. Lembre-se que a convenção é de que uma constante sempre possui letras maiúsculas e de que uma constante é sensível a alteração de maiúsculas e minúsculas.

Veja no exemplo, a seguir, como são utilizadas constantes em um código PHP:

Arquivo: exemplo10.php

```
<?php
define("VERSAO_APLICACAO", "1.10");
define("MOSTRAR_ERROS", "1");

echo VERSAO_APLICACAO;
echo " ";
echo MOSTRAR_ERROS;
?>
```

A saída deste código será 1.10 1

A documentação oficial do PHP aponta algumas diferenças entre constantes e variáveis conforme pode ser observado no quadro, a seguir:

Quadro 4 - Diferenças entre uma constante e variável

DIFERENÇA	DESCRIÇÃO
Declaração	Constantes não possuem um sinal de cifrão (\$) antes delas.
Acessibilidade	Constantes podem ser definidas e acessadas de qualquer lugar sem que a regras de escopo de variáveis sejam aplicadas.
Imutabilidade	Constantes não podem ser redefinidas ou eliminadas depois de criadas.
Constantes não recebem todos os tipos de dados	Constantes só podem ter valores escalares. A partir do PHP 5.6 é possível definir um array constante com a instrução <i>const</i> e a partir do PHP 7 arrays constantes também podem ser definidas utilizando a <i>define()</i> . Pode-se utilizar arrays em expressões escalares constantes (por exemplo <i>const FOO = array(1,2,3)[0];</i>), cujo resultado final deve ser um valor de um tipo permitido.

Fonte: Documentação Oficial PHP ([2018], on-line)³.

Por fim, uma variável variável é um recurso muito incomum em projetos, mas que é bom que você saiba que ele existe, pois um dia quem sabe poderá precisar dele.

Este recurso é, geralmente, utilizado quando se faz necessário possuir nome de variáveis dinâmicas, ou seja, eu poder acessar o valor de uma variável através de um valor variável. Parece bem complicado, não é? Acredito que com um exemplo fique mais fácil.

Arquivo: exemplo9.php

```
<?php
    $nome = "Eduardo";
    $sobrenome = "Bona";
    $variavel = "nome";
    echo $$variavel;
?>
```

E o resultado será: Eduardo.

SAIBA MAIS



Em projetos mais avançados, o uso de valores variáveis na composição do nome de uma variável não é tão comum, é mais comum ver chamada de funções ou métodos (quando orientado a objetos o projeto), por meio de um valor de variável. Quando chegarmos em funções, lembre-se deste recurso.

Fonte: o autor.

Seu primeiro contato com PHP passará obrigatoriamente pela declaração de variáveis e ainda que você não se depare em um primeiro momento com variáveis por referência, constantes ou até mesmo variáveis variáveis (utilização rara) é muito importante que você as conheça desde o início.

Também é importante ter conhecimento que a passagem de variáveis por referência é um recurso muito presente em funções nativas do PHP, principalmente de manipulação de arrays, e que constantes são muito utilizadas em projetos bem estruturados como forma de definir valores que não sofrerão alterações durante a execução da requisição e que precisam estar acessíveis dentro de todos os scripts PHP.

CONSIDERAÇÕES FINAIS

O PHP é uma das linguagens mais utilizadas para programar para web. Ele é preferido por muitos desenvolvedores e empresas web por se tratar de uma linguagem de programação com uma curva de aprendizado simples. Isso significa que, para poder colocar a mão na massa e entregar alguma aplicação PHP, é uma das linguagens que menos demanda tempo e/ou experiência.

Isso é verdade, não é uma ilusão. Porém, é preciso atentar-se que a programação para web demanda mais habilidades do que programar para desktop ou até mesmo mobile. Quando afirmo isso, é no sentido que a programação para web envolve conhecimento em HTML (que vimos anteriormente), CSS e Javascript na camada de apresentação e aí PHP, banco de dados, servidor Apache e protocolo HTTP na parte de backend, o que é muita informação para desenvolvedores de primeira viagem.

Começar a conhecer o PHP pela sua sintaxe e seus comandos básicos é um estudo que deve ser repetido até que estes conceitos consigam fixar muito bem em seus códigos. Só com esta segurança básica será possível você aplicar na prática sem maiores complicações tais conhecimentos como declaração de variáveis, terminação de comandos, tipos de variáveis, operadores e muito mais.

Boa parte de seus futuros códigos se concentrarão no que aprendeu até aqui. Se tiver outros códigos desenvolvidos, já em outras linguagens, um bom exercício neste momento seria praticar o desenvolvimento das mesmas atividades em PHP. Seria um excelente começo poder comparar o PHP com outras funcionalidades de outras linguagens.

De todo modo, conhecer pelo menos duas ou três linguagens de programação é importantíssimo. Se você me pedisse uma sugestão de três linguagens de programação para se aprender eu iria sugerir PHP, Javascript e Python. São as três linguagens que mais são usadas para novos desenvolvedores.

Bons estudos!

ATIVIDADES



1. O PHP é uma linguagem interpretada e por isso usaremos quase sempre um servidor web para realizar esta interpretação. Até por isso, quando você salvar um arquivo dentro de uma pasta deste servidor deverá realizar o acesso por meio de um navegador.

Supondo que esteja usando XAMPP e a pasta do servidor seja a padrão C:\XAMPP\htdocs e que você criou uma pasta chamada projeto1 e dentro desta pasta um arquivo chamado teste.php como qual url você deverá digitar na URL para obter o resultado?

- a) teste.php
 - b) projeto1/teste.php
 - c) http://localhost/projeto1/teste.php
 - d) http://localhost/teste.php
 - e) Nenhuma das opções anteriores.
2. As variáveis do tipo Array no PHP são muito comuns e podem ser declaradas e manipuladas de diversas maneiras no PHP. Analise os códigos abaixo:

I. `$marcas = array("BMW", "Mercedes", "Ferrari");`

II. `$marcas = array(0=>"BMW", 1=>"Mercedes", 2=>"Ferrari");`

III. `$marcas = array(1=>"BMW", 3=>"Mercedes", 4=>"Ferrari");`

IV. `$marcas = ["BMW", "Mercedes", "Ferrari"];`

Assinale a alternativa correta com relação a sintaxe do código e seu funcionamento (considere o PHP a partir da versão 7).

- a) Apenas as alternativas I e II funcionariam no PHP.
 - b) Apenas as alternativas I e III funcionariam no PHP.
 - c) Apenas as alternativas I, II e III funcionariam no PHP.
 - d) Apenas as alternativas II e III funcionariam no PHP.
 - e) Todas as alternativas funcionariam no PHP.
3. As variáveis no PHP diferentemente de outras linguagens de programação não exigem anteriormente a declaração de seu tipo, e deste modo, sempre que atribuído um valor o PHP automaticamente atribuirá um tipo a variável. Analise as afirmações a seguir:

ATIVIDADES



- I. \$idade = 14; declara uma variável \$idade do tipo inteiro.
- II. \$salario = 12.550,25 declara uma variável do tipo float.
- III. \$nome = "Eduardo Bona"; declara uma variável do tipo string.
- IV. \$estados = 1,2,3 declara uma variável do tipo array.

Analise as afirmações abaixo:

- a) Apenas as alternativas I e II funcionariam no PHP.
- b) Apenas as alternativas I e III funcionariam no PHP.
- c) Apenas as alternativas I, II e III funcionariam no PHP.
- d) Apenas as alternativas II e III funcionariam no PHP.
- e) Todas as alternativas funcionariam no PHP.

4. Analise o código abaixo:

```
<?php
$numero1 = 12.50;
$numero2 = "12";
$numero3 = 2.00;

$resultado = $numero1 + $numero2 * $numero3;
?>
```

Ao término do script, qual será o resultado que a variável \$resultado receberá?

5. As variáveis por referência são muito utilizadas em algumas funções nativas do PHP, além de funções que futuramente você possa vir a criar dentro de suas aplicações. Analise o código abaixo:

```
<?php
$numero1 = 10;
$numero2 = $numero1;
$numero3 = &$numero1;
$numero3++;

$resultado = $numero1 + $numero2 + $numero3;
?>
```

Ao término do script, qual será o resultado que a variável \$resultado receberá?



Você já parou para pensar como funciona um projeto na internet e quais são os passos mais comuns para a sua construção?

Vamos supor uma coisa bem simples: **um site na internet com a página principal e um formulário de contato.**

O primeiro passo de tudo em um projeto é a construção de alguma documentação para conseguir transcrever de forma visual ou textual qual a ideia do cliente sobre o projeto que ele demanda. Geralmente, vamos nos utilizar de briefings como documentação principal em um site. Na internet, existem vários modelos de briefings, caso queira passar a usar.

Em seguida, iremos compor a identidade visual deste projeto, escolher cores, desenvolver logotipo e elementos principais do site para apresentar ao cliente e definir a melhor proposta de identidade visual. Este processo de prova consiste em uma pesquisa geralmente em empresas do porte do cliente, concorrentes ou projetos semelhantes podendo ser inclusive fora do Brasil. O cliente tendo esta definição passaremos para a produção. Até esta etapa nada mais fizemos do que arquitetos e engenheiros fazem em novos projetos.

Com tudo definido, um web designer ou designer poderá definir como será o layout da página, ou seja, sua estrutura, posicionamento e tipografia dos títulos e elementos textuais. Normalmente, um web designer faz uso nesta etapa de programas como Adobe Photoshop ou Illustrator e costuma entregar uma imagem mesmo no formato JPEG ou o arquivo original do projeto para que seja dado prosseguimento ao restante do projeto. Um novo aceite do cliente, é obtido.

Até aqui, destaquei três etapas e nenhuma ainda envolvendo código. Projetos de sites, por exemplo, até este momento tem aproximadamente 40% de sua execução já concluída e os próximos passos serão a diagramação e a programação backend.

A diagramação consiste em analisar todo o layout desenvolvido pelo webdesigner e transcrever para HTML e CSS (se necessário também Javascript para componentes de interação com usuário) ainda tudo de forma estática (fixa). Será possível com o HTML desenvolvido ver no navegador como o trabalho ficou e já realizar os ajustes necessários caso sejam identificados. Este trabalho em equipes maiores, geralmente, é feito por um desenvolvedor front-end que possui muitas habilidades com a parte de programação visual se é que assim podemos chamar.

Com tudo pronto, chegou a vez do programador! Colocar tudo em um servidor web local primeiro, transformar os arquivos HTML e PHP e fazer o recebimento do formulário de contato e envio para o e-mail do cliente. O programador ainda faz os testes nas páginas e verifica se tudo está em perfeito estado. Organiza os documentos em pedaços menores para facilitar a manutenção e em seguida coloca finalmente em produção, ou seja, em um servidor final no domínio do cliente, que poderia ser por exemplo: `<http://www.meusite.com.br>`.





Parece que o programador backend fez pouco para um projeto assim, mas a responsabilidade e as tarefas aumentam sempre conforme o tamanho do projeto. O trabalho do desenvolvedor backend é muito maior em sistemas para internet e aplicações, representando mais de 60% do projeto geralmente.

Em empresas menores, no entanto, o desenvolvedor faz o trabalho do webdesigner, do frontend e do backend e isso, junto, é praticamente 90% do projeto, quando não é o próprio desenvolvedor que faz tudo, até o contato com o cliente, o que é fácil de detectar que é um problema muito relevante manter todo o processo focado apenas em um profissional.

Em empresas maiores em que o projeto é bem dividido entre as funções, qualidades são muito exigidas de cada papel, enquanto em empresas menores o que mais é exigido de seus profissionais é o conhecimento amplo das tecnologias para poder exercer razoavelmente todos os papéis.

Depois de pronto, fica apenas para a empresa ou o profissional o suporte ao cliente por atendimentos e consequentes manutenções no projeto. Um projeto ainda que simples passou pela mão de diversas pessoas ou papéis e prova que não existe projeto simples para a internet. Este processo varia de empresa para empresa, mas o mais praticado dentro de empresas e agências para websites é baseado neste fluxo. Você pode melhorá-lo ou adaptá-lo a sua realidade.

Agora chegou sua vez! A partir de agora, você poderá definir o seu processo e até mesmo começar alguns projetos pessoais com base em um processo bem definido para entender o quanto antes como funciona um projeto web do começo ao fim. Da venda com o cliente até a entrega e suporte.

A emoção de colocar os primeiros sites (e projetos) no ar não tem preço.

Fonte: o autor.





LIVRO

Desenvolvendo Websites com PHP

Juliano Niederauer

Editora: Novatec

Sinopse: 'Desenvolvendo Websites com PHP' apresenta técnicas de programação fundamentais para o desenvolvimento de sites dinâmicos e interativos. Você aprenderá a desenvolver sites com uma linguagem utilizada em mais de 10 milhões de sites no mundo inteiro. O livro abrange desde noções básicas de programação até a criação e manutenção de bancos de dados, mostrando como são feitas inclusões, exclusões, alterações e consultas a tabelas de uma base de dados. O autor apresenta diversos exemplos de programas para facilitar a compreensão da linguagem.

Comentário: Dos livros escritos para PHP para programadores iniciantes este é muito importante. Traz conceitos básicos e é construído a partir de algumas demandas bem procuradas para quem está em seus primeiros códigos com PHP. Conceitualmente é muito relevante.





NA WEB

Manual do PHP (Oficial)

No site do PHP, a documentação oficial deve ser sua primeira referência em buscas sobre funções do PHP e códigos. Nada estará mais atualizado e em conformidade com o PHP do que a própria documentação. Além disso, lendo todo o Manual do PHP você provavelmente conhecerá funções e recursos do PHP que nunca imaginou existir. Vale a pena a leitura e a releitura anualmente.

Acesse: <http://php.net/manual/pt_BR>.

No site da w3schools é possível conhecer de uma forma diferente da documentação oficial do PHP boa parte de suas funções e aplicações básicas. Uma vantagem que eu vejo neste site é que ele é referência de várias tecnologias não só do PHP e, desta forma, você consegue aprender de uma forma padronizada diversos conhecimentos. Além disso, é possível fazer um Quiz (um questionário estilo prova objetiva) para validar seus conhecimentos.

Acesse: <<https://www.w3schools.com/php>>.

REFERÊNCIAS

APACHE FRIENDS. **Xampp**. Disponível em: <<https://www.apachefriends.org/index.html>>. Acesso em: 18 março de 2019.

DALL'OGGIO, Pablo. **PHP Programando com Orientação a Objetos**. São Paulo: Novatec, 2007.

IMASTERS. **PHP 7**: até nove vezes mais rápido que PHP 5.6.

Disponível em: <<https://imasters.com.br/back-end/php-7-ate-9-vezes-mais-rapido-que-o-php-5-6/>>. Acesso em: 18 março de 2019.

IMASTERS. **O que é uma estratégia API First**. Disponível em: <<https://imasters.com.br/apis/o-que-e-uma-estrategia-api-first/>>. Acesso em: 01 dez. 2017.

LISBOA, Flávio Gomes da Silva. **Zend Framework**: Componentes Poderosos para PHP. 2. ed. São Paulo: Novatec, 2013.

MANUAL DO PHP. **Sintaxe**. <http://php.net/manual/pt_BR/language.constants.syntax.php>. Acesso em 01 dez 2017.

MANUAL DO PHP. **Arrays**.

<http://php.net/manual/pt_BR/language.types.array.php>. Acesso em 01 dez 2017.

MANUAL DO PHP. **O que o PHP pode fazer?** <http://php.net/manual/pt_BR/intro-whatcando.php>. Acesso em 18 março de 2019.

MANUAL DO PHP. **Manual do PHP**. <http://php.net/manual/pt_BR/preface.php> Acesso em: 01 dez. 2017.

W3TECHS. **Usage of server-side programming languages for websites** Disponível em: <https://w3techs.com/technologies/overview/programming_language/all>. Acesso em: 01 dez. 2017.

REFERÊNCIA ON-LINE

¹ Em: <http://php.net/manual/pt_BR/preface.php>. Acesso em: 15 fev. 2018.

² Em: <https://w3techs.com/technologies/overview/programming_language/all>. Acesso em: 15 fev. 2018.

³ Em: <<https://imasters.com.br/linguagens/php/php-7-ate-9-vezes-mais-rapido-que-o-php-5-6/>>. Acesso em: 15 fev. 2018.

⁴ Em: <http://php.net/manual/pt_BR/intro-whatcando.php>. Acesso em: 15 fev. 2018.

⁵ Em: <https://www.apachefriends.org/pt_br/index.html>. Acesso em: 01 dez. 2017.

⁶ Em: <http://php.net/manual/pt_BR/language.constants.syntax.php>. Acesso em: 15 fev. 2018.



1. C) `http://localhost/projeto1/teste.php`

Todo script PHP fisicamente deve ser colocado dentro de um diretório dentro do servidor, no caso do XAMPP (instalação padrão no Windows) `c:\XAMPP\htdocs` e a partir desta localização física seus arquivos podem ser acessados de maneira lógica através do navegador pela URL. A URL (pelo menos local) sempre será `http://localhost/` e na existência de um arquivo dentro de uma pasta esta indicação relativa deve ser feita formando assim: `http://localhost/projeto1/teste.php`

2. E) Todas as afirmativas funcionariam no PHP.

Todas as quatro alternativas para declaração de arrays no PHP seriam aceitas. Isso não quer dizer que todas resultariam em um array idêntico umas às outras, mas com a relação a quais funcionariam a resposta seria todas. Arrays podem ter índices como na alternativa II e III e não necessariamente precisam ser sequenciais, você é quem determina os índices. E na alternativa IV a declaração simples de um array, que passou a ser aceita mais recentemente e que se equipara com Javascript também é aceita não sendo mais necessário `array(...)` e sendo usado somente o uso de colchetes. Já é, inclusive, uma declaração que caiu no gosto dos desenvolvedores como pode ser visto na documentação oficial.

3. B) Apenas as alternativas I e III funcionariam no PHP.

A declaração de decimais (flutuantes) não pode de maneira alguma utilizar vírgulas pois o separador decimal é feito através do símbolo ponto. O valor 12.500 (doze mil e quinhentos) em PHP deveria ser 12500 e o valor 12.500,50 (doze mil, quinhentos e cinquenta) deveria ser 12500.50 não sendo utilizado o símbolo vírgula. Lembre-se: você pode até apresentar ao usuário algo mais coerente com a realidade dele (valores monetários com separadores de milhar e decimal com uso de funções como `number_format` ou `money_format`), mas nunca deverá salvar assim, pois o PHP ou vai interpretar errado o valor ou causar um erro. Outro caso é o array 1,2,3 que na verdade ficou faltando a declaração `array(1,2,3)` ou `[1,2,3]`.

4. Resposta: 36.50

A declaração dos valores pode até causar uma ligeira confusão, mas como o PHP tem variáveis com tipagem automática no momento de uma multiplicação e soma o PHP converterá os valores para números. Neste caso, o único detalhe é que assim como em uma operação matemática padrão, a multiplicação tem prioridade sobre a soma de modo que `$numero2 * $numero3` será executado primeiramente resultando em 24 e só em seguida a soma com `$numero1` resultando em 36.50 será obtida. Esta atividade é apenas um lembrete de atenção e análise.

5. Resposta: 32

Variáveis por referência sempre causarão uma confusão até que se acostume e passe a analisar com mais calma os recebimentos e operações com valores. Na atividade em questão, \$numero1 inicialmente recebe 10 e \$numero2 passa a receber o valor de \$numero1 que é 10 e até aqui, tudo normal. A atividade fica mais complexa quando \$numero3 recebe por referência a variável \$numero1 e em seguida é incrementada. Isso quer dizer que tanto a variável \$numero3 que era 10 quanto a variável \$numero1 passaram a ser 11, pois estão vinculadas por referência. A partir daquele momento, qualquer alteração em \$numero1 ou \$numero3 resultariam em ambas. Deste modo, $11+10+11$ que seriam os valores para \$numero1, \$numero2 e \$numero3, respectivamente dariam 32.



PRIMEIROS PASSOS COM PHP



Objetivos de Aprendizagem

- Apresentar as formas com que o PHP realiza a saída de conteúdo simples, bem como de conteúdo embutido em HTML.
- Compreender o funcionamento de estruturas de controle condicional IF/ELSEIF/ELSE e SWITCH CASE.
- Compreender o funcionamento de estruturas de controle de repetição WHILE, DO-WHILE, FOR e FOREACH, assim como a funcionalidade dos comandos de desvio.
- Conhecer as variáveis pré-definidas do PHP que possibilitam além de informações de ambiente muitas outras informações do servidor e dos clientes.
- Apresentar através de códigos o recebimento de informações por meio requisição GET através de uma URL.
- Apresentar através de códigos o envio e o recebimento de informações por meio de requisição POST através de formulários HTML.

Plano de Estudo

A seguir, apresentam-se os tópicos que você estudará nesta unidade:

- Saídas de conteúdo e HTML com scripts PHP
- Estruturas de Controle Condicional
- Estruturas de Controle de Repetição e Comandos de Desvio
- Variáveis Pré-Definidas
- Recebimento de dados pela URL via GET
- Recebimento de dados por formulário via POST

INTRODUÇÃO

Caro(a) aluno(a), neste momento você está se aproximando de um marco único nos seus estudos sobre Backend, no qual já terá condições de aplicar de forma prática seus conhecimentos em torno de aplicações web mais simples.

Antes disso, apresentarei de uma maneira mais pontual e prática como o PHP realiza a saída de dados, que é fundamental para o desenvolvimento de aplicações. Vale a pena salientar que o PHP entrega para o navegador basicamente HTML. Na etapa de processamento da informação regras de negócios poderão ser aplicadas e você precisa conhecer muito bem as oportunidades deixadas pelo PHP.

As estruturas de controle condicional farão parte do seu dia a dia em praticamente todas suas aplicações e, além de entender seu funcionamento, é importantíssimo que você pratique estes conceitos, pois, neste caso, a experiência prática com o código é o que vai te possibilitar uma melhor análise e produtividade. Enquanto estruturas de controle condicional tomarão as decisões dentro de sua aplicação, estruturas de controle de repetição serão responsáveis por, na maior parte dos casos, fazer a apresentação de dados, como em relatórios.

Para concluir seus estudos, nesta unidade, será apresentado a você alguns conceitos fundamentais sobre o envio e recebimento de dados via GET e POST seja através de formulários HTML ou de URL (via GET). Estes conceitos serão os primeiros com os quais você terá contato e, com base no estudo de caso presente no final do material, poderá desenvolver suas primeiras aplicações de HTML com PHP através de envio e recebimento de formulários com processamento, regra de negócios e apresentação através de laços de repetição.

Bons estudos!



SAÍDAS DE CONTEÚDO HTML COM SCRIPTS PHP

Toda linguagem que trabalhe para páginas web tem como seu recurso básico a saída de comandos em tela que poderão emitir textos e inclusive HTML. Aquele famoso arquivo que imprime a mensagem "Olá mundo!" em PHP ficaria assim:

Arquivo: olamundo.php

```
<?php
echo "Olá mundo!";
?>
```

O comando echo faz a função de impressão na resposta que será dada e exibida no navegador, mas ele não é o único. Existe o comando print que faz a mesma coisa que echo. Desenvolvedores preferem o echo e é importante apenas que você não pegue o costume de misturar muito, deste modo, escolha um e mantenha o uso.

O comando de saída de texto também pode envolver estruturas HTML sem maiores complicações. Veja abaixo:

Arquivo: olamundo_html.php

```
<?php
echo "<h1>Olá mundo!</h1>";
?>
```

Este tipo de comando que resulta em uma saída de texto misturado com HTML é muito comum em projetos pequenos para internet e também em códigos de programadores iniciantes. O ideal é que saibamos organizar os projetos de modo a separar a camada de interface HTML da camada do PHP e para isso usamos o código PHP embutido no código HTML. O mesmo código apresentado acima ficaria assim com o PHP embutido:

Arquivo: olamundo_html.php

```
<h1><?php echo "Olá mundo!"; ?></h1>
```

O código com PHP embutido pode conter o PHP em várias linhas ou apenas declarado dentro do HTML como no exemplo acima. Geralmente, quando se trata de apenas uma instrução e curta, o código acima é bem comum de ser visto. Outra opção ainda melhor é com a utilização do recurso de impressão curta, desde que a configuração `short_open_tags` esteja habilitada dentre as configurações do PHP. Veja abaixo:

Arquivo: olamundo_html_curto.php

```
<h1><?= "Olá mundo!"; ?></h1>
```

Deste modo, `<?=` substitui `<?php echo` e isso auxilia muito na integração entre páginas HTML com textos que venham a ser gerados pelo PHP. Além da instrução `echo` e `print` também temos algumas outras opções que vieram da herança com linguagem de programação C. Temos, por exemplo, o `printf` e o `sprintf`, que realizam um trabalho bem interessante em alguns casos.

REFLITA



A saída de conteúdo por meio do PHP será feita sempre por você a partir do momento que iniciar seu caminho com PHP. Este conceito servirá para qualquer linguagem backend cliente/servidor. Lembre-se sempre que existe a entrada, o processamento (PHP) e a saída (conteúdo + HTML).

Arquivo: olamundo_com_printf.php

```
<?php
    $nome = "Eduardo Bona";
    $idade = 31;
    printf("O nome do usuário é %s e a idade %s", $nome, $idade);
?>
```

Arquivo: olamundo_com_sprintf.php

```
<?php
    $nome = "Eduardo Bona";
    $idade = 31;
    $mensagem = sprintf("O nome do usuário é %s e a idade %s",
    $nome, $idade);
?>
    <p><?=$mensagem?></p>
```

A utilização de `printf` funciona como um `print/echo` e demanda como primeiro parâmetro da função o formato e em seguida as variáveis que deverão ser substituídas. No caso, `%s` é substituído por uma string informada posteriormente e é a opção mais utilizada dentro das inúmeras possibilidades existentes, presentes neste link: http://php.net/manual/pt_BR/function.sprintf.php.

Já o comando `sprintf` ao invés de realizar diretamente a impressão como um comando `echo/print` apenas retorna o valor e assim uma variável pode receber o retorno e posteriormente ser utilizada em outro lugar.

A utilização de PHP embutido no HTML ou de um script PHP que imprime HTML com conteúdo pode ser usado quantas vezes for necessário dentro de um arquivo com extensão PHP.

ESTRUTURAS DE CONTROLE CONDICIONAL



Estruturas de controle permitem às aplicações que sejam tomadas decisões. As estruturas de controle mais conhecidas são partes dos estudos de algoritmos já no início de nossas carreiras e compartilhadas entre uma esmagadora maioria de linguagens de programação, sendo as mais lembradas as condicionais e as estruturas de repetição.

O construtor IF (se) é um dos recursos mais importantes em muitas linguagens e isso não seria diferente no PHP. Além do IF (se), que permite uma instrução como condicional no PHP, podemos também fazer uso do ELSE (senão) e do ELSEIF (senão se), que sempre dependerão da condicional imposta no IF e podem ser usados em conjunto ou não. Não há limites para o uso do ELSEIF (senão se).

Arquivo: condicional_if.php

```
<?php

$nivel_agua = 24;
$limite_agua = 20;

if ($nivel_agua > 100) {
    echo "O reservatório está acima de 100%";
}

if ($nivel_agua > $limite_agua) {
    echo "O nível da água está acima do limite";
}else{
    echo "O nível da água está igual ou abaixo do limite";
}

if ($nivel_agua < 10) {
    echo "O nível da água está quase acabando";
} else if ($nivel_agua < 5) {
    echo "Se não economizar vai acabar.";
} else if ($nivel_agua <= 0){
    echo "Acabou...";
}
?>
```

Se é seu primeiro contato com uma estrutura condicional IF saiba que ela é muito comum nas linguagens de programação. A estrutura em si termina a tomada de decisão (caminho), mas o que determina mesmo a regra de negócios das decisões é obtida com os operadores de comparação e operadores lógicos.

Tabela 1 - Operadores de Comparação

EXEMPLO	NOME	RESULTADO
<code>\$a == \$b</code>	Igual	Verdadeiro (TRUE) se \$a é igual a \$b
<code>\$a === \$b</code>	Idêntico	Verdadeiro se \$a é igual a \$b, e eles são do mesmo tipo (int, float, etc)
<code>\$a != \$b</code> <code>\$a <> \$b</code>	Diferente	Verdadeiro se \$a não é igual a \$b
<code>\$a !== \$b</code>	Não Idêntico	Verdadeiro se \$a não é igual a \$b, ou eles não são dos mesmo tipo
<code>\$a < \$b</code>	Menor que	Verdadeiro se \$a é estritamente menor que \$b
<code>\$a > \$b</code>	Maior que	Verdadeiro se \$a é estritamente maior que \$b
<code>\$a <= \$b</code>	Menor ou igual	Verdadeiro se \$a é menor ou igual a \$b
<code>\$a >= \$b</code>	Maior ou igual	Verdadeiro se \$a é maior ou igual a \$b

Fonte: Manual do PHP ([2018], on-line)¹.

Com os operadores lógicos é possível criar condições de combinações entre expressões. Neste caso, poderíamos por exemplo validar se a idade tem idade superior ou igual a dezoito e resida no estado da Bahia (`$idade >= 18 and $uf = 'BA'`).

Tabela 2 - Operadores Lógicos do PHP

EXEMPLO	NOME	RESULTADO
<code>\$a and \$b</code> <code>\$a && \$b</code>	E	Verdadeiro (TRUE) se tanto \$a quanto \$b são verdadeiros.
<code>\$a or \$b</code> <code>\$a \$b</code>	OU	Verdadeiro se \$a ou \$b são verdadeiros
<code>\$a xor \$b</code>	XOR	Verdadeiro se \$a ou \$b são verdadeiros, mas não ambos.
<code>! \$a</code>	NÃO	Verdadeiro se \$a não é verdadeiro

Fonte: Manual do PHP ([2018], on-line)².

Temos também no PHP a declaração SWITCH que é basicamente uma sequência da mesma expressão condicional sendo executada (comparada) com diferentes valores. Em muitos casos fica muito mais organizado e legível utilizar SWITCH ao invés do IF.

Nos casos de estruturas com SWITCH em cada opção apresentada precisamos fazer a comparação com case e adicionar após a sua instrução o comando break; para sair da estrutura de controle. Você pode default: como última opção obrigando a execução da instrução default caso seu código chegue até ali.

Arquivo: condicional_switch.php

```
<?php
$opcao = 1;
switch ($opcao) {
    case 1:
        echo "Você escolheu opção 1: saque.";
        break;
    case 2:
        echo "Você escolheu opção 2: depósito.";
        break;
    default:
        echo "Opção inválida";
        break;
}
?>
```

Segundo a documentação oficial do PHP, é importante entender como a declaração switch é executada a fim de evitar enganos, pois a declaração switch executa linha por linha (na verdade, declaração por declaração) sendo que no início nenhum código é executado. Somente quando uma declaração case é encontrada com um valor correspondente ao valor da expressão do switch, o PHP começará a executar a declaração. O PHP continuará a executar a declaração até o fim do bloco switch ou até a primeira declaração break encontrada. Se não for escrita uma declaração break ao final da lista de declarações do case, o PHP continuará executando as declarações dos próximos cases.



ESTRUTURAS DE CONTROLE DE REPETIÇÃO E COMANDOS DE DESVIO

Segundo Niederauer (2017, p. 74), as estruturas de controle de repetição "são comandos utilizados para que um conjunto de instruções seja executado repetidamente por um número determinado de vezes, ou até que uma determinada condição seja atingida".

Assim como acontece com a estrutura condicional, a maior parte das estruturas de controle de repetição utilizados no PHP são as mesmas já conhecidas e utilizadas em outras linguagens como WHILE, DO-WHILE e FOR.

A estrutura while tem um objetivo e uma declaração muito simples. Seu objetivo é executar um bloco de código enquanto uma condicional não esteja sendo atendida.

Arquivo: repeticao_while.php

```
<?php
$a = 10;
$b = 21;
while ($a < $b) {
    $a = $a + 2;
}
echo $a;
?>
```


Resultado: 22

O que se declara após a chamada de `while` é a condição. No caso acima, a condição é apenas uma comparação "menor que", mas você pode fazer uso de uma expressão com operadores lógicos como demonstrado em uma estrutura condicional IF.

A repetição DO-WHILE tem o mesmo objetivo de WHILE com a única diferença que a condicional é validada ao término do bloco e não no início. Isso tem uma diferença fundamental: como a validação é feita ao final a primeira iteração sempre irá ocorrer.

Arquivo: `repeticao_dowhile.php`

```
<?php
$a = 30;
$b = 21;
do{
    $a = $a + 2;
}while ($a < $b);
echo $a;
?>
```

Resultado: 32

Note, em um WHILE se a condição não for atendida ele nem acessa o bloco declarado, mas com DO-WHILE, como é possível observar no exemplo acima, ainda que a variável `$a` seja maior que a variável `$b` fazendo o bloco de código, seria executado uma vez até a validação encerrar a repetição.

Outra repetição muito tradicional e famosa é a FOR. Como o PHP é proveniente de linguagem C, tanto WHILE, DO-WHILE e FOR possuem sintaxe muito semelhante.

FOR tem por vantagem executar em uma única declaração a inicialização de uma variável, a condição de repetição e a instrução de iteração do laço de repetição. Esta característica faz de FOR uma estrutura muito conhecida e utilizada.

Arquivo: `repeticao_for.php`

```
<?php
for ($inicio=2014; $inicio <= 2017; $inicio++) {
    echo $inicio;
}
?>
```

Resultado: 2014201520162017

O mais comum em uma declaração FOR é que sua última expressão sempre seja uma incrementação, mas é importante ressaltar que isso não é uma regra.

Dentre as estruturas de repetição, a que mostrarei a seguir sem dúvida será uma das mais utilizadas por você. O FOREACH tem uma estrutura simples e eficiente para interagir com elementos do tipo array e como a maioria dos resultados que mostramos nas telas são oriundas de resultados contidos em um array, esta estrutura deverá ser objeto de muito estudo seu.

Arquivo: repeticao_foreach.php

```
<?php
$carros = array("Focus", "Fiesta", "Ka");
foreach ($carros as $carro) {
    echo $carro;
}
?>
```

Resultado: FocusFiestaKa

FOREACH também permite que você receba não só o valor do elemento contido no array bem como seu índice, desde que você também informe isso em sua construção, conforme exemplo a seguir:

Arquivo: repeticao_foreach_indice.php

```
<?php
$carros = array("Focus", "Fiesta", "Ka");
foreach ($carros as $indice => $carro) {
    echo $indice;
    echo ": ";
    echo $carro;
    echo " ";
}
?>
```

Resultado: 0: Focus 1: Fiesta 2: Ka

Sabemos que arrays podem ser simples como demonstrado acima ou mais complexos como as matrizes que são arrays dentro de arrays. A seguir um exemplo de um FOREACH com uma matriz:

Arquivo: repeticao_foreach_matriz.php

```
<?php
$carros = array(
    "ford" => array(
        "Focus", "Fiesta", "Ka"
    ),
    "vw" => array(
        "Fusca"
    )
);
foreach ($carros as $marca => $carros) {
    echo "Marca: $marca - ";
    foreach ($carros as $carro) {
        echo $carro;
    }
    echo "<br />";
}
?>
```

Resultado:

```
Marca: ford - FocusFiestaKa
Marca: vw - Fusca
```

Arrays com índices como strings (caracteres) também podem ser acessados normalmente, conforme exemplo a seguir:

Arquivo: repeticao_foreach_array_indice.php

```
<?php
$carros = array(
    array("modelo"=>"Focus", "fabricacao"=>"2013"),
    array("modelo"=>"Fiesta", "fabricacao"=>"2015"),
    array("modelo"=>"Fusca", "fabricacao"=>"1982")
);
foreach ($carros as $carro) {
    echo "Modelo " . $carro["modelo"];
    echo " e ano " . $carro["fabricacao"];
    echo "<br />";
}
?>
```

Resultado:

```
Modelo Focus e ano 2013
Modelo Fiesta e ano 2015
Modelo Fusca e ano 1982
```

Além das estruturas de controle de repetição mostradas acima, contamos ainda com alguns recursos de controle bem interessante, sendo dois deles muito importantes e que exigem um bom raciocínio lógico em sua utilização. Destes controles, um deles já foi usado inclusive em um SWITCH CASE. São eles: o break e o continue presentes no PHP como **comandos de desvio**.

Para exemplificar ainda mais a utilização dos mesmos, é possível que no mesmo código tenhamos ambos e você consiga compreender ainda melhor o recurso.

Arquivo: repeticao_desvio.php

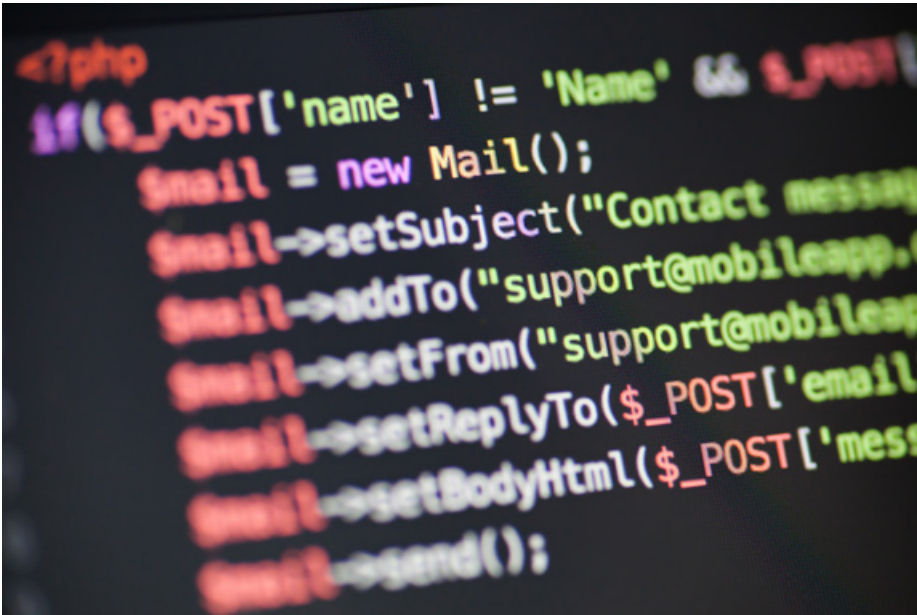
```
<?php
$carros = array("Focus", "Fusca", "Fiesta", "Ka", "Belina");
foreach ($carros as $carro) {
    if ($carro == "Fusca") {
        continue;
    }
    if ($carro == "Ka") {
        break;
    }

    echo $carro;
}
?>
```

Resultado: FocusFiesta

O comando **continue** faz com aquela iteração (naquele ponto da repetição) seja finalizada naquele momento, mas que o laço de repetição continue, ou seja, continue seu fluxo "pulando" aquela iteração. Isso faz com que seja impresso "Focus" e que na iteração seguinte ele pule "Fusca" e, em seguida, continue imprimindo "Fiesta".

Por outro lado, o comando **break** faz com que no momento da execução a repetição seja encerrada. Neste caso, no momento em que "Ka" é testado a repetição seria interrompida (concluída) e no caso não seria nem impresso "Ka" nem "Belina";



VARIÁVEIS PRÉ-DEFINIDAS

As variáveis pré-definidas do PHP apresentam desde variáveis nativas de ambiente do servidor bem como variáveis externas. As mais conhecidas são as variáveis pré-definidas para recebimento de dados vindos de formulário que podem ser via GET ou POST.

\$GLOBALS referencia todas as variáveis disponíveis no escopo global retornando um array de índices com as variáveis disponíveis. O escopo das variáveis geralmente será notado quando estiver criando funções e houver algum conflito entre demandas externas e internas, conforme apresentado a seguir:

Arquivo: variaveis_globais.php

```
<?php
function imprimir() {
    $mensagem = "Oi";
    echo $GLOBALS["mensagem"];
    echo $mensagem;
}

$mensagem = "Mensagem 1";
imprimir();
?>
```

Resultado: Mensagem 10i

Outra variável muito interessante é a variável `$_SERVER`, que mostra informações do servidor e do ambiente de execução. Dentre as possibilidades existentes e disponíveis na Documentação do PHP ([2018], on-line)³ destaca-se a `$_SERVER["HTTP_REFERER"]` que apresenta o endereço da página (se houver) através do qual o usuário chegou à página atual e também `$_SERVER["REMOTE_ADDR"]` que mostra o IP do usuário que está realizando o acesso à página.

`$_ENV` retorna um array associativo com valores informados por variáveis de ambiente do servidor web ou do sistema operacional. Este recurso é muito utilizado hoje em dia para sistemas dedicados e únicos em um servidor e armazenam de forma compartilhada com o sistema operacional ou servidor dados como acesso ao banco de dados e parametrizações do sistema.

Já para quem irá trabalhar com formulários e que será objeto de estudo dedicado mais abaixo, três variáveis serão importantes sendo `$_GET`, `$_POST` e `$_FILES` respectivamente, recebendo os valores via método GET, POST e arquivos vinculados à solicitação.

`$_COOKIE` representa os dados armazenados em cookie pelo navegador dentro da máquina do usuário (cliente) e `$_SERVER` os valores armazenados em sessão pelo usuário. Ambos os recursos são mais avançados, mas necessários. Enquanto cookies geralmente são usados para armazenar dados não sensíveis dentro da máquina dos usuários, as sessões são utilizadas para guardar a autorização de acesso dos usuários e permitir seu uso em ambientes restritos. Será um recurso fundamental para sistemas.

Por fim, destacarei para você o `$_REQUEST` que une todas as variáveis disponíveis em `$_GET`, `$_POST` e `$_COOKIE`.



RECEBIMENTO DE DADOS PELA URL VIA GET

Existem duas formas dentre as mais comuns de se estabelecer a troca de informações pelas aplicações web que são através dos métodos HTTP GET e HTTP POST. Aliás, esta atividade se fará presente em boa parte de suas aplicações principalmente se elas forem sistemas.

Segundo o blog da empresa PagSeguro, do UOL, que é responsável por uma API de pagamento eletrônicos, no método GET "toda informação é enviada para o sistema de destino junto ao endereço do sistema" enquanto que no método POST "toda informação é enviada de forma mais escondida e não irá aparecer na URL" (BLOG PAGSEGURO, 2012, on-line)⁴.

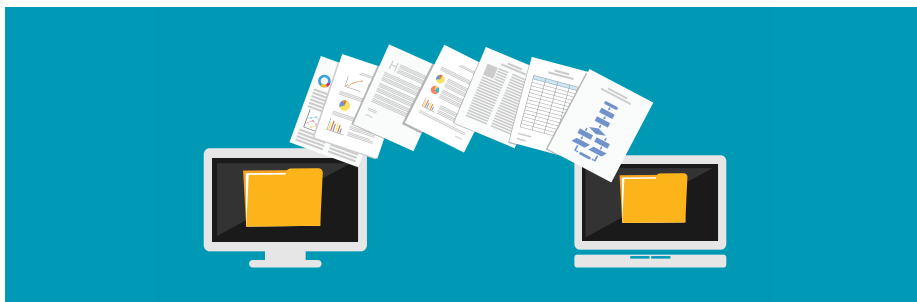
A utilização do método GET está muito presente na navegação entre as páginas através dos links (urls) e seus parâmetros e apesar de possível é pouco frequente em formulários que se aproveitam muito do método POST. Isso se dá porque o método GET trafega no que conhecemos como QUERY STRING (consulta em texto) no endereço do navegador. Analise o exemplo a seguir:

Arquivo: menu.php

```

<ul>
  <li><a href="categoria.php?id=tv">TV</a></li>
  <li><a href="categoria.php?id=fogao">Fogão</a></li>
  <li><a href="categoria.php?id=chuveiro">Chuveiro</a></li>
</ul>
Arquivo: categoria.php
<?php
$categoria = $_GET["id"];
switch($categoria){
  case 'tv':
    echo "Destaques: LG, TLC, Philco";
    break;
  case 'fogao':
    echo "Destaques: Arno, Dako";
    break;
  case 'chuveiro':
    echo "Destaques: Lorenzetti, Bella Ducha";
    break;
  default:
    echo "Categoria não encontrada";
    break;
}
?>

```



RECEBIMENTO DE DADOS POR FORMULÁRIO VIA POST

Em formulários HTML, através da tag **<form>** é possível definir, por meio do atributo **method**, qual o método utilizado para submeter as informações do formulário para o script PHP, destino através do atributo **action**. Formulários suportam tanto o envio via GET ou via POST.

Para exemplificar este caso, vamos demonstrar um arquivo que contenha um formulário e outro que receba, mas é importante saber que muitos desenvolvedores

optam por um código onde o mesmo arquivo (script) que possui o formulário seja o que recebe também. No exemplo, usarei apenas o HTML do formulário, mas é sempre bom lembrar que é necessário haver uma estrutura HTML mínima com a tag <html>, <head>, <meta> do tipo charset e <body>.

Arquivo: formulario.php

```
<form action="enviar.php" method="post">
  Nome: <input type="text" name="nome" /><br />
  Email: <input type="text" name="email" /><br />
  <input type="submit" value="Enviar" />
</form>
Arquivo: enviar.php
<?php
  $nome = $_POST["nome"];
  $email = $_POST["email"];

  echo "Recebendo dados com nome: $nome e email: $email";
?>
```

Neste caso, de maneira prática, a diferença entre utilizar o método POST ou o método GET consistiria apenas em especificar no atributo method do formulário entre os valores POST ou GET e na utilização da variável pré-definida do PHP que pode ser \$_POST e \$_GET para recebimento de dados via POST ou GET, respectivamente.

Ainda que não seja muito comum, o PHP disponibiliza a variável pré-definida \$_REQUEST, que recupera os valores tanto via GET ou POST. No entanto, não é muito comum utilizar esta variável mais genérica.

Se no código, a diferença entre POST e GET é relativamente pouca, em sua aplicação a diferença é muito grande.

Uma explicação mais técnica para esta diferença pode se tornar uma sopa de letrinhas em sua cabeça, mas a principal diferença entre POST e GET é exatamente como o blog do PagSeguro procura explicar. No método GET, você irá notar que se seu formulário possuir dois campos de texto, tanto o nome dos campos quanto os valores seria enviados junto com a URL enquanto que via POST, estes dados iriam ser enviados de uma maneira menos exposta.

É preciso estar claro que formulários como de login, por exemplo, que enviam o usuário e a senha devem estar em método POST justamente para que sejam menos expostos e que de maneira alguma apareçam na URL, o que aconteceria

se você optasse pelo uso via GET. Sobre este assunto, ainda é extremamente relevante deixar claro que mesmo sendo melhor que GET, o método POST não é 100% seguro, pois ele esconde a informação apenas dos olhos do usuário, não do tráfego de sua rede. Deste modo, para que tudo fique mais seguro, é importante um estudo sobre protocolo HTTPS e, aí sim, garantir mais segurança com o tráfego de dados criptografados.



SAIBA MAIS

A utilização do protocolo HTTPS é fundamental para alguns tipos de aplicações e em breve será comum para todo e qualquer conteúdo que esteja na internet. Ainda segundo artigo publicado pelo portal TechTudo (on-line), qualquer aplicação que seja necessário digitar senhas ou números sensíveis como de cartão de crédito é imprescindível a utilização de HTTPS.

Fonte: o autor.

Usando ainda como exemplo o formulário anterior, se o método atribuído a ele fosse GET e eu tivesse digitado o nome Eduardo Bona e o e-mail ead@eduardobona.com.br, ao submeter o formulário, os dados seriam enviados para o endereço `enviar.php?nome=Eduardo Bona&email=ead@eduardobona.com.br` devidamente expostos na URL automaticamente e deveriam ser recebidos por `$_GET` no PHP.

CONSIDERAÇÕES FINAIS

A saída de conteúdo com utilização do PHP é algo trivial para seu trabalho seja desenvolvendo um site, um comércio eletrônico ou uma aplicação web, como uma rede social ou um sistema de farmácia. A organização do código é fundamental e por isso entender sobre PHP embutido será um diferencial.

Estruturas de controle condicional e de repetição são mais do que comuns em qualquer linguagem de programação e com PHP não será diferente. No PHP temos inclusive o uso de FOREACH que para estruturas em formato de array são muito utilizadas e você deve praticar muito este recurso, pois é essencial e atualmente ainda tenho visto muitos programadores, fazendo uso desnecessário de FOR ou WHILE quando FOREACH resolve muito bem. Voltamos à questão de códigos organizados e simples e nisso o FOREACH atende muito bem aos arrays.

Estruturas de controle condicional parecem simples, mas não são. Expressões com várias combinações são muito possíveis e comuns em aplicações. A boa escrita e a boa leitura de estruturas de controle são objetos de discussão em todas as equipes de programação seja esta equipe iniciante ou não. Estruturas condicionais mal feitas são sempre a primeira crítica que desenvolvedores farão a códigos de outros desenvolvedores e você pode desde já trabalhar para fugir disto e ser visto como um desenvolvedor limpo, organizado e claro.

Desenvolvedores web, principalmente aqueles que terão foco em aplicações, trabalharão muito com formulários em seu dia a dia. Enviar e receber adequadamente será determinante para a produtividade de seu trabalho e para a qualidade dele.

Estamos já caminhando para a parte final dos estudos fundamentais sobre PHP e você já está começando a ser exigido. Atividades presentes até aqui já podem ser praticadas em cenários reais como criação e envio de um formulário de contato ou para pedir um lanche e emitir o valor da comanda.

Revise o conteúdo e bons estudos!

ATIVIDADES



1. Analise a seguinte estrutura de controle condicional IF e aponte qual será o resultado ao término da execução do código abaixo:

```
<?php
$num = 2;
$num *= 3;

if ($num > 4) {
    echo "caiu no if";
} else if ($num > 6) {
    echo "caiu no elseif ";
} else {
    echo "caiu no else";
}
```

- a) caiu no if caiu no elseif
- b) caiu no if
- c) caiu no elseif
- d) caiu no else
- e) caiu no if caiu no elseif caiu no else

2. Ao submeter uma tag `<input type="text" name="nome" />` através de `<form action="teste.php" method="post">` como deveria ser recebido o valor deste campo dentro de uma variável \$nome?

```
$nome = $_POST["nome"];
```

3. Analise o código abaixo:

```
Arquivo: bemvindo.php
<?php
$codigo = $_GET["cod"];
switch($codigo) {
    case 1:
        echo "Seja bem vindo Alberto";
        break;
    case 2:
        echo "Seja bem vindo Brenner";
        break;
    case 3:
        echo "Seja bem vindo Joaquim";
        break;
}
```

Agora, aponte qual url deveria ser acessada no navegador para que fosse mostrada a mensagem "Seja bem vindo Joaquim" (supondo que o endereço base seja `http://localhost/bemvindo.php`)?

ATIVIDADES



4. O evento Copa do Mundo é realizado de quatro em quatro anos e a última foi realizada no Brasil, em 2014. Sendo assim, imagine um script que receba por parâmetro GET (na url) a variável ano_limite. Com isso, (usando FOR) você precisa desenvolver um script PHP que imprima na tela do usuário quantas copas do mundo haverão até o ano limite informado a partir do ano de 2014 que deve ser desconsiderado pois uma Copa já fora realizada (lembre-se que se o ano for menor ou igual ao ano atual de 2014 deve retornar 0).
5. Qual elemento do formulário do tipo input é fundamental para que seja criado um botão que tenha função de enviar o formulário?

```
<input type="submit" />
```

6. Imaginando que um código PHP possua as seguintes variáveis: \$nota1, \$nota2, \$nota3 e \$nota4 e que estas notas podem receber valores de 0 a 10.

Faça um script PHP que imprima na tela uma mensagem se o aluno está aprovado ou reprovado (lembrando que um aluno estaria reprovado neste caso se a média estivesse abaixo de 6).



O que é um programa realmente?

É assim que o autor Max Kanat-Alexander sugere o início de um tópico em seu livro *As leis fundamentais do projeto de software*, cuja resposta não podia ser mais clara do que "uma sequência de instruções dados ao computador" e "as ações executadas por um computador como resultado de ter recebido instruções".

Com base nesta afirmação, que até parece óbvia, temos que entender que uma sequência de instruções dadas ao computador precisa ser feita em conformidade com a linguagem adotada mas antes de mais nada dentro de uma lógica desenvolvida por seu desenvolvedor. E o que diferencia o código de um bom desenvolvedor é a lógica, a forma adotada para resolver os problemas.

Estruturas de controle condicional e estruturas de controle de repetição podem ter uma lógica simples ou muitas vezes complexa, mas quem determina se uma estrutura será simples ou complexa não é o problema a ser resolvido e sim o desenvolvedor.

Já me deparei com muitas estruturas condicionais que resolviam problemas complexos terem sido desenvolvidas de maneira simples o que facilitava muito o entendimento do código, a leitura sequencial dele e a manutenção posterior. Também já vi soluções que eram para ser simples se tornarem complexas pelo fato do desenvolvedor não ter se preocupado com organização, complexidade ou utilização correta das instruções dadas pelo PHP.

Um exemplo disso é que temos algumas estruturas de controle condicional disponíveis e diversas estruturas de repetição e a pergunta sempre vai ser qual é a melhor para ser usada e quando deve ser usada. A resposta para isso dependerá do entendimento de cada desenvolvedor, mas eu sugiro sempre o que muitas empresas chamam de revisão de código ("code review" em inglês), que é quando outro desenvolvedor revisa o código de alguém da equipe. Se outra pessoa leu e entendeu o que desenvolveu é um excelente sinal. Pode continuar. Do contrário, é o momento para um alinhamento, uma discussão e uma definição de padrão. O envio e o recebimento de formulários também pode ser uma tarefa bem difícil para quem está iniciando e só a prática vai fazer com que o código de cada desenvolvedor amadureça.

Uma forma de pular este caminho é consultando projetos de código aberto desenvolvidos e analisar como bons desenvolvedores resolvem problemas. Outra dica é estudar o que chamamos de frameworks que são estruturas já prontas para auxiliar no desenvolvimento de novas soluções baseado no padrão já estabelecido.

Toda aplicação em PHP, seja ela um sistema para uma pequena farmácia ou um sistema grande para uma rede de farmácias, possui uma arquitetura de aplicação. Dentro desta arquitetura temos nossas regras de negócios concentradas e estas regras são, muitas vezes, produzidas por estruturas de controle de repetição e de controle condicional.

Por isso, não complique, simplifique. Dica para programar de 30 em 30 minutos: pense durante os primeiros 15 minutos, digite durante outros 5 minutos, teste durante outros 5 minutos e refatore (melhore) seu código nos últimos 5 minutos.





Um excelente lugar para praticar suas habilidades, seja na resolução de problemas rápidos, bem como em soluções mais pensadas, é o site <<https://codefights.com/>> que propõe uma série de atividades com um ar de competição entre você e uma máquina para resolver problemas de programação. O site é em inglês, mas conhecendo o PHP e sua sintaxe é bem tranquilo de interagir. Não custa nada e é um lugar que até o Uber tem usado para recrutar desenvolvedores.

Conhecer, praticar e evoluir não é um diferencial, é uma necessidade.

Fonte: o autor.





LIVRO

Desenvolvimento Web com PHP e MYSQL

Evaldo Junior Bento

Editora: Casa do Código

Sinopse: Aprenda com esse livro os primeiros passos para usar o PHP, uma ferramenta que possibilita a criação de páginas web dinâmicas. Veja também como integrar o PHP ao MySQL, um dos bancos de dados mais usados no mundo. A dupla PHP e MySQL é usada por diversos tipos de aplicações e sites, de blogs à portais de notícias e grandes redes sociais.

Comentário: Este livro é bem completo e apresenta uma aplicação web de uma forma bem organizada dentro de um contexto de uma arquitetura. Os conceitos abordados sobre banco de dados MYSQL, que ainda não são alvo de nossos estudos, poderão já embasar os primeiros conhecimentos sobre PHP + MYSQL que você precisará no decorrer de outras necessidades.



NA WEB

PHP 7 por Guilherme Blanco

Nesta palestra apresentada na iMasters Experience em 2016, Guilherme Blanco, um dos principais desenvolvedores de PHP do Brasil, conhecido mundialmente por suas contribuições e lideranças em projetos de código aberto com a comunidade, descreve as melhorias e os impactos presentes nestas melhorias.

Acesse: <https://www.youtube.com/watch?v=vpqNw91eD1c&index=13&list=PLASrXUpwQG6fzm4BFxK-8ccJ_APZlmgm5>.

iMasters Experience 2017

Esta playlist do Youtube, no canal do portal iMasters, apresenta uma série de vídeos sobre desenvolvimento web e PHP, focado principalmente nas boas práticas do desenvolvimento. São várias palestras, foque principalmente nas que tratam de tecnologias e boas práticas.

Acesse: <https://www.youtube.com/watch?v=HaMedLkwe8I&list=PLASrXUpwQG6fZDwfpAlooy_LSci--adxQ>.

REFERÊNCIAS

MANUAL DO PHP. **Manual do PHP**. <http://php.net/manual/pt_BR/preface.php>. Acesso em: 01 dez. 2017.

MANUAL DO PHP. **Função sprintf**. <http://php.net/manual/pt_BR/function.sprintf.php>. Acesso em: 01 dez. 2017.

MANUAL DO PHP. **Operadores Lógicos** <http://php.net/manual/pt_BR/language.operators.logical.php>. Acesso em: 01 dez. 2017.

MANUAL DO PHP. **Variável SERVER**. <https://secure.php.net/manual/pt_BR/reserved.variables.server.php>. Acesso em: 19 março 2019.

REFERÊNCIA ON-LINE

¹ Em: <http://php.net/manual/pt_BR/language.operators.comparison.php>. Acesso em: 15 fev. 2018.

² Em: <http://php.net/manual/pt_BR/language.operators.logical.php>. Acesso em: 15 fev. 2018.

³ Em: <http://php.net/manual/pt_BR/reserved.variables.server.php>. Acesso em: 15 fev. 2018.

⁴ Em: <<https://blogpagseguro.com.br/2012/03/get-e-post-o-que-sao/>>. Acesso em: 15 fev. 2018.



1. B) caiu no if

Relembrando a questão de atribuição de valores a uma variável é possível notar que na atividade, inicialmente, recebemos o valor 2 para ela e em seguida atribuímos a ela, o próprio valor dela multiplicado por 3, resultando em 6.

Posteriormente, criando-se a condicional para \$num maior que 4, temos que o valor atual da variável sendo 6, se adequaria a primeira opção resultando em uma resposta "caiu no if"

2.

\$nome = \$_POST["nome"];

Como o método proposto pelo formulário é o POST faremos uso da variável pré-definida \$_POST dentro de nosso código para receber o valor. Todo formulário POST submetido irá ser representado dentro um array na variável \$_POST com o índice representando a referência através do atributo name do campo do formulário.

Neste caso então, será recebido em \$_POST["nome"] uma vez que nome é o valor atribuído para atributo name.

3.

http://localhost/bemvindo.php?cod=3

Tendo como base o endereço <http://localhost/bemvindo.php> e o código apresentado, podemos pressupor que para que a mensagem "Seja bem-vindo Joaquim" seja apresentada será necessário informar ao parâmetro "cod" o valor 3. Deste modo, na URL, podemos informar isso através do nome do script seguido de ?cod=3 ficando então a URL completa <http://localhost/bemvindo.php?cod=3>.

O recebimento de parâmetros pela URL é obtido por meio da variável pré-definida \$_GET do PHP. Submissões por método POST não vão para URL.

4.

```
<?php
$ano_limite = $_GET["ano_limite"];
if($ano_limite <= 2014) {
    echo 0;
    exit();
}
$anos = -1;
for ($i=2014; $i<=$ano_limite; $i = $i+4) {
    $anos++;
}
echo $anos;
```

5.

```
<input type="submit" />
```

Todo formulário precisa de um botão para submissão do mesmo e o principal meio de se obter isso é através do `<input type="submit" />`

6.

```
// poderia ser feita a média anteriormente (ideal)
// importante usar parênteses para a conta funcionar matem-
// aticamente correta
$media = ($nota1+$nota2+$nota3+$nota4) / 4;
if ($media >= 6) {
    echo "Aprovado";
} else {
    echo "Reprovado";
}
// ou poderia toda a expressão estar dentro da condicional
// (não recomendado)
// $media ou é >= 6 para aprovado ou seria <6 para reprovado.
```



FUNÇÕES NO PHP

UNIDADE

IV

Objetivos de Aprendizagem

- Conceituar funções pré-definidas e funções definidas pelo usuário.
- Apresentar as funções pré-definidas da linguagem PHP para tratamento de cadeia de caracteres (strings).
- Apresentar as funções pré-definidas para utilização, manipulação e aplicação matemática.
- Apresentar as funções pré-definidas para utilização, manipulação e aplicação de recursos em arrays.
- Apresentar as funções pré-definidas para manipulação em sistemas de arquivos (filesystem).

Plano de Estudo

A seguir, apresentam-se os tópicos que você estudará nesta unidade:

- Funções definidas pelo usuário
- Funções para processamento de texto (strings)
- Funções matemáticas
- Funções para manipulação de arrays
- Funções para sistemas de arquivos

INTRODUÇÃO

A utilização de funções é uma característica presente nas linguagens de programação e você poderá encontrar funções não só em backend, como é o caso do PHP, mas também poderá se deparar com o seu uso em Javascript, que é a linguagem de programação mais utilizada para frontend e até em banco de dados dentro da linguagem SQL.

No PHP, você fará uma rápida imersão em dois aspectos diferentes sobre funções, sendo inicialmente conceituado a utilização de funções e como você poderá criar suas próprias funções e modelá-la de maneira a atender efetivamente seu código. Funções podem receber argumentos ou parâmetros que podem ser obrigatórios ou não e podem ser passados até por referência. Funções podem ser utilizadas no PHP como procedimentos com o único objetivo de executar operações ou podem ser utilizadas como recursos que recebem dados, processam e retornam algum resultado.

As funções pré-definidas da linguagem são aquelas que já existem e estão prontas para seu uso. Conhecê-las trará vantagens para você, pois economizará seu tempo e evitará o retrabalho em códigos desnecessários uma vez que elas já estão construídas e estão largamente testados pelo PHP.

Você irá conhecer funções para cadeia de caracteres (strings), manipulação de dados arrays e sistema de arquivos (filesystem) e com certeza muitas delas no seu dia a dia profissional não sairão mais da sua cabeça.

Em uma próxima etapa de seus estudos quando conhecer a orientação a objetos se deparará com o que conhecemos como métodos de uma classe e estas terão comportamento muito semelhante a uma função com passagem de parâmetros e retornos e por isso é muito importante que você se empenhe desde já.

Bons estudos!

FUNÇÕES DEFINIDAS PELO USUÁRIO



Assim como muitas linguagens de programação, o PHP também permite que os próprios desenvolvedores criem suas próprias funções. A criação de funções permite o encapsulamento de instruções proporcionando aos desenvolvedores muitas vantagens como:

- reutilização: ao utilizar uma função muitas vezes, você evita precisar reescrever os códigos desta função diversas vezes em sua aplicação.
- simplicidade: executar uma instrução complexa (muitas linhas) com apenas uma chamada (uma linha) torna seu código mais legível e mais simples.
- organização: separar funções fora do escopo da aplicação de modo a centralizar diversas funções por áreas ou finalidades.
- diminuição da complexidade: pequenas funções são mais simples de serem analisadas, testadas e corrigidas quando for o caso.

No PHP, existem as funções definidas pela linguagem que são aquelas que já estão disponíveis e que sempre iremos utilizar em nossas aplicações. Não precisamos reinventar o que já foi criado, apenas utilizá-lo. A vantagem das funções nativas do PHP é que atendem a objetivos muito específicos, como manipular strings, arrays, datas ou arquivos, por exemplo.

Por que desenvolver funções então?

- O PHP pensou em função para muitas utilidades, mas com certeza não pensou em tudo. Em alguns momentos, não existirá uma função para te atender especificamente e será preciso com isso desenvolver a sua versão (modificação) ou uma nova função.
- Sua aplicação possui regras de negócios e uma construção totalmente particular. Deste modo, você pode criar funções para auxiliar no desenvolvimento de seu projeto.

De maneira muito simples, uma função pode ser criada e executada da seguinte forma no PHP:

Arquivo: funcao_1.php

```
<?php
function ola_mundo() {
    echo 'Olá Mundo!';
}
ola_mundo();
ola_mundo();
?>
```

Resultado: Olá Mundo!Olá mundo!

A criação de uma função sempre é precedida da utilização da palavra reservada "function" + "nome da função" e a execução de uma função de fato é feito por meio do seu próprio nome e a utilização dos parênteses "()" que são sempre utilizados quando criamos funções que não possuem parâmetros.

O nome de uma função respeita os mesmos princípios de uma variável e seu escopo é definido sempre dentro de um bloco de códigos, assim como nas estruturas condicionais e de repetição.

SAIBA MAIS



Uma função possui escopo próprio e por isso atente-se às variáveis criadas dentro de uma função que não terão visibilidade nenhuma externa à função. No início, evite criar variáveis com o mesmo nome para evitar a confusão na leitura dos códigos e se lembre que o que foi declarado dentro de uma função morre dentro da função (escopo).

Fonte: o autor.

Sobre o escopo de variáveis dentro de uma função, o exemplo a seguir demonstra bem um simples exemplo:

Arquivo: funcao_escopo.php

```
<?php
$a = 1;
function imprime_a() {
    $a = 2;
    $a = $a + 5;
    echo $a;
}
imprime_a();
echo $a;

?>
```

Resultado: 71

O resultado será 71, pois a execução da função "imprime_a()" será 7 e depois a função de impressão "echo \$a" imprimirá 1, pois este valor não sofreu alteração dentro da função, devido ao escopo ser externo. Apenas com o uso de escopo global ou estático do PHP estes fundamentos poderiam ser alterados, mas para um primeiro contato este exemplo é o suficiente para demonstrar possíveis problemas com o entendimento equivocado do escopo das variáveis.

Esclarecido a declaração de uma função e a chamada de uma função nosso segundo fundamento é com relação ao retorno ou não dentro de um conjunto de instruções. Como visto no exemplo acima, dentro de uma função além de declarações e somas, a própria função se encarregou de imprimir por meio do comando "echo" o resultado. Nestes casos, dizemos que a função não tem retorno, pois ela mesma se encarregou internamente de executar as instruções. Podemos chamar funções, assim como procedimentos.

O ideal sempre é que funções tenham retorno. Veja abaixo que a demonstração trata do mesmo código presente no primeiro exemplo, porém com utilização de retorno:

Arquivo: funcao_2.php

```
<?php
function ola_mundo() {
    return 'Olá Mundo!';
}
$mensagem = ola_mundo();
echo $mensagem;

?>
```

Resultado: Olá Mundo!

Com a utilização de um retorno, uma função não faria imediatamente a impressão de uma mensagem e sim o retorno desta mensagem para quem de fato a chamou e, posteriormente, sim a impressão seria feita. Esta forma sempre será a melhor, evitando que funções realizem impressões em tela, por meio de comandos "echo".

Por fim, o terceiro destaque para uma função é que ela pode possuir um ou mais parâmetros e estes parâmetros, por sua vez, podem ser obrigatórios ou não. Veja o mesmo exemplo com algumas modificações.

Arquivo: funcao_3.php

```
<?php
function imprimir_mensagem($mensagem, $saudacao = 'Bom dia')
{
    $texto = $saudacao . '. ';
    $texto .= $mensagem;
    return $texto;
}
echo imprimir_mensagem('Seja Bem Vindo.');
```

// retorno: Bom dia. Seja Bem vindo.

```

echo imprimir_mensagem('Opa!', 'Boa noite');
```

// retorno: Boa noite. Opa!

```

echo imprimir_mensagem();
// erro: pois o primeiro parâmetro é obrigatório
?>
```

É possível analisar os códigos acima e verificar que os parâmetros passados a uma função respeitam muito o que já existe em diversas linguagens de programação. Os parâmetros são separados por vírgula e são citados com atribuição direta a nome de variáveis que estão perfeitamente legíveis no decorrer do escopo da função. Se for a uma variável, atribuído um valor padrão já na declaração da função, caso este parâmetro não seja fornecido ou seja informado NULL na chamada da função, a variável passará a utilizar este valor previamente definido.

Uma boa atividade agora seria criar uma função que receba quatro notas (variáveis) e retorne a média. O melhor de uma função é que pequenos pedaços são mais simples de serem escritos, entendidos, testados e logicamente desenvolvidos.

Sobre os parâmetros, gostaria de destacar que, assim como as variáveis, é possível informar um parâmetro como referência e, deste modo, atribuir a esta

variável informada o mesmo comportamento demonstrado anteriormente. Um exemplo simples para demonstrar:

```
<?php
$nome = 'eduardo BONA';
function normalizar_nome(&$novo_nome) {
    $novo_nome = ucwords(strtolower($novo_nome));
}
normalizar_nome($nome);
echo $nome;
?>
```

Resultado: Eduardo Bona

Note que diferentemente de uma função com um retorno (que é mais comum), geralmente funções que passam parâmetros por referência tem como objetivo principal alterar os valores destas variáveis já dentro da própria função. Um exemplo disso poderá ser visto em funções que realizam manipulação em arrays.



FUNÇÕES PARA PROCESSAMENTO DE TEXTO (STRINGS)

Agora que sabemos como funciona uma função e suas características como parâmetros e retorno, é fundamental conhecer o que a própria linguagem de programação já nos proporciona.

A lista mais extensa de funções do PHP estará relacionada ao processamento e tratamento de textos. Estas funções irão trabalhar diretamente com nossas famosas strings. Começando das mais óbvias, separei esta primeira lista para você:

Quadro 1 - Funções mais comuns do PHP para Strings

EXEMPLO	DESCRIÇÃO	RETORNO
<code>ucfirst("eduardo é meu primeiro nome")</code>	Retorna uma string com o primeiro caractere em letra maiúscula	Eduardo é meu primeiro nome
<code>ucwords("eduardo é meu primeiro nome")</code>	Retorna uma string com todas as primeiras letras das palavras em letras maiúsculas	Eduardo É Meu Primeiro Nome
<code>strtolower("Eduardo Bona")</code>	Retorna uma string com todas as letras minúsculas	eduardo bona
<code>strtoupper("Eduardo Bona")</code>	Retorna uma string com todas as letras maiúsculas	EDUARDO BONA

Fonte: o autor.

Estes primeiros exemplos demonstram o tratamento de textos que podem ser feitos por meio do PHP. Outras funções ainda irão permitir à análise e algumas transformações em strings:

Quadro 2 - Funções mais comuns para String com comportamento de análise

EXEMPLO	DESCRIÇÃO	RETORNO
<code>strlen("Eduardo Bona");</code>	Retorna a quantidade de caracteres presentes na string (inclui espaços e outros caracteres)	12
<code>str_replace("Eduardo", "Eduardo Herbert", "Eduardo Bona")</code>	Realiza a busca dentro de uma string "Eduardo Bona" que é o terceiro parâmetros, buscando por uma string chamada "Eduardo" que é o primeiro parâmetro e substitui por "Eduardo Herbert" que é o segundo parâmetro. Caso queira fazer alterações em lote, ao invés de passar strings, pode informar arrays.	Eduardo Herbert Bona
<code>strcmp("Eduardo Bona", "André Bona")</code>	Retorna menor que 0 se a primeira string é menor que a segunda string. Retorna >0 se a primeira for maior e retorna 0 se ambas forem iguais.	-1

EXEMPLO	DESCRIÇÃO	RETORNO
ex1: <code>strpos("Eduardo Bona", "Bona")</code> ex2: <code>strpos("Eduardo Bona", "André")</code> ex3: <code>strpos("Eduardo Bona", "Eduardo")</code>	Retorna a primeira posição encontrada conforme primeiro parâmetro informado e segundo parâmetro pretendido. Retorna false se não encontrar a busca.	ex1: 8 ex2: return false ex3: 0 (lembre-se que a primeira posição é 0 para esta função)
<code>strrev("Hello world!");</code>	Reverte uma string do fim para o início	<code>!dlrow olleH</code>

Fonte: o autor.

A lista de funções para strings é imensa e pode ser encontrada na documentação oficial, seu primeiro local para busca por referências e dúvidas (MANUAL DA PHP, [2018], on-line)¹.



REFLITA

Antes de desenvolver alguma função no PHP para resolver um problema seu, é sempre bom verificar se o PHP já não resolveu este problema e, em seguida, se alguém também não disponibilizou algum código na internet (github ou stackoverflow). Otimize seu tempo e aprenda a pesquisar.

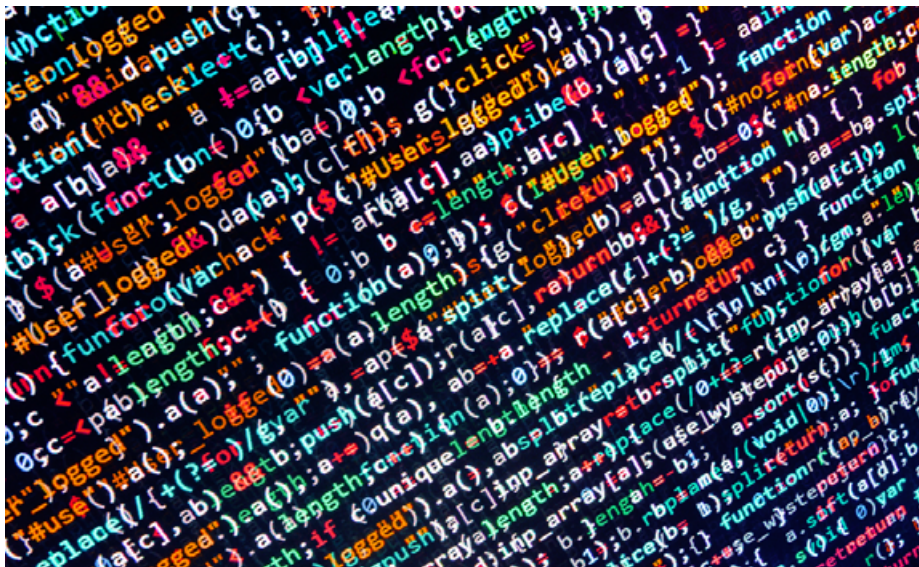
Para implementação de recursos de segurança, o PHP também fornece inúmeras funções para escape de caracteres especiais, filtros e criptografia. Estes recursos são muito comuns para o tratamento de strings tanto para salvar uma informação em banco de dados quanto imprimir uma mensagem no navegador (escape seguro):

Quadro 3 - Funções para escapes de texto

EXEMPLO	DESCRIÇÃO	RETORNO
<code>addslashes("Eduardo o' Bona")</code>	Adiciona barras invertidas a uma string. Estas barras não são perceptíveis no navegador e são um exemplo de como se devem ser salvos dados em um banco de dados, por exemplo.	Eduardo o\ Bona
<code>htmlentities("Eduardo > Bona")</code>	Converte todos os caracteres aplicáveis em entidades html deste modo uma saída incorreta não prejudica seu HTML, pois o navegador não interpretará HTML.	Eduardo>Bona
<code>strip_tags("<h1>Eduardo Bona</h1>")</code>	Retorna a string removendo qualquer codificação HTML (tags) existente. É possível adicionar como segundo parâmetro tags permitidas.	Eduardo Bona
<code>crypt('Eduardo Bona', 'hash-teste')</code>	Fará com que a string 'Eduardo Bona' seja criptografada com algoritmo Unix Standard DES com base na hash informada no segundo parâmetro.	haDBnyre3rJ5c
<code>md5('Eduardo Bona')</code>	Fará com que a string 'Eduardo Bona' seja criptografada em algoritmo MD5 que gera uma string de 32 caracteres.	61df83b5673ad47d0aabf-781078d021c
<code>sha1('Eduardo Bona')</code>	Fará com que a string 'Eduardo Bona' seja criptografada em algoritmo SHA1 que gera uma string de 41 caracteres.	3010ca904bb1399312a-71650b5e7abad0009fde3

Fonte: o autor.

Provavelmente as funções citadas nestas referências iniciais representam menos da metade das disponíveis para sua utilização. Além disso, como visto em funções definidas pelo usuário, você também poderá criar as suas, caso as que lá existam não lhe atendam.



FUNÇÕES MATEMÁTICAS

A matemática definitivamente se faz presente em nossa área com força total. E, em nossos programas, além da própria lógica de programação que exige uma boa matemática, também temos as funções matemáticas propriamente ditas. Estas funções têm objetivos mais claros e são até mais simples de serem utilizadas do que as funções para textos (strings).

Quadro 4 - Funções Matemáticas mais comuns

EXEMPLO	DESCRIÇÃO	RETORNO
ex1: abs(-1) ex2: abs(1)	Retorna o valor absoluto de um número inteiro (positivo ou negativo).	ex1: 1 ex2: 1
ex1: round(45.90) ex2: round(45.25) ex3: round(45.25, 1)	Retorna o valor arredondado com a precisão, caso seja informado o segundo parâmetro. Se não for informado, arredonda para inteiro.	ex1: 46 ex2: 45 ex3: 45.3
ex1: ceil(45.90) ex2: ceil(45.25)	Retorna o maior valor próximo inteiro arredondado.	ex1: 46 ex2: 46
ex1: floor(45.90) ex2: floor(45.25)	Retorna o menor valor anterior inteiro arredondado.	ex1: 45 ex2: 45

ex1: <code>max(1,4,2,5,2)</code> ex2: <code>max(array(1,4,2))</code>	Retorna o maior valor de uma lista ou de um array.	ex1: 5 ex2: 4
ex1: <code>min(1,4,2,5,2)</code> ex2: <code>min(array(1,4,2))</code>	Retorna o menor valor de uma lista ou de um array.	ex1: 1 ex2: 1
<code>rand(1, 100)</code>	Gera um número inteiro aleatório de acordo com o mínimo (primeiro parâmetro) e o máximo (segundo parâmetro).	ex: 55 ex: 33 ex: ...
<code>number_format(1500, 2, ",", ".")</code>	Transforma um número (primeiro parâmetro) em uma string com X casas decimais (segundo parâmetro) e com separador de decimal e milhar (terceiro e quarto parâmetro).	1.500,00

Fonte: o autor.

SAIBA MAIS



Apesar de não serem listadas aqui, as funções matemáticas do PHP também estão disponíveis para cálculos como seno, cosseno, tangente, raiz quadrada e números exponenciais. Teoricamente, tudo que uma calculadora científica tem o PHP tem também.

Fonte: o autor.

Chega a ser quase impossível memorizar tantas funções e, por isso, estou me concentrando com você apenas nas mais comuns em aplicações. Na dúvida, como de costume, pesquise a Documentação Oficial da PHP, na qual todas estas funções estão listadas, com exemplos e todo o material completo.

FUNÇÕES PARA MANIPULAÇÃO DE ARRAYS

Arrays ou vetores ou matrizes trata-se de um recurso comum em todas as linguagens de programação. No PHP, o array poderá representar tanto um vetor simples como uma sequência de valores bem como uma matriz mais complexa multidimensional.

Quadro 5 - Funções para Manipulação de Arrays

<code>array_chunk(\$array, \$tamanho)</code>	Transforma um array de N posições em um novo array dividido de acordo com o tamanho informado.
<code>array_combine(\$chaves, \$valores)</code>	Une duas variáveis do tipo array em uma nova variável fazendo com que valor presente em <code>\$chaves[0]</code> seja a chave para <code>\$valores[0]</code> .
<code>array_diff(\$a, \$b)</code>	Compara dois arrays e seu resultado será a diferença entre <code>\$a</code> e <code>\$b</code> .
<code>array_flip(\$array)</code>	Permuta todas as chaves e seus valores associados em um array. Deste modo, o que era chave, se torna valor e o que era valor, se torna chave.
<code>array_key_exists('nome', \$pessoa)</code>	Checa se uma chave ou índice existe em um array e retorna verdadeira caso exista.
<code>array_keys(\$array)</code>	Retorna todas as chaves ou uma parte das chaves de um array.
<code>array_map("strtoupper", \$nomes)</code>	Aplica uma função em todos os elementos dos arrays dados. Pode tanto ser uma função existente ou uma função nova criada.
<code>array_merge(\$array1, \$array2)</code>	Combina um ou mais arrays.
<code>array_pop(\$nomes)</code>	Extrai um elemento do final do array.
<code>array_push(\$nomes, "Eduardo Bona")</code>	Adiciona um ou mais elementos no final de um array.
<code>array_rand(\$nomes, 2)</code>	Escolhe um ou mais elementos aleatórios de um array.
<code>array_sum(\$numeros)</code>	Calcula a soma dos elementos de um array.
<code>array_unique(\$nomes)</code>	Remove o valores duplicados de um array.

<code>array_values(\$nomes)</code>	Retorna todos os valores de um array. Se houver índices customizados os mesmos serão perdidos e transformados em sequências como um novo array.
<code>range(1,10, 1)</code>	cria um array sequencial de acordo regra e limites informados sendo o primeiro e o segundo parâmetro o início e fim. O último parâmetro é o intervalo que se deseja entre um valor e outro podendo ser inteiro ou flutuante.

Fonte: o autor.

Como nem todo array é composto por índices automáticos, algumas das funções acima realizam atividades importantes no array, como a função que verifica se um índice existe bem como retornar todos os índices ou somente os valores.

Arquivo: `array_keys.php`

```
<?php
$automovel = array(
    'marca' => 'Ford',
    'modelo' => 'Focus',
    'ano' => 2017
);
if (array_key_exists('modelo', $automovel)) {
    echo "Existe a chave modelo.";
}
$automovel_chaves = array_keys($automovel);
var_dump($automovel_chaves);
$automovel_valores = array_values($automovel);
var_dump($automovel_valores);
?>
```

Resultado:

```
Existe a chave modelo.
array(3) {[0]=> string(5) "marca" [1]=> string(6) "modelo"
[2]=> string(3) "ano" }
array(3) {[0]=> string(4) "Ford" [1]=> string(5) "Focus" [2]=>
int(2017) }
```

Outras funções por outro lado vão permitir a manipulação adicionando, removendo e/ou tornando únicos os valores dentro de um array como no exemplo a seguir:

Arquivo: `array_manipulacao.php`

```
<?php
$nomes = array("Eduardo", "Danillo", "Rafael", "Eduardo", "Vic-
tor");
var_dump($nomes);
array_pop($nomes);
var_dump($nomes);
array_push($nomes, "Arthur");
var_dump($nomes);
$nomes_unicos = array_unique($nomes);
var_dump($nomes_unicos);
?>
```

Resultado:

```
array(5) { [0] => string(7) "Eduardo" [1]=> string(7) "Danil-
lo" [2]=> string(6) "Rafael" [3]=> string(7) "Eduardo" [4]=>
string(6) "Victor" }
```

```
array(4) { [0] => string(7) "Eduardo" [1]=> string(7) "Danillo"
[2]=> string(6) "Rafael" [3]=> string(7) "Eduardo" }
```

```
array(4) { [0] => string(7) "Eduardo" [1]=> string(7) "Danil-
lo" [2]=> string(6) "Rafael" [3]=> string(7) "Eduardo" [4]=>
string(6) "Arthur" }
```

```
array(5) { [0]=> string(7) "Eduardo" [1]=> string(7) "Danil-
lo" [2]=> string(6) "Rafael" [3]=> string(7) "Eduardo" [4]=>
string(6) "Arthur" }
```

```
array(4) {[0]=> string(7) "Eduardo" [1]=> string(7) "Danillo"
[2] => string(6) "Rafael" [4] => string(6) "Arthur" }
```

Com certeza existe para manipulação de arrays muito mais funções e propósitos além dos que você imagina. Por isso, primeiramente é importante a prática no livro e em seguida especificamente na Documentação Oficial do PHP na seção de Funções para Arrays.

Nem toda função para manipulação de array necessariamente se inicia pelo prefixo `array_`. Da lista de cinco funções destacadas no quadro, a seguir, as quatro primeiras funções são muito utilizadas durante a manipulação de arrays, principalmente para validação e tratamento destes dados.

Quadro 6 - Funções básicas para validação em arrays

count(\$cidades)	retorna a quantidade de registros presentes no array e caso seja uma matriz retornará apenas a quantidade de registros no principal
isset(\$cidades['Maringa'])	verifica se uma determinada chave de um array foi setada e retorna TRUE caso exista. Isso não chega a ser uma validação do valor, apenas da chave. Deste modo, pode ser que o valor atribuído seja NULL sem problemas.
unset(\$cidades['Maringa'])	remove a chave juntamente com seu valor de um array informado.
in_array('PR', \$estados)	verifica se um determinado valor informado está presente em um array passado para comparação.

Fonte: o autor.

Arquivo: array_validacao.php

```
<?php
$cidades = array('Maringa' => 1000, 'Sarandi' => 500);
echo "Quantidade de cidades: ";
echo count($cidades);
if (isset($cidades['Maringa'])) {
    echo " e Maringa é um índice na lista";
} else {
    echo " e Maringa não é um índice na lista";
}
// tirando Maringa da lista
unset($cidades['Maringa']);
if (isset($cidades['Maringa'])) {
    echo " e Maringa ainda é um índice na lista";
} else {
    echo " e Maringa já não é um índice na lista";
}
$estados = array("PR", "SP", "AM", "PA", "PI", "MG");
if (in_array('PR', $estados)){
    echo " e por fim PR está presente nos valores da lista de estados";
} else {
    echo " e PR não está na lista";
}
```

Resultado:

Quantidade de cidades: 2 e Maringa é um índice na lista e Mar-
inga já não é mais um índice na lista e por fim PR está presente
nos valores da lista de estados.

Importante para complementar este assunto, segue uma lista de algumas funções que ordenam um array. É fundamental que, nestes casos, você estude e aplique com calma cada uma das funções citadas, pois elas possuem características interessantes e, em alguns casos, resultados que, inicialmente, podem até parecerem iguais, mas, na prática, são totalmente diferentes e o uso incorreto trará consequências ao seu projeto.

Quadro 7 - Funções para ordenação de um array

sort(\$nomes) sort(\$nomes, SORT_REGULAR)	Os elementos serão ordenados do menor para o maior ao final da execução dessa função. O segundo parâmetro (opcional) é para determinar a forma como se deseja a ordenação e a lista de possibilidades é demonstrada no quadro abaixo sobre Short Flags.
rsort(\$nomes) rsort(\$nomes, SORT_REGULAR)	Assim como sort, porém Ordena um array em ordem decrescente
asort(\$nomes) asort(\$nomes, SORT_NATURAL)	Essa função ordena um array de forma que a correlação entre índices e valores é mantida. É usada principalmente para ordenar arrays associativos onde a ordem dos elementos é um fator importante.
arsort(\$nomes)	Ordena um array em ordem decrescente mantendo a associação entre índices e valores
ksort(\$estados) ksort(\$estados, SORT_STRING)	Ordena um array pelas chaves, mantendo a correlação entre as chaves e os valores. Essa função é bastante útil principalmente para arrays associativos. No caso, se usa krsort para obter o resultado inverso a esta função.
shuffle(\$nomes)	Mistura os elementos de um array

Fonte: Manual do PHP ([2018], on-line)¹.

Um exemplo do uso deste tipo de função para ordenação poderá ser visto em duas situações no exemplo a seguir:

Arquivo: array_ordenacao.php

```
<?php
$nomes = array ("Eduardo", "Rafael", "Arthur", "Danillo");
sort($nomes); // ordenando crescente
var_dump($nomes);
rsort($nomes); // ordenando decrescente
var_dump($nomes);
?>
```

Resultado:

```
array(4) {[0]=> string(6) "Arthur" [1]=> string(7) "Danillo"
[2]=> string(7) "Eduardo" [3]=> string(6) "Rafael"}
array(4) {[0]=> string(7) "Rafael" [1]=> string(7) "Eduardo"
[2]=> string(6) "Danillo" [3]=> string(9) "Arthur"}
```

Como visto acima, geralmente o segundo parâmetro da lista das funções de ordenação são as SORT_FLAGS, que definem qual algoritmo deve ser utilizado para ordenação. As constantes mostradas abaixo contemplam as possibilidades que você poderá fazer uso. Caso não informe nada à função de ordenação, o padrão será SORT_REGULAR.

Quadro 8 - Constantes para ordenação de arrays

SORT_REGULAR	Compara os itens normalmente (não modifica o tipo)
SORT_NUMERIC	Compara os itens numericamente
SORT_STRING	Compara os itens como strings
SORT_LOCALE_STRING	Compara os itens como strings, utilizando o locale atual. Utiliza o locale que pode ser modificado com setlocale()
SORT_NATURAL	Compara os itens como strings utilizando "ordenação natural" tipo natsort()
SORT_FLAG_CASE	Pode ser combinado (bitwise OR) com SORT_STRING ou SORT_NATURAL para ordenar strings sem considerar maiúsculas e minúsculas

Fonte: Manual do PHP ([2018], on-line)¹.

Se ainda assim, nenhuma das formas de ordenação lhe atender, é possível, com um pouco mais de estudo, criar sua própria função e utilizar, para isso, as funções de ordenação usort, uksort e uasort. Estas funções esperam seu algoritmo por parâmetro como funções e serão mais uma possibilidade de ordenação para seu projeto.



FUNÇÕES PARA SISTEMAS DE ARQUIVOS

Um grupo de funções altamente presente em aplicações web é o grupo das funções para manipulação de sistema de arquivos. Por exemplo, se você pretende realizar a submissão de um arquivo do seu computador para seu sistema (upload), irá recorrer a estes recursos. Em muitos momentos de uma aplicação você também será levado a entregar documentos para downloads ao cliente, até mesmo sendo preciso gerar alguns deles em alguns casos.

Veja no quadro, a seguir, algumas funções importantes para manipulação de arquivos, validação e informação dos mesmos:

Quadro 9 - Funções para Manipulação de Arquivos

<code>is_file(\$arquivo)</code>	Retorna TRUE se o caminho informado apontar para um arquivo ou FALSE caso contrário
<code>file_exists(\$arquivo)</code>	Retorna TRUE se o caminho informado for válido. Isso serve tanto para diretórios quanto para arquivos. É fundamental, por exemplo, para verificar se um diretório existe antes de mover um arquivo para o destino evitando erros.
<code>filesize(\$arquivo)</code>	Retorna o tamanho do arquivo em bytes ou FALSE em caso de erro.

pathinfo(\$arquivo)	Retorna um array separando a localização completa de um arquivo em diretório raiz (dirname), nome do arquivo com extensão (basename), extensão do arquivo (extension) e some nome do arquivo (filename).
mkdir(\$diretorio, 0777)	Cria um diretório com o nome informado. O segundo parâmetro é qual modo de permissão deseja dar para o arquivo. Para servidores compartilhados com outros clientes, é altamente inseguro utilizar 0777, pois dá permissão total ao diretório. Caso o caminho informado contenha mais de um diretório e alguns destes ainda não exista você pode informar no terceiro parâmetro a opção de recursividade e o PHP criará a árvore toda.
copy(\$origem, \$destino)	Faz uma cópia do arquivo com base na localização origem do arquivo para o destino.
rename(\$arquivo, \$novonome)	Renomeia um arquivo existente.
unlink(\$arquivo)	Apaga um arquivo. Para apagar um diretório use rmdir.

Fonte: Manual do PHP ([2018], on-line)¹.

```
<?php
```

```
// verifica se a pasta chamda testes existe
// se não existir ele vai criar a pasta testes
$pasta = 'testes';
if (file_exists($pasta)) {
    echo "Pasta testes existe<br />";
} else {
    echo "Pasta testes não existe <br />";
    $retorno = mkdir('testes', 0777);
    if (! $retorno) {
        echo "Erro ao criar pasta testes <br />";
        exit;
    }
}

// verifica se o arquivo meu_texto1.txt existe na pasta
// se existir ele vai copiar para meu_texto2.txt
$arquivol = $pasta . '/meu_texto1.txt';
$arquivol_copia = $pasta . '/meu_texto2.txt';
if (file_exists($arquivol)) {
    $retorno = copy($arquivol, $arquivol_copia);
    var_dump(pathinfo($arquivol_copia));
    if (! $retorno) {
        echo "<br />Erro ao copiar arquivo $arquivol_copia";
    }
}
```

```

    }
}

// se existir o arquivo meu_texto3.txt ele vai excluir
$arquivo3 = $pasta . '/meu_texto3.txt';
if (file_exists($arquivo3)) {
    if (unlink($arquivo3)) {
        echo "Arquivo excluído";
    } else {
        echo 'Erro ao excluir arquivo';
    }
}
}

```

Outra habilidade recorrentemente solicitada de desenvolvedores é a de criação de arquivos de texto e leitura dos mesmos. Materiais mais antigos do PHP fazem uso especialmente de funções como `fopen` que trabalham como ponteiros e leitores de arquivos. Hoje em dia, eu posso afirmar que boa parte do que precisamos basicamente, que é ler arquivos em strings e escrever strings em arquivos de texto, é possível ser feito mediante apenas duas funções: `file_get_contents` e `file_put_contents`. No quadro abaixo segue uma descrição de ambas e um exemplo bem simples.

Quadro 10 - Função para recuperar e salvar arquivos em PHP

<code>file_get_contents(\$arquivo)</code>	Retorna o conteúdo de um arquivo em uma string. O endereço informado também pode ser de uma URL e a depender das permissões do servidor destino ele entregará as informações desejadas.
<code>file_put_contents(\$arquivo, \$dados)</code>	Salva dentro de um arquivo existente ou novo os dados informados por parâmetro. <code>file_put_contents</code> tem em uma única chamada equivalência com chamar a função <code>fopen()</code> , <code>fwrite()</code> e <code>fclose()</code> .
<code>file_put_contents(\$arquivo, \$dados, FILE_APPEND)</code>	O terceiro parâmetro (opcional) é uma flag e indica como você deseja que seja feita a execução da operação. A constante <code>FILE_APPEND</code> por exemplo permite que seja adicionado no final do arquivo o texto. Caso não seja informado nada o texto irá sobrescrever o já existente.

Fonte: o autor.

Arquivo: exemplo_file_get_put_contents.php

```
<?php
$arquivo = 'texto.txt';

if (file_exists($arquivo)){
    echo "Arquivo Original = " . file_get_contents($arquivo);

    echo "<br />Adicionando conteúdo dentro do arquivo";
    $dados = "a disciplina de Programação Back End I";
    if(file_put_contents($arquivo, $dados, FILE_APPEND)){
        echo "Arquivo Modificado = " . file_get_contents($arquivo);
    }else{
        echo "Erro ao editar o arquivo";
    }
}
```

Resultado: (obtido na primeira execução pressupondo que o arquivo texto.txt tivesse somente o texto "abc" dentro dele).

Arquivo Original = abc

a disciplina de Programação Back End I

Das funções existentes no PHP e presentes no Manual do PHP, os quadros demonstrados até agora representam uma pequena parte de todo o repertório do PHP para manipulação de arquivos, porém, são as principais que eu procurei destacar para você e tenho certeza que, para seu primeiro contato, são as mais relevantes.

Com relação ao processo de upload e o fluxo necessário, posteriormente será demonstrado através de um estudo de casos esta questão.

CONSIDERAÇÕES FINAIS

Dentro do contexto de uma linguagem de programação provavelmente o que mais irá distinguir a linguagem A da linguagem B serão as funções. Digo isso porque a sintaxe, por exemplo, por mais que tenha suas particularidades, continuará tendo os mesmos recursos como condicionais e laços de repetição quem não mudam quase nada.

Porém, as funções de uma linguagem de programação são como aquela mala de ferramentas que cada profissional leva consigo para realizar um trabalho contendo inúmeros itens à disposição e utilizado de diversas maneiras em cada projeto. As funções são como estes itens de uma mala de ferramentas: sempre estarão lá a sua disposição para cada projeto. O mais importante de tudo é que você saiba sempre quais itens estão à sua disposição e, se possível, que tenha conhecimento necessário para usá-los.

As funções apresentadas aqui são aquelas que todo programador fará uso em um curto espaço de tempo e ainda existem outras tantas que você precisará estudar no Manual do PHP para seus projetos. Funções para manipulação de data e hora ou funções para manipulação de imagens, por exemplo, são alguns dos casos que demandarão de você novos estudos. Mais uma vez: um bom programador sempre está se atualizando e, muitas vezes, se reciclando.

As funções dentro de uma linguagem têm o objetivo de facilitar a sua vida e poupar seu trabalho. Por isso, tente não reinventar a roda: antes de desenvolver qualquer função procure saber se ela já não existe. Muitas vezes, pode ser que ela não exista no PHP, mas você poderá encontrar alguém que já teve seu problema e desenvolveu uma função para isso. Provavelmente, isso acontecerá mais do que você pensa.

Eu já fiz algumas coisas que depois de anos descobri que já existia pronto por puro desconhecimento das funções do PHP. Não sei se não tive este ensinamento ou se não ouvi corretamente, mas deixo para vocês esta preciosa dica.

Bons estudos!

ATIVIDADES



1. Supondo a criação de uma variável chamada \$nome com o valor "eduardo bona" qual das funções abaixo transformaria "eduardo bona" em "EDUARDO BONA"
 - a) ucwords
 - b) ucfirst
 - c) strtolower
 - d) strtoupper
 - e) ucall
2. Imaginando a criação de uma variável chamada \$dados do tipo array com três registros dentro dela, qual função do PHP retornaria o valor 3 após acionada informando a variável \$dados?
 - a) total
 - b) sum
 - c) quantidade
 - d) count
 - e) get
3. Crie uma função chamada validar_senha que receba um único parâmetro chamado \$senha e que realize a validação:
 - se o tamanho da string \$senha for maior ou igual a 8 retorne TRUE
 - senão, retorne FALSE

4. Analise o código abaixo:

```
<?php
$dados = array(
    "9.0 Sobre o PHP", "10.0 Conclusão", "8.0 Sobre o HTML",
    "1.0 Introdução"
);
sort($dados);
var_dump($dados);
?>
```

Somente a função `sort($dados)` não será capaz, sozinha, de ordenar este array de modo a produzir o resultado indo do 1.0, 8.0, 9.0 e 10.0.

Qual parâmetro poderá ser adicionado nesta função `sort` que tornará o resultado, de fato, ordenado da maneira como esperada (1.0, 8.0, 9.0 e 10.0)?

ATIVIDADES



5. Desenvolva uma função chamada `remover_arquivo` que receba o nome de um arquivo (parâmetro `$arquivo`) e em seguida:

- verifique se o arquivo existe
- caso o arquivo não exista, retorne `FALSE`
- caso o arquivo exista, exclua-o e retorne `TRUE`

6. Analise o código abaixo:

```
<?php
$campeoes_mundiais = array(
    "Santos", "Flamengo", "Grêmio",
    "São Paulo", "Internacional", "Corinthians"
);

$time = "São Paulo";

if (__qual_funcao__($time, $campeoes_mundiais)) {
    echo "Seu time foi campeão mundial";
} else {
    echo "Seu time não foi campeão mundial";
}
```

Qual função deveria ser utilizada na estrutura condicional de modo a validar se a variável `time` passada como primeiro parâmetro existe no array de lista de times campeões mundiais?



De alguns anos para cá, um assunto que tem tomado conta de eventos e equipes são os testes. Isso porque não há motivos para desenvolvermos códigos que não estejam devidamente testados. Desenvolvedores e empresas ao longo de muito tempo têm trabalhado forte para criar uma cultura de testes, bem como ferramentas que auxiliem na execução, na criação e na apresentação destes testes.

Principalmente programadores no início de carreira tem uma certa dificuldade em assimilar isso, pois acabam pensando: por que gastar oito horas em uma tarefa que, em tese, levaria apenas quatro? Mas se esquecem que tarefas sem teste possuem custos posteriores ao projeto como correções para *bugs*, ou manutenções que podem impactar em outros códigos que não estando testados vão causar novos *bugs*, etc. E um *bug*, que é um erro, seja ainda em ambiente de desenvolvimento ou, no pior dos casos, em ambiente de produção, não é fácil de mensurar seu valor. O custo de um erro pode ser, no melhor dos casos, o tempo (hora) do desenvolvedor para arrumar, mas, em outros, pode ser uma invasão ao sistema, o término de um contrato, o vazamento de dados sigilosos ou até uma grande perda monetária - quando o sistema "bugado" faz transações monetárias.

Para entender mais sobre o assunto, sugiro a leitura a seguir.

Testes

Escrever testes automatizados para o seu código PHP é considerado uma boa prática, e leva a aplicações bem escritas. Testes automatizados são uma excelente ferramenta para garantir que sua aplicação não irá quebrar quando você estiver fazendo alterações ou adicionando novas funcionalidades, e não deveriam ser ignorados.

Existem vários tipos diferentes de ferramentas de testes (ou frameworks) disponíveis para PHP, com diferentes abordagens: todas elas tentam evitar os testes manuais e a necessidade de equipes grandes de Garantia de Qualidade Quality Assurance, ou QA) apenas para garantir que alterações recentes não interrompam funcionalidade existentes.

Desenvolvimento Guiado por Testes

O desenvolvimento guiado por testes (TDD) é um processo de desenvolvimento que se baseia na repetição de um ciclo de desenvolvimento bem curto: primeiro o desenvolvedor escreve um caso de teste automatizado que falha, definindo uma melhoria ou uma nova função desejada, em seguida produz o código para passar no teste, e finalmente refatora o novo código pelos padrões aceitáveis. Kent Beck, que é creditado como quem desenvolveu ou "redescobriu" essa técnica, afirmou em 2003 que o TDD encoraja design simples e inspira confiança.

Existem vários tipos diferentes de testes que você pode fazer para sua aplicação.

Testes Unitários

Testes unitários são uma metodologia de programação que garante que as funções, as classes e os métodos estão funcionando como esperado, desde o momento que você





os constrói até o fim do ciclo de desenvolvimento. Verificando como os valores entram e saem em várias funções e métodos, você pode garantir que a lógica interna está funcionando corretamente. Utilizando Injeção de Dependências e construindo classes "mock" e stubs, você pode verificar se as dependências foram utilizadas corretamente para uma cobertura de testes ainda melhor.

Quando for criar uma classe ou função, você deveria criar um teste unitário para cada comportamento que ela deveria ter. Em um nível bem básico, você deveria garantir que são emitidos erros quando você envia argumentos errados e garantir que tudo funciona bem se você enviar argumentos válidos. Isso ajudará a garantir que, quando você alterar sua classe ou sua função posteriormente no ciclo de desenvolvimento, as funcionalidades antigas continuarão funcionando como esperado. A única alternativa a isso seria usar `var_dump()` em um `test.php`, o que não é o certo a fazer na construção de uma aplicação - grande ou pequena.

O outro uso para testes unitários é contribuir para projetos open source. Se você puder escrever um teste que demonstra uma funcionalidade incorreta (i.e. uma falha), em seguida consertá-la e mostrar o teste passando, os patches serão muito mais suscetíveis a serem aceitos. Se você estiver em um projeto que aceite pull requests, você deveria sugerir isso como um requisito.

O PHPUnit é o framework de testes de fato para escrever testes unitários em aplicações PHP, mas existem várias alternativas:

- SimpleTest
- Enhance PHP
- PUnit
- atoum

Testes de Integração

Testes de integração (chamado às vezes de "Integração e Teste", abreviado como "I&T") é a fase do teste do software na qual módulos individuais do sistema são combinados e testados como um grupo. Ela acontece após os testes unitários e antes dos testes de validação. Os testes de integração recebem como entrada os módulos que foram testados unitariamente, os agrupa em grandes blocos, aplica testes definidos em um plano de teste de integração, e entrega como saída o sistema integrado pronto para os testes de sistema.

Muitas das mesmas ferramentas que são usadas para testes unitários podem ser usadas para testes de integração, pois muitos dos mesmos princípios são usados.

Testes Funcionais

Algumas vezes conhecidos também como testes de aceitação, os testes funcionais consistem em utilizar ferramentas para criar testes automatizados que usem de verdade sua





aplicação, em vez de apenas verificar se unidades individuais de código se comportam adequadamente ou se essas partes conversam entre si do jeito certo. Essas ferramentas geralmente trabalham usando dados reais e simulam usuários verdadeiros da sua aplicação.

Ferramentas para Testes Funcionais

- Selenium
- Mink
- O Codeception é um framework de testes full-stack que inclui ferramentas para testes de aceitação
- O Storyplayer é um framework de testes full-stack que inclui suporte para criação e destruição de ambientes sob demanda

Fonte: PHP do jeito certo. Disponível em: <http://br.phptherightway.com/#desenvolvimento_guiado_por_testes>.





LIVRO

PHP para quem conhece PHP

Juliano Niederauer

Editora: Novatec

Sinopse: Apresenta recursos avançados desta poderosa linguagem de programação para a Web. Aborda diversos assuntos úteis ao desenvolvedor, como cookies e sessões, upload de arquivos, geração de imagens e gráficos, arquivos PDF, templates, abstração de bancos de dados, entre outros. Além disso, contém uma abrangente revisão sobre PHP, para aqueles que tiveram pouco contato com a linguagem.

Comentário: Este livro traz conceitos mais aprofundados sobre a utilização do PHP e também contempla um estudo de casos ao final do livro bem interessante. É indicado para quem já está se sentindo confortável com a linguagem ou já conheça um pouco de PHP.





NA WEB

Stackoverflow

O portal Stackoverflow provavelmente estará presente na maioria das dúvidas que você tiver e realizar uma busca no Google. É um sistema colaborativo e comunitário. Você pode utilizá-lo para fazer perguntas e também consultar respostas para dúvidas que outras pessoas já tiveram e é uma boa fonte de conhecimento.

Acesse: <<http://stackoverflow.com>>

PHP Classes

Apesar de você ainda não ter iniciado seus estudos em orientação a objetos, este portal que, se chama PHP Classes, possui uma imensa biblioteca de Classes e Funções já desenvolvidas e classificadas por categoria. Também é uma excelente fonte de pesquisa.

Acesse: <<https://www.phpclasses.org>>

Github

Por fim, o maior e mais completo repositório de códigos-fonte do mundo. No Github é possível encontrar diversos códigos e projetos já desenvolvidos e com o código aberto para sua consulta e utilização. Se ainda não tem cadastro, recomendo que o faça já e aprenda a utilizar os repositórios do GitHub e os comandos GIT.

Acesse: <<http://www.github.com>>

REFERÊNCIAS

MANUAL DO PHP. **Funções para string**. Disponível em: <http://php.net/manual/pt_BR/ref.strings.php> Acesso em: 01 dez. 2017.

MANUAL DO PHP. **Funções Matemáticas**. Disponível em: <http://php.net/manual/pt_BR/ref.math.php>. Acesso em: 01 dez. 2017.

MANUAL DO PHP. **Funções para arrays**. Disponível em: <https://secure.php.net/manual/pt_BR/ref.array.php>. Acesso em: 01 dez. 2017.

MANUAL DO PHP. **Função sort**. Disponível em: <https://secure.php.net/manual/pt_BR/function.sort.php>. Acesso em: 01 dez. 2017

MANUAL DO PHP. **Funções para Sistema de Arquivos**. Disponível em: <http://php.net/manual/pt_BR/ref.filesystem.php>. Acesso em: 19 março 2019.

PHP. **Do jeito certo**. Disponível em: <http://br.phptherightway.com/#desenvolvimento_guiado_por_testes>. Acesso em: 01 dez. 2017.

REFERÊNCIA ON-LINE

¹ Em: <https://secure.php.net/manual/pt_BR/ref.filesystem.php>. Acesso em: 15 fev. 2018.

1. D) strtoupper

strtoupper retorna a string informada por parâmetro convertida em letras maiúsculas enquanto strtolower retorna o contrário, em minúsculas. ucfirst e ucwords tratam da questão voltada a primeira letra da string ou da palavra.

2. C) count

Esta função count retorna a quantidade de elementos de um array.

3.

```
function validar_senha($senha) {  
    if (strlen($senha) >= 8) {  
        return true;  
    }else{  
        return false;  
    }  
}
```

4.

sort(\$dados, SORT_NATURAL);

ou sort(\$dados, SORT_NUMERIC);

Como por padrão o PHP realiza SORT_REGULAR ele acaba entendendo que 1.0 e 10.0 por começarem com o número 1, são menores que os demais itens que começam com 8 e 9, ou seja, ele realiza, por padrão uma ordenação caracter a caracter.

Quando alteramos o comportamento padrão para NATURAL ou NUMERIC ele passa a exercer outro tipo de ordenação. No caso da atividade, por se tratar de números, recomendaria SORT_NUMERIC mas funcionaria também com SORT_NATURAL.

5.

```
function remover_arquivo($arquivo) {  
    if (file_exists($arquivo)) {  
        unlink($arquivo);  
        return true;  
    } else {  
        return false;  
    }  
}
```

6. in_array

A função in_array valida se um valor existe dentro de um array. Você não pode confundir in_array com isset pois isset é usada para validar se um índice foi setado. No caso do array citado na questão, os nomes dos times são valores. Esta função é muito utilizada e por isso se você ainda não conhecia é importante que pratique muitas atividades com ela.

LABORATÓRIO PHP



Objetivos de Aprendizagem

- Demonstrar a organização de códigos através de particionamento do código, organização em pastas e inclusões através de recursos do PHP.
- Exemplificar casos de uso onde valores são recebidos por meio da URL e de formulários, permitindo aplicação de validações e demonstrar como o recurso de redirecionamento do PHP através da função header.
- Apresentar de maneira prática como funciona o upload de um ou vários arquivos e como o PHP recebe os dados, bem como o processo de manipulação de arquivos.
- Demonstrar o envio de email com texto e HTML através do PHP.

Plano de Estudo

A seguir, apresentam-se os tópicos que você estudará nesta unidade:

- Organização e Reutilização de códigos PHP com include e require
- Validação de dados e redirecionamento entre as páginas
- Upload e Manipulação de Arquivos com formulários
- Envio de Email

INTRODUÇÃO

A prática leva à perfeição! Lembre-se sempre disso. Muitos consideram o desenvolvimento de software como uma ciência e outros mais criativos como uma arte. Eu prefiro o desenvolvimento de software como um esporte que pode ser coletivo ou não. Deste modo, a parte técnica é importante, a parte prática que pode ser como os treinamentos também são importantes, mas nada como um jogo de verdade para que as coisas rodem de verdade.

A esta altura, depois de tantos estudos em teorias e questões práticas, precisamos de estudos de casos mais completos e até um pouco mais complexos não é mesmo? É a hora de praticar de verdade, caso ainda não tenha praticado anteriormente. Por isso, a seguir serão apresentados seis estudos de casos diferentes, utilizando conteúdos já verificados anteriormente e em outros casos com utilização de códigos inéditos até o momento, como o caso de manipulação de imagens e envio de emails.

Você aprenderá a utilizar os conceitos de inclusões e requisições do PHP e isto permitirá além da organização do seu código um melhor reaproveitamento do mesmo.

Para quem desenvolve aplicações web ou até mesmo sistemas internos, tanto a validação de dados, que podem vir da URL ou formulários, é muito importante, além do mecanismo de upload e manipulação de arquivos. O tratamento de imagens será essencial para sua aplicação e para seu dia a dia, pois se tornará rotina a utilização destes recursos.

Ao término destes estudos, a se somar com o conhecimento adquirido anteriormente, será possível você desenvolver seus primeiros scripts e mini-aplicações em PHP, como um formulário de contato que envia para um email ou um upload de imagem com redimensionamento.

Assim como em esportes coletivos, o que posso desejar para você neste momento é uma boa leitura e a aplicação prática de todos estes conceitos até que comecem a se tornar rotina em sua vida acadêmica e profissional.

Bons estudos!



ORGANIZAÇÃO E REUTILIZAÇÃO DE CÓDIGOS PHP COM INCLUDE E REQUIRE

A linguagem de programação PHP é vista por muitos como uma linguagem de script que possibilita o processamento e o retorno de respostas HTML de maneira simples. Nem tudo é um mar de rosas e muitas das críticas que o PHP recebe vem justamente da liberdade que ele dá para os desenvolvedores e da maldade que estes desenvolvedores exercem em virtude desta liberdade. Esta maldade - que eu citei acima - é com o código, com o projeto.

A liberdade que o PHP dá é imensa e se você não for um desenvolvedor organizado provavelmente seu projeto será "à moda da casa" como costumamos ver em pizzarias por todo o Brasil.

Preciso defender a linguagem PHP e dizer que isso não é culpa da linguagem e sim dos desenvolvedores e que a liberdade que o PHP dá, não pode ser visto como negativa, pois permite que desenvolvedores em pouco tempo comecem a desenvolver projetos web.

E fique tranquilo! O PHP possui recursos suficientes para organizar seus códigos. Nosso intuito aqui não é demonstrar arquiteturas ou padrões de projetos - o que recomendarei como leitura complementar - mas sim fornecer um pouco da visão de organização em um projeto e das possibilidades ainda em projetos pequenos.

Baseado na lista a seguir, faça um trabalho mental e indique dentre as possibilidades quais dos pontos citados são mais importantes para você:

- Fácil localização dos arquivos.
- Separação das regras de negócios (lógica) da interface (HTML).
- Padrão para nome dos arquivos.
- Padrão para nome das funções.
- Padrão para estilo de código.
- Separação do layout.
- Separação de partes do layout (templates).
- Fácil acesso e localização a dados de configuração ou constantes.
- Reaproveitamento de funções (e códigos PHP).
- Reaproveitamento de layouts.

Mentalmente, você conseguiu classificar na lista o que é mais importante para você? Provavelmente sim, acredito eu. Mas... é possível a lista acima apontar que algum destes itens citados não seja importante? Provavelmente não (e assim espero!).

Esta pequena lista exemplifica itens que desde o início podemos pensar em organizar e facilitar para nosso projeto.

As funções que o PHP inicialmente fornece para a organização de nosso código é a **include** e a **require**.

Estas funções irão permitir que nós possamos realizar a importação, por meio de inclusão ou requisição, de qualquer código PHP para dentro de nosso código e, assim, possamos organizar nosso código em pedaços menores. Segundo a Documentação Oficial ([2018], on-line)¹,

os arquivos são incluídos baseando-se no caminho do arquivo informado ou, se não informado, o `include_path` especificado. Se o arquivo não for encontrado no `include_path`, a declaração `include` irá checar no diretório do script que o executa e no diretório de trabalho corrente, antes de falhar. O construtor `include` emitirá um aviso se não localizar o arquivo; possui um comportamento diferente do construtor `require`, que emitirá um erro fatal.

Vamos então começar nosso projeto!

Ao longo deste estudo, irei mostrar passo a passo a criação deste mini-projeto. Caso tenha alguma dúvida ou queira já conhecer o projeto de maneira macro o seu código se encontra no GitHub no endereço: <<https://github.com/eduardobona/livro-backend1-2017>>.

Antes de mais nada, é preciso criarmos uma pasta para nosso projeto em nosso servidor web e criar o arquivo `index.php` que é responsável pelo primeiro acesso ao nosso projeto. Por isso, apenas para fins de conferência vamos ao primeiro passo.

Passo 1: criar dentro da pasta `htdocs` uma pasta chamada *grupobona* e um arquivo chamado `index.php` com o seguinte código:

Arquivo: `index.php`

```
<?php
    echo "Olá mundo!";
?>
```

Em seguida, vamos testar este endereço acessando pelo navegador a URL: <<http://localhost/grupobona/index.php>>.

Se aparecer uma mensagem "Olá Mundo!" quer dizer que está tudo correto. Do contrário, reveja o serviço do servidor web Apache.

Conforme os requisitos do projeto, nosso sistema possuirá em sua primeira versão três telas, sendo a abertura, o formulário (após escolhida a área) e a mensagem de confirmação.

Para as três telas, é possível identificar que a identidade visual principal (cabeçalho e rodapé) são os mesmos para todas as telas, ou seja, se repetem e só é alterada a parte central da interface, o famoso "miolo". Com isso, podemos pensar em reaproveitar este código que vou chamar de **layout**. O layout como podemos ver é composto do cabeçalho e do rodapé e, por isso, vamos então organizá-los em dois arquivos que serão devidamente reaproveitados e que estão dentro de uma pasta chamada `layout`.

Passo 2: diagramar a página inicial (primeira tela) somente em html para se ter uma ideia do código e substituir no arquivo `index.php`

Arquivo: `index.php`

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8" />
    <title>Grupo Bona - Currículo Online</title>
    <link href="css/estilo.css" type="text/css" />
  </head>
  <body>
    <div class="central">
      <div class="cabecalho">
        
      </div>
      <div class="principal">
        <div class="lista-empresas">
          <h3>Escolha para qual empresa do grupo deseja enviar
seu currículo:</h3>
          <ul>
            <li><a href="curriculo.php">GB EAD</a></li>
            <li><a href="curriculo.php">GB Palestras</a></li>
          </ul>
        </div>
      </div><!-- fim principal -->
    </div><!-- fim central -->
  </body>
</html>
```

É possível notar no arquivo uma chamada para um arquivo do tipo CSS, que é responsável por aplicar estilos à página. O arquivo completo estará disponível para download junto ao link do estudo de caso completo e irei recomendar que você estude sobre folhas de estilo CSS em dois portais muito conceituados que adicionei como leitura complementar, o W3Schools e o MDN Mozilla Developers Network.

Analisando o código acima já é possível também concluir que podemos reaproveitar dois pedaços de código pois o cabeçalho compreende o código inicial HTML até a tag <div> que implementa o logotipo e que irá se repetir em todas as demais páginas e o rodapé compreende somente o fechamento da <div> principal e o restante do código HTML. No nosso estudo de caso, o rodapé acaba sendo bem mais simples, mas ainda assim é bom separar o código e reaproveitar. Por isso, mãos à obra!

Passo 3: criar uma pasta chamada layout e, dentro desta pasta, dois arquivos, sendo um chamado cabecalho.php e outro chamado rodape.php. Os conteúdos de ambos os arquivos ficarão conforme demonstrado a seguir:

Arquivo: layout/cabecalho.php

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8" />
    <title>Grupo Bona - Currículo Online</title>
    <link href="css/estilo.css" type="text/css" />
  </head>
  <body>
    <div class="central">
      <div class="cabecalho">
        
      </div>
      <div class="principal">
```

Arquivo: layout/rodape.php

```
      </div><!-- fim principal -->
    </div><!-- fim central -->
  </body>
</html>
```

E com isso, agora vem a parte mágica desta primeira atividade nossa que é como utilizar as funções do PHP para reaproveitar estes códigos utilizando a função **include**. Vamos atualizar agora nosso arquivo index.php reaproveitando os pedaços de código HTML criados anteriormente.

Arquivo: index.php

```
<?php
  include 'layout/cabecalho.php';
?>

  <div class="lista-empresas">
    <h3>Escolha para qual empresa do grupo deseja enviar
    seu currículo:</h3>
    <ul>
      <li><a href="curriculo.php?codigo=1">GB EAD</a></li>
      <li><a href="curriculo.php?codigo=2">GB Palestras</li>
    </ul>
  </div>

<?php
  include 'layout/rodape.php';
?>
```

Com a utilização da função include agora podemos para todas as outras páginas apenas incluir o arquivo cabeçalho.php e rodape.php e, deste modo, economizar boas linhas de código HTML, além de que com a utilização deste recurso caso seja necessário alterar algum trecho do cabeçalho ou do rodapé só iremos necessitar tal alteração em apenas um arquivo. Interessante não é? Acompanhe o resultado acessando sempre a URL do projeto pelo navegador. Está ficando interessante, não é?

REFLITA



A organização do código, assim como do projeto, é uma preocupação que precisa vir desde o início da construção do trabalho. Lembre-se que é mais fácil aumentar o tamanho de um banheiro antes da casa ficar pronta. Seu código funciona assim também.

VALIDAÇÃO DE DADOS E REDIRECIONAMENTO ENTRE AS PÁGINAS



Partindo sempre do requisito do projeto, para cada empresa do Grupo Bona existem departamentos específicos que devem receber em seus emails os currículos enviados. Podemos concluir, então, que nem sempre um departamento dentro de EAD estará dentro de Pós-Graduação e, por isso, vamos precisar receber esta escolha feita na

primeira página dentro da segunda página que é a responsável por mostrar o formulário do envio do currículo.

Antes disso, vamos precisar tornar ainda melhor as coisas: deixar esta informação centralizada e organizada. Como ainda não temos utilização de banco de dados, vamos utilizar um array em uma matriz de dados e, mais uma vez, contar com a ajuda do nosso include. Veja esta preparação:

Passo 4: criar a pasta dados e dentro dela construir arquivo chamado empresa_departamento.php com uma matriz de dados em array

Arquivo: dados/empresa_departamento.php

```
<?php
return array(
    1 => array(
        'empresa' => 'GB EAD',
        'departamentos' => array(
            'autor' => 'autor@gb.com.br',
            'aulas' => 'aulas@gb.com.br',
            'comercial' => 'comercial@gb.com.br',
        )
    ),
    2 => array(
        'empresa' => 'GB Palestras',
        'departamentos' => array(
            'comercial' => 'comercial@gb.com.br',
            'eventos' => 'eventos@gb.com.br'
        )
    )
);
?>
```

Com estes dados estruturados, podemos agora realizar duas tarefas importantes em nosso projeto, sendo a primeira construir nosso formulário de maneira dinâmica na hora de apresentar os departamentos e a segunda seria validar se o parâmetro informado (código da empresa) de fato existe. Com base nos dados, vemos que código 1 é GB EAD e código 2 é GB Palestras e qualquer coisa diferente disso não representaria uma empresa do Grupo Bona e, deste modo, deveríamos validar e devolver o usuário para a página principal.

Parâmetros incorretos em projetos web são conseguidos geralmente de duas formas: erro de código ou uso mal-intencionado de usuários. Por isso, tratar isto passa a ser tarefa fundamental para todo desenvolvedor evitando brechas de segurança.

Vamos ao código da segunda tela então, aquela que exibe o formulário:

Passo 5: criar o arquivo `curriculo.php` e construir o formulário de envio do currículo

Arquivo: `curriculo.php`

```
<?php
    include 'layout/cabecalho.php';
    $empresa = $_GET['codigo'];
?>

    <div class="formulario-curriculo">
        <h3>Envie seu currículo com foto:</h3>
        <form action="enviar_curriculo.php" method="post">
            <label>Nome:</label>
            <input type="text" name="nome" /><br />
            <label>Email:</label>
            <input type="email" name="email" /><br />
            <label>Departamento:</label>
            <select name="departamento">

<?php
    $empresa_departamento = include 'dados/empresa_departamento.
php';
    $departamentos = $empresa_departamento[$empresa]['departamen-
tos'];
    foreach ($departamentos as $indice => $departamento) {
?>

        <option value="<?php echo $indice?>"><?php echo
ucfirst($indice) ?></option>

<?php
    }
?>

        </select><br />
        <label>Foto:</label>
        <input type="file" name="foto" /><br />
        <label>Currículo:</label>
        <input type="file" name="curriculo" /><br />
        <input type="hidden" name="empresa" value="<?php
echo $empresa?>" />
        <input type="submit" value="Enviar" />
        </form>
    </div>

<?php
    include 'layout/rodape.php';
?>
```

Nesta segunda tela já é possível verificar que o código HTML se mistura bastante com o código PHP justamente porque utilizamos as informações do array de departamentos por empresa para construir nosso elemento `<select>` que fará a lista de departamentos que podem ser escolhidos para enviar um currículo.

De destaques neste ponto, é possível notar que:

- O layout do cabeçalho e do rodapé continua a ser reaproveitado;
- A variável `$empresa` recebe uma informação vinda de parâmetros do tipo GET sem qualquer tipo de validação ou filtragem;
- A variável `$empresa_departamento` recebe um arquivo vindo de um include. Ela recebe justamente porque o arquivo a ser incluído faz um return (como o de funções) e por isso pode ser recebido por uma variável. É um recurso muito utilizado atualmente;
- A construção do elemento `<select>` usa o índice do laço de repetição tanto para construir o seu valor como para a sua descrição;
- A descrição do elemento de `<select>` `<option>` utiliza a função `ucfirst` que no PHP faz com que as primeiras letras das palavras fiquem maiúsculas;
- O elemento `<input>` do tipo "hidden" está sendo utilizado para enviar uma informação não visível para o usuário junto com o formulário.

Mas... e a tal validação? Nada foi validado e no código existe um problema pois a variável `$empresa` recebe `$_GET['codigo']` e usa este valor depois para compor a lista, mas sabemos que da forma como foi feito só funcionaria sendo passado valor 1 ou 2.

Correto! Se você pensou assim e viu ali um problema está corretíssimo! Deixei para um segundo momento a validação, pois o código desta página já está grande o suficiente e, em um segundo momento, iremos separar esta regra de negócios.

Passo 6: criar um arquivo dentro da pasta "funcoes" chamado validadores.php e, nele, criar uma função que recebe um código e valida se o mesmo existe ou não, chamada `validar_empresa`. Em seguida, utilizar esta função para validar se um código foi informado corretamente e, caso não tenha sido, interromper o código causando um redirecionamento através da função `header`.

Lembrando que segundo o Manual do PHP ([2018], on-line), "qualquer código PHP válido pode aparecer dentro de uma função, mesmo outras funções e definições de classes" e por isso usaremos nossas funções com o principal objetivo de estruturar o código e sua responsabilidade em arquivos menores, facilitando a manutenção também.

Arquivo: `funcoes/validadores.php`

```
<?php
function validar_empresa($codigo){
    $empresa_departamento = include(__DIR__ . '/../dados/empresa_
    departamento.php');
    if (isset ($empresa_departamento[$codigo])) {
        return true;
    } else {
        return 'Código da empresa não foi encontrado';
    }
}
?>
```

Arquivo: `curriculo.php` (manutenção no primeiro bloco)

```
<?php
include 'funcoes/validadores.php';

$empresa = (int) $_GET['codigo'];
if (validar_empresa($empresa) == false) {
    header('location: index.php?erro=erro');
    exit();
}

include 'layout/cabecalho.php';
?>
```

A se destacar nos últimos códigos:

- A ideia do arquivo `validadores.php` é compor todos os validadores necessários para o projeto posteriormente;
- A utilização de `include` dentro da função `validadores` e da utilização da constante `__DIR__` é de obter a posição exata do diretório daquele arquivo e por isso, para incluir os dados é necessário descer um diretório para em seguida buscar a pasta `dados`. A descida de diretórios é feita por meio de `'../'`;
- A função `header` emite um cabeçalho http na página e, combinado com o usuário de `"location: url"`, faz o redirecionamento da página imediatamente.

- A função `exit()` garante em algum caso extremo a interrupção do script;
- A função `header` pode ser usada tanto para validação quanto navegação entre páginas e direcionamentos após envio de formulários ou fluxos como de uma compra em um *e-commerce*, por exemplo;
- A utilização de `(int)` no início do recebimento da variável aplica a conversão obrigatória do valor passado por parâmetro em valor inteiro, o que garante que estaremos validando números e impedindo alguns usos indevidos dos parâmetros aumentando a segurança.
- Só foi tratado o erro (comparação com `false`), pois no caso de tudo estar em conformidade o código deverá seguir normalmente.

O tratamento inicial deste formulário que será submetido poderá ser feito por meio da combinação do script que recebe estas informações e de novas funções que serão introduzidas no arquivo `validadores.php`. Por isso, o trabalho continua.

Passo 7: criar o script `enviar_curriculo.php` recebendo os dados pelo formulário e método POST e realizando primeiras validações baseado em novas funções criadas em `validadores.php`

Arquivo: `enviar_curriculo.php`

```
<?php
// recebendo e validando dados
$dados = $_POST;
include 'funcoes/validadores.php';
$validacao = validar_curriculo($dados);
if ($validacao !== true) {
    header('location: curriculo.php?erro=' .
        $validacao.'&codigo=' . $dados['empresa']);
    exit();
}
// recebendo dados em variáveis
$nome = $dados['nome'];
$email = $dados['email'];
$cod_departamento = $dados['departamento'];
$cod_empresa = $dados['empresa'];
?>
```

Arquivo: funcoes/validadores.php (adicionar função)

```
function validar_curriculo($dados) {
    if( isset($dados['nome']) and isset($dados['email']) ) {
        if( empty($dados['nome']) and empty($dados['email']) ){
            return true;
        } else {
            return 'O preenchimento de nome e email é obrigatório';
        }
    } else {
        return 'O preenchimento de nome e email é obrigatório';
    }
}
```

Por enquanto, estamos apenas garantindo que os campos nome e email estejam presentes em \$dados e que eles não estão vazios. Mais validações em arquivos serão adicionadas posteriormente. O recebimento dos dados e envio por meio do email também serão trabalhados posteriormente.

É importante o entendimento do trabalho em partes em um projeto e saber o momento em que é possível fazer uma pausa. Este momento do projeto, por exemplo, é o típico momento bom para uma pausa.

Já temos uma cara de projeto, pastas organizando bem as coisas e funções sendo criadas facilitando o entendimento e melhorias ou manutenções futuras.



UPLOAD E MANIPULAÇÃO DE ARQUIVOS COM FORMULÁRIOS

Diferentemente dos demais tipos de dados enviados por um formulário HTML, que, em sua essência, é entendido como texto puro, os arquivos exigem alguns cuidados a mais e funções diferentes.

O primeiro ponto a ser sempre observado é que no elemento de formulário que enviará os dados não basta a utilização do elemento `<input>` do tipo arquivo (`type="file"`). É necessário no formulário constar o atributo `enctype="multipart/form-data"`. Sem a utilização do atributo o formulário não envia devidamente os dados, conforme aponta o artigo "Upload de arquivos com o método POST" vinculado ao Manual do PHP ([2018], on-line)².

Por isso, nosso formulário passará a ficar da seguinte forma:

```
<form action="enviar_curriculo.php" method="post" enctype="multipart/form-data">
    <label>Nome:</label>
    <input type="text" name="nome" />
    [...]
```

Além das informações que serão enviadas por meio da variável global `$_POST` é importante ficar atento ao que será enviado pela outra variável global `$_FILES`.

Quadro 1 - Índices contidos em um array de input type=file (upload) para cada arquivo

PARÂMETRO	DESCRIÇÃO
name	O nome original do arquivo na máquina do cliente.
type	O tipo mime do arquivo, se o navegador fornecer essa informação. Um exemplo poderia ser <i>"image/gif"</i> . O tipo mime no entanto não é verificado pelo PHP portanto não considere que esse valor será concedido.
size	O tamanho, em bytes, do arquivo enviado.
tmp_name	O nome temporário com o qual o arquivo enviado foi armazenado no servidor.
error	O código de erro associado a esse upload de arquivo. (se for igual a 0 quer dizer que não houve erro)

Fonte: Manual PHP ([2018], on-line)².

É importante ressaltar que como podem ser enviado um ou mais elementos do tipo arquivo em um formulário os parâmetros demonstrados acima deverão ser acessados por meio do atributo name do próprio elemento.

Por exemplo, se foi criado `<input type="file" name="foto" />` seu nome original será recebido por meio de `$_FILES['foto']['name']` transformando, assim como em `$_POST`, a variável `$_FILES` em um array de elementos.

Com base no requisito do projeto todas as informações serão enviadas por email, mas temos que pensar que o email conterá uma URL para acesso aos documentos, de modo que os arquivos feitos uploads precisarão ser armazenados e, em seguida, disponibilizados para acesso através do email.

Isso quer dizer que precisamos dar nomes aos arquivos e movê-los para uma pasta do projeto pois, após o upload, eles estarão apenas acessíveis ao PHP dentro de uma pasta temporária (sem acesso para emails, por exemplo).

Passo 8: criar uma pasta chamada `curriculos` e criar um novo arquivo em funções chamado `arquivos.php` e uma função chamada `mover_arquivo` que receba os dados `$_FILES` do arquivo, o tipo que pode ser foto ou currículo e realize o upload retornando o nome definido para o arquivo.

Arquivo: `funcoes/arquivos.php`

```

<?php
function mover_arquivo($arquivo, $tipo) {
    $datahora = new \DateTime();
    $datahora_string = $datahora->format('Y-m-d-His');
    $pathinfo = pathinfo($arquivo['name']);
    $novo_nome = $datahora_string . '_' . $tipo;
    $extensao = $pathinfo['extension'];
    $pasta = 'curriculos';
    $novo_nome_completo = $pasta . '/' . $novo_nome . '.' . $extensao;

    if (! file_exists(__DIR__ . '/../' . $pasta)) {
        mkdir($pasta, '0777');
    }

    if(! move_uploaded_file(
        $arquivo['tmp_name'],
        __DIR__ . '/../' . $novo_nome_completo
    )) {
        return false;
    }
    return $novo_nome_completo;
}
?>

```

Passo 9: atualizar o script `enviar_curriculo.php` de modo que faça inicialmente uma validação em arquivos (como feito anteriormente para dados) e, em seguida, que realize as duas chamadas para os arquivos de foto e de currículo.

Arquivo: `enviar_curriculo.php` (continuação)

```

// validando arquivos
$arquivos = $_FILES;
$valdacao = validar_curriculo_arquivos($arquivos);
if ($valdacao !== true) {
    header('location: curriculo.php?erro=' .
        $valdacao.'&codigo=' . $dados['empresa']);
    exit();
}
// realizando o upload
include 'funcoes/arquivos.php';
$foto = mover_arquivo($arquivos['foto'], 'foto');
if ($foto === false) {
    $mensagem_erro = "Erro ao enviar foto";
    header('location: curriculo.php?erro=' .
        $mensagem_erro . '&codigo=' . $dados['empresa']);
    exit();
}

```



```
$curriculo = mover_arquivo($arquivos['curriculo'],
'curriculo');
if ($curriculo === false) {
    $mensagem_erro = "Erro ao enviar currículo";
    header('location: curriculo.php?erro='.
        $mensagem_erro . '&codigo=' . $dados['empresa']);
    exit();
}
```

Arquivo: validadores.php (nova função)

```
function validar_curriculo_arquivos($arquivos) {
    if (isset($arquivos['foto'])) {
        if ($arquivos['foto']['error']!=0) {
            return "Houve um erro no envio da foto pelo formulário";
        }
        if (! in_array($arquivos['foto']['type'], array(
            'image/jpeg', 'image/png'
        ))) {
            return "A foto enviada deve ser compatível com image/jpeg
ou image/png";
        }
    } else {
        return "O envio da foto (anexo) é obrigatória";
    }

    if (isset($arquivos['curriculo'])) {
        if ($arquivos['foto']['error'] != 0) {
            return "Houve um erro no envio do currículo pelo formulá-
rio";
        }
        if (! in_array($arquivos['curriculo']['type'], array(
            'application/msword', 'application/pdf'
        ))) {
            return "O currículo enviado deve ser compatível com
application/msword e application/pdf";
        }
    } else {
        return "O envio do currículo (anexo) é obrigatório";
    }

    return true;
}
```

Apesar do código ter ficado bem mais complexo agora, ainda estamos com ele bem dividido e é mais fácil de entender seus objetivos. Vamos aos destaques:

- Na função `mover_arquivo` acabou sendo usado um método do PHP que retorna a data e a hora para que os arquivos fiquem salvos seguindo este padrão 2017-10-01-234020_foto.jpg e 2017-10-01-234020_curriculo.pdf (ou doc);
- Não realizar o upload do arquivo com seu nome original é uma técnica necessária pois é totalmente inseguro mover um arquivo exatamente como proposto pelo upload via formulário;
- Utilizar a data e hora com os segundos como nome do arquivo garante (em tese) que os nomes dos arquivos nunca irão se repetir. Para sistemas com maior volume de uploads será necessário ainda criar um valor randômico ao final para garantir que os mesmos não se repitam;
- A função de validação realiza três verificações: se o arquivo foi enviado, se o arquivo foi enviado sem erro e se o tipo do arquivo (mime type) condiz com o desejado. Como exemplo, permite o uso de jpeg e png para imagens e de word e pdf para currículos;
- O uso das funções possibilitou o script de `enviar_curriculo` ficar bem limpo e organizado deixando para as funções a concentração de todas as demais códigos e regras de negócio.



ENVIO DE EMAIL

No PHP, a função **mail** é a função nativa responsável por enviar emails diretamente pelo código. Esta função usará o servidor de envio de emails padrão habilitado para o PHP que, muitas vezes, em nossas máquinas locais, não estão instalados e, por isso, a função mail geralmente funciona apenas em ambientes de produção.

Como estaremos tratando de um ambiente de desenvolvimento, geralmente não teremos um servidor de email habilitado para enviar mensagens ainda que sejam apenas de testes. Por isso, se você instalou o XAMPP a primeira coisa que deve fazer é alterar duas configurações no arquivo `php.ini` localizado em `C:/XAMPP/php/php.ini`:

```
sendmail_from = meuemail@meuservidor.com.br  
sendmail_path = "\"C:\xampp\sendmail\sendmail.exe\" -t"
```

Após esta alteração é importante reiniciar o serviço do XAMPP (e caso não saiba fazer isso e ache mais fácil pode reiniciar sua máquina).

Os parâmetros recebidos pela função mail serão demonstrados no quadro a seguir em sua ordem exata:

Quadro 2 - Parâmetros para Envio de Email

PARÂMETRO	DESCRIÇÃO	EXEMPLO
Destinatário	Destinatário ou destinatários do email conforme padrão de email	ead@eduardobona.com.br ead@gb.com.br, abc@gb.com.br
Assunto	Título do email que será enviado	Meu Assunto
Mensagem	Texto com a mensagem a ser enviada. É importante se atentar que a mensagem pode ser apenas um texto puro (limpo) como pode ser um HTML e, neste caso, mais parâmetros deverão ser informados.	Minha Mensagem de texto com quebra de linhas.
Cabeçalhos Adicionais (opcional)	É a composição Header do email e poderá conter desde informações sobre o formato do documento como se é um HTML, bem como adicionar as cópias e cópias ocultas do email.	Cc: ead@eduardobona.com.br Bcc: contato@eduardobona.com.br
Parâmetros Adicionais (opcional)	Parâmetros utilizados de acordo com configuração do PHP.	-

Fonte: Manual do PHP ([2018], on-line)².

Partindo agora para nosso estudo de caso, tendo já as informações acerca dos dados do currículo e dos documentos, já poderemos proceder com o resultado final do projeto que é a submissão do email.

Lembrando que são dois emails que devem ser enviados, sendo o primeiro para quem cadastrou o currículo, agradecendo o envio, e o segundo para os emails configurados para receberem as informações de acordo com empresa e departamento.

Passo 9: criar dentro da pasta funções um arquivo chamado emails.php com duas funções, sendo a primeira denominada agradecer_cadastro e que recebe o nome e o email da pessoa e confirma o cadastro, e a segunda chamada enviar_curriculo_email, que recebe todos os parâmetros necessários para enviar o email.

Arquivo: funcoes/emails.php

```
function agradecer_contato ($nome, $email) {
    $assunto = "GB: Agradecemos seu cadastro";
    $mensagem = "
```

```

        Olá, este email está sendo enviado para
        confirmar o recebimento do seu cadastro.
        Agradecemos seu contato.
    ";
    return mail($email, $assunto, $mensagem);
}

```

O primeiro email é mais simples, pois não demanda em tese HTML. Contudo, atualmente, todos os clientes e principalmente seus respectivos departamentos de marketing optam pelo uso de HTML permitindo, assim, mensagens customizadas e com estilo.

Para o envio de HTML, no entanto, tratamentos adicionais são necessários e fique muito atento, pois nem tudo é aceito pelos clientes de email.

Arquivo: `funcoes/emails.php` (nova função)

```

function enviar_curriculo_email ($dados, $arquivos) {
    $assunto = "GB: Novo Currículo Cadastrado";
    $mensagem = "
        <strong>Dados: <br />
        Nome: %s / Email: %s <br /><br /></strong>
    ";
    $mensagem = sprintf($mensagem, $dados['nome'],
    $dados['email']);
    $mensagem .= "
        Foto: <a href='http://localhost/%s'>Ver Foto</a><br />
        Currículo: <a href='http://localhost/%s'>Ver Currículo</a>
    ";
    $empresa = include __DIR__ . '/../dados/empresa_departamento.
    php';
    $departamentos = $empresa[$dados['cod_empresa']]['departamen-
    tos'];
    $destinatario = $departamentos[$dados['cod_departamento']];

    $headers = 'MIME-Version: 1.0' . "\r\n";
    $headers .= 'Content-type: text/html; charset=iso-8859-1' .
    "\r\n";
    return mail($destinatario, $assunto, $mensagem, $headers);
}

```

Sobre as duas funções de email, podemos considerar que:

A primeira função é bem simples, pois só faz um disparo de email com poucas informações;

A segunda função já é mais complexa e utiliza diversas técnicas para construção e envio do email;

Na construção do email é utilizado concatenação constante da variável \$mensagem e também é utilizado a função sprintf, que substituiu o valor %s pela variável informada, permitindo assim que nosso código fique mais limpo e não se misture tanto variáveis PHP com códigos HTML;

Foi necessário criar um link dentro do email para uma URL no próprio servidor e ela funcionará para você, pois somente sua máquina (ambiente local) conhece o link <<http://localhost>>, sendo necessário alterar esta URL, caso esteja sendo colocado em produção;

É possível anexar arquivos por email, mas normalmente o que se faz mesmo é enviar os links mantendo os arquivos dentro do servidor que recebeu os cadastros;

Enviar apenas por email estes dados pode não ser a melhor ideia, pois o servidor de email nem sempre está funcionando e, por isso, optamos por manter os arquivos dentro do próprio servidor;

Ainda assim, quando você tiver conhecimento de banco de dados recomendo fortemente que estes dados estejam também salvos em um banco de dados não dependendo tanto assim de emails.

Passo 10: uma vez estas duas funções funcionando, é necessário atualizar o script de enviar_curriculo com a utilização destas funções e, ao final, redirecionar para a tela final de agradecimento e confirmação pelo envio, chamado de curriculo_cadastrado.php e que possui somente uma mensagem de agradecimento.

Arquivo: enviar_curriculo.php (continuação)

```
// enviando email e concluindo cadastro
include 'funcoes/emails.php';
agradecer_contato($nome, $email);
enviar_curriculo_email(
    array('nome' => $nome, 'email' => $email),
    array('foto' => $foto, 'curriculo' => $curriculo)
);
header('location: curriculo_cadastrado.php');
exit();
?>
Arquivo: curriculo_cadastrado.php
<?php
    include 'layout/cabecalho.php';
?>
<h3>Obrigado pelo seu cadastro.</h3>
```

```
<h3>Esperamos em breve poder entrar em contato.</h3>
<?php
    include 'layout/rodape.php';
?>
```

E, por fim, teremos então a chamada das duas funções sendo realizadas e a última tela confirmando todo o fluxo realizado.

Se você for analisar, um requisito simples dedicou um material bem extenso de pastas e códigos devidamente organizados para que resultasse em um projeto ainda que muito pequeno.

Por isso, se você imaginar que em uma demanda tão pequena já teremos um código bem extenso, a sua preocupação com organização deve ser redobrada a partir de agora, caso ainda não tenha começado a se preparar pelo que virá pela frente.

SAIBA MAIS



O envio de emails é um recurso básico das aplicações, utilizado para avisos ou comunicações, confirmações, lembretes e propaganda. O PHP e sua função até simples de email permite, de maneira mais fácil, que você envie uma mensagem. Emails mais complexos começam a ficar mais difíceis com estas funções, apesar de ser possível também. Caso deseje utilizar recursos mais avançados de email como autenticar em seu servidor Gmail ou da empresa, enviar anexos, entre outras possibilidades recomendo o estudo na biblioteca PHP-MAILER, disponível em: <<https://github.com/PHPMailer/PHPMailer>>.

Fonte: o autor.

Pronto!

O resultado atingido agora e concluído foram três telas mais um script fazendo todo o fluxo exigido no requisito. Criamos diversas funções e temos um mini-projeto do começo ao fim.

Agora é necessário rever todo o material e implementar quantas vezes forem necessárias para "afinar" a parte prática e de conhecimentos básicos para um desenvolvedor backend.

CONSIDERAÇÕES FINAIS

Com a prática neste estudo de caso e sua aplicação, uma pergunta que você poderá se fazer (e me fazer) após reler o material e analisá-lo é que diversas melhorias ainda podem ser feitas no projeto. Se isso ainda lhe causar dúvidas eu vou facilitar respondendo que você está coberto de razão.

O objetivo com este estudo de caso foi orientá-lo sobre a perspectiva de um desenvolvedor que inicia um projeto a partir de um requisito escrito e finaliza entregando o projeto do início ao fim, passando por técnicas de projeto e recursos importantes do PHP.

Ainda assim, nosso código pode melhorar muito.

Vimos a utilização de inclusões de arquivos, navegação, envio e validação de formulários bem como manipulação de uploads e envio de emails. Em cada um destes exemplos deixei margem para melhorias futuras.

Na primeira página, por exemplo, onde listo as empresas através do array criado de empresa e departamento você pode tornar o código dinâmico. No formulário de currículos, você poderá pesquisar e implementar técnicas do HTML5 ou da biblioteca javascript jQuery para ajudar na validação do formulário. Além disso, no email você poderá enviar os documentos por anexo.

A prática irá levar você ao conhecimento da linguagem, mas somente o seu instinto contínuo em busca de melhorar sempre o código vai fazer com que você passe a se tornar um bom desenvolvedor. Posso garantir que de todos os livros que puder ler, a maior parte deles terá um impacto de, no máximo, 10% em sua vida profissional.

Vamos nos lembrar da analogia com os esportes. O treinamento é necessário e praticar é importante. Recursos do PHP que você não conheça ou não tenha tido experiência devem ser estudados para que um dia, quando profissionalmente for exigido, esteja preparado. A experiência por outro lado está atrelada com a aplicação do seu conhecimento teórico e prático, é o jogo de verdade e pode envolver diversos fatores como pressão, prazo, trabalho em equipe, qualidade, entre outros.

ATIVIDADES



1. Quais as duas funções no PHP que nos permitem incluir outros scripts aos códigos e com isso facilitar a organização de códigos em pedaços menores?
2. Qual deveria ser o trecho de um código PHP desenvolvido para redirecionar imediatamente o script para a URL <http://www.unicesumar.edu.br>?
3. Qual a diferença existente nas funções PHP `isset` e `empty`?
4. Um formulário que contenha um arquivo ao ser submetido para um script PHP é recebido pela variável `$_FILES`. Para cada input, esta variável recebe um array com 5 informações sobre o arquivo em questão. Cite pelo menos 3 informações presentes neste arquivo.
5. Após submetido um upload para um script PHP é necessário movê-lo da pasta temporária em que se encontra para uma pasta da sua aplicação. Qual função no PHP é necessária para que isso seja possível?
6. Qual código seria necessário escrever para enviar um email para ead@educar-dobona.com.br com o assunto do email sendo "Oi professor" e o conteúdo do email "Bom dia Professor. Fiz a atividade 6!"?



Vários autores de livros técnicos (ou não) gostam de se utilizar do conceito do "não confie em ninguém, nem em sua própria sombra". Não à toa, este tipo de abordagem é posto à prova em filmes, seriados, novelas e note que até em livros técnicos como o nosso.

Brechas de segurança concedem à invasores mal-intencionados a oportunidade perfeita para atacar suas informações.

Vamos pensar em uma casa, dentro de um bairro residencial distante do centro da cidade e com um índice relativamente alto de assaltos, você a deixaria sem elementos de segurança? Por exemplo, um portão eletrônico, câmeras, interfone, alarme, cerca elétrica, grades nos vidros e tudo mais que for necessário? Provavelmente não.

Entenda que cada tipo de sistema exige o nível de segurança mais apropriado. Sites e sistemas web que realizam vendas são extremamente visados devido às informações que eles detêm, mas hoje em dia todo cuidado é pouco. Sites de partidos políticos, entidades públicas e de personalidades costumam ser objeto comum de ataques.

As invasões podem ocorrer por diversas brechas deixadas, mas a que mais importa neste momento para nós é a feita pela aplicação de entradas maliciosas de dados. Por isso, muitos autores gostam de classificar o processo de tratamento destes dados em três camadas, com o processo de sanitizar, validar e escapar bem as informações.

O processo de sanitizar visa garantir que mesmo um dado entrando de maneira desconhecida, ele seja filtrado como fazemos com a água. Não sabemos o que vem da caixa da água e a qualidade ali presente, mas uma vez passando pelo filtro, temos confiança de que aquilo é potável. Validar, por outro lado, tenta garantir que o que está sendo enviado está dentro do que esperávamos, como um campo que espera um número inteiro e não pode receber um texto. Este processo fornece feedback para quem está tentando enviar estes dados e geralmente pede o preenchimento adequado da informação.

Por fim, dados que são fornecidos de maneira dinâmica também podem ser escapados para nossa aplicação. Deste modo, ao saber que deve ser impresso em tela (saída de código) um texto sem HTML, eu posso remover qualquer coisa diferente de um texto comum desta saída, criando mais uma camada de segurança na aplicação.

Questões como estas são muito importantes e você precisa estar atualizado e estudar muito o assunto. Para te ajudar, em seu material complementar está presente uma série sobre segurança web.

Assim como hoje em dia é muito popular uso de cerca elétrica, câmeras e alarme (três camadas), se atente também ao que sua aplicação demanda e cuide muito bem dela, pois na internet tudo está à disposição de todos, e indivíduos mal-intencionados são mais comuns que assaltantes nas ruas por aí.

Fonte: o autor.





LIVRO

PHP Moderno: Novos recursos e boas práticas

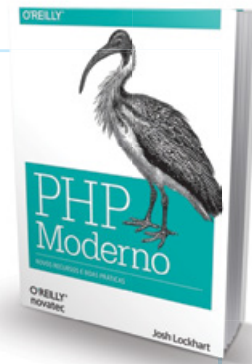
Josh Lockhart

Editora: O'REILLY / Novatec

Sinopse: O PHP está passando por um renascimento, embora possa ser difícil perceber, com tantos tutoriais PHP online desatualizados. Com este guia prático, você verá como o PHP se tornou uma linguagem cheia de recursos e madura, orientada a objetos, com namespaces e uma coleção crescente de bibliotecas de componentes reutilizáveis.

Comentário: Concluindo esta primeira imersão em fundamentos para programação backend com foco em aplicações em PHP, é importante conhecer questões além do PHP e das referências de sua linguagem.

Neste livro alguns conceitos muito legais são abordados, principalmente o foco nos novos recursos, nos padrões de projeto mais conhecidos e nas boas práticas. O desenvolvedor PHP Moderno não conhece só a linguagem, mas também questões relacionadas aos projetos, aos servidores e às ferramentas disponíveis.



NA WEB

Série de vídeos sobre segurança pela empresa de treinamentos Visie, no comando do instrutor Élcio Ferreira, que é, sem dúvidas, um dos programadores mais influentes da web 2.0 com o blog tableless.com.br

Acesse: <<https://www.youtube.com/watch?v=kh7J8VHvRGs>>.

REFERÊNCIAS

MANUAL DO PHP. **Função include**. Disponível em: <http://php.net/manual/pt_BR/function.include.php>. Acesso em: 01 dez. 2017.

MANUAL DO PHP. **Funções definidas pelo usuário**. Disponível em: <https://secure.php.net/manual/pt_BR/functions.user-defined.php>. Acesso em: 01 dez. 2017.

MANUAL DO PHP. **Upload de arquivos com o método POST**. Disponível em: <http://php.net/manual/pt_BR/features.file-upload.post-method.php>. Acesso em: 01 dez. 2017.

MANUAL DO PHP. **Função mail**. Disponível em: <http://php.net/manual/pt_BR/function.mail.php>. Acesso em: 01 dez. 2017.

GITHUB - EDUARDO BONA. **Livro Backend I** - 2017. Disponível em: <<https://github.com/eduardobona/livro-backend1-2017>>. Acesso em: 20 dez. 2017.

REFERÊNCIA ON-LINE

¹ Em: <http://php.net/manual/pt_BR/function.include.php>. Acesso em: 15 fev. 2018.

² Em: <http://php.net/manual/pt_BR/features.file-upload.post-method.php>. Acesso em: 15 fev. 2018.



1. Include e Require.

O funcionamento e características para ambos são os mesmos. O que diferencia um do outro basicamente é o tratamento dos erros quando o arquivo não existe, por exemplo. A documentação oficial descreve perfeitamente a semelhança e a pequena diferença entre os dois recursos (online).

2.

```
header('location: http://www.unicesumar.edu.br');  
exit; // opcional para garantir o término da execução
```

3. A função `isset` verifica se determinada variável (ou no caso de arrays se determinada posição do array) foi setada, ou seja, "is set", não determinando ali se possui valor ou não. No caso da função `empty` ela faz diversas validações para saber se o valor presente na informação passada está vazia, em branco, nula ou é igual a zero, por exemplo. É possível entender ainda melhor a diferença das duas através da documentação oficial presente em `isset` (online) e `empty` (online).

4.

- 4.1 name com o nome original do arquivo;
- 4.2 type com o tipo mime do arquivo
- 4.3 size com o tamanho em bytes do arquivo
- 4.4 tmp_name com o nome temporário do arquivo (enviado para pasta temporária)
- 4.5 error com o código do erro associado ao upload em caso de falhas

5.

```
move_uploaded_file($nome_arquivo, $destino)
```

Verifica se o arquivo designado por `$nome_arquivo` é um arquivo de upload válido (que tenha sido enviado pelo mecanismo PHP de envio por POST HTTP). Se o arquivo for válido, ele será movido para o nome de arquivo dado por `$destino` conforme documentação oficial (online).

6.

```
<?php  
  
$destinatario = "ead@eduardobona.com.br";  
$assunto = "Oi professor";  
$conteudo = "Bom dia Professor. Fiz a atividade 6!";  
  
mail($destinatario, $assunto, $conteudo);
```



CONCLUSÃO

Primeiramente, gostaria de dar os parabéns para você, que chegou ao término destes estudos e espero que tenha aproveitado ao máximo tudo que foi demonstrado ao longo destas cinco unidades.

Vale o famoso aviso que ainda não chegamos ao fim de nossos estudos!

Isso porque um desenvolvedor – e principalmente o desenvolvedor web – nunca para de estudar para que se aprofundem os conhecimentos adquiridos e também para novas tecnologias, pois o mercado muda muito, e muda rápido.

Por meio do PHP, você poderá desenvolver sites simples institucionais, bem como portais com um volume maior de acessos. Também poderá desenvolver sistemas internos simples ou sistemas mais complexos com integrações. Aliás, o Facebook é desenvolvido parte em PHP, sabia? Por isso, aplicações para startups que são estas empresas de tecnologia que criam soluções inovadoras podem na maioria dos casos utilizar o PHP.

Enfim, o PHP pode ser para você o que aquela mala de ferramentas é para o marceneiro: essencial. Vocês agora trabalham juntos e você precisa usar cada um dos itens desta caixa de ferramentas da forma correta, na proporção correta e no lugar correto. Quando algo não estiver mais lhe atendendo é hora de trocar e isso basicamente significa estudar mais para desenvolvedores.

O ideal e o esperado de qualquer profissional, após aprender novas tecnologias, é que se aplique de forma prática e isso poderá ser realizado com HTML e PHP em sua empresa ou em seus estudos particulares mesmo. Recomendo que comece a pensar em um projeto de cunho pessoal ou até mesmo para ONGs (sem fins lucrativos) e que, então, isto passe a ser uma meta sua.

Com certeza o conhecimento adquirido aqui lhe ajudará nos primeiros passos e, ao longo do curso e de seus próximos estudos, seu projeto sairá definitivamente do papel até sua disponibilização.

Se assim for, quero ser o primeiro a conhecer seu projeto!

Nunca desista de seus sonhos, de seus projetos e mantenha-se atualizado, sempre!





ANOTAÇÕES

